
W_x
OSF/Motif Text Editor
Version 4.0

User's Guide

Nova Software Labs

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:



Nova Software Labs

8, rue de Hanovre

75002 Paris, France

Tel: +33 1 58 18 61 61

Fax: +33 1 58 18 61 60

email: sales@NovaSoftwareLabs.com or sales@nsl.fr.

Web: <http://www.NovaSoftwareLabs.com> or <http://www.nsl.fr>.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker, XDrawMaker, XnslDrw are registered trademarks of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: WxRefMan v. 7.1.1.

Document date: November 29, 2002

Acknowledgments

The following people have contributed to the development of the XFaceMaker suite:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

Preface	viii
Getting Started	1
Launching Wx	1
Opening Wx Windows	2
Using the Manager Box	3
Opening Files	4
Selecting Files	4
Showing/Hiding Windows	5
Saving Files	5
The Drag-and-Drop Feature	5
The Menu Bar Commands	6
Using Wx Windows	7
Opening a Wx Window	7
The Client Area	8
The Menu Bar	10
The Icon Bar	11
The Message Zone	14
The Title Bar	14
A Summary of Wx Menu Commands	14
Editing Techniques	23
Opening Files	23
Changing the Directory	26
Entering Text	28
Selecting Fonts	29
Moving in Your File	29
Selecting Text	30
Cutting, Copying, and Pasting	32
The Kill Stack	33
Undo	35
Searching and Replacing	35
Searching for Text	36
Replacing Text	36
Searching by Regular Expression	38
Saving Files	41

The Save As... Command 42
Inserting Files 43
Closing Windows 43
Wx Modes 44
Using Macros 45
Compiling a Buffer 46
Execute UNIX Command... 47
Using the Mini Buffer 48
Help 48
The Help Index 50
Quitting Wx 51
Keyboard-Only Commands 52
Keyboard Command Symbols 52
Delete-Text Commands 52
Metacommands 53
The Load Tags Command 53
Reload File 54
The Load Macro File Command 54
List of Keyboard-Only Commands 55
Summary of All Wx Commands 57
Manipulating Files 57
Inserting Characters 57
Deleting, Copying and Clearing Text 58
Keyboard Macros 59
Matching Parentheses 59
Moving About Text 60
Searching and Replacing Text 61
Selecting Text With The Mouse 61
Scrolling The Window 62
Commands on Words 62
Window Manipulation 63
Other Commands 63
Wx Modes 65
Defining Modes 65

Keywords 66
Rules for Alias Definition 68
Rules for Keytable Definition 69
Format for Mode Definitions 70
Wx Keyboard Macros 71
Creating Macros 71
Registering a Macro Script 72
Executing and Editing Macros 72
Loading and Saving Macros 73
Using FACE in Wx Scripts 73
Keywords in FACE 74
Special Characters 74
Comments 74
Arithmetic Expressions 74
Include Files 75
Basic objects 75
Object Types 75
Constants 77
Variables 77
Defined Constants 79
X Properties 80
Type Casters 81
Declaring New Type Casters 81
Type Conversion 81
Statements 82
Control Structures 82
Built-in Functions 82
Wx Editing Functions 83
Definition of Functions in the FACE Language 83
Floating-Point Variables 85
A Macro Example. 85
Configuring Wx 93
Creating Your Own .wxrc File 93
Resources 93

Standard Motif Widgets in Wx	94
The Wx Window Widget Hierarchy	94
The XnslWix Widget Resources	95
The XnslWix Widget translations	97
The wxd Program Resources	98
Example of a .Xdefaults File	100
Command Line Options	102
Environment variable	104
The TMPDIR environment variable	104
The NSLHOME environment variable	104
Configuring .wxrc	104
Keywords	104
Rules for Mode and Macro Definitions	105
Rules for Menu Definition	105
Key Combinations for Menu Items	106
What Cannot Be Configured in .wxrc	107
Internationalization	107
Wx and Special ISO Latin-1 Characters	108
ISO Keyboard	108
wxkeys and the insert-accent Action	108
Entering Octal Codes	108
The Wx Commands	109
FACE Typecasting Functions	117
Built-in Face Functions	119
The Wx Functions	147
Undo Grouped Commands	229
Glossary	231

The Wx Manager Box 1
The Wx Rectangle 1
A Wx Window 2
The Wx Manager Box 3
A typical Wx window just after opening 7
A Wx Window Split into Two Panels 8
The File Menu 15
The Edit Menu 16
The Search Menu 17
The Window Menu 18
The Modes Menu 19
The Macro Menu 20
The Misc (Miscellaneous) Menu 21
The Help Menu 22
The File Selector 24
The "Changes not saved" message 25
The Directory File Selector 26
A Wx window with newly entered text. 28
The Wx Font Selector 29
The Wx Clipboard showing the kill stack 33
The Search window 36
The Replace window. 37
Alert message during a *Save As* 41
The Macros sub-menu 45
The Compilation Diagnostics panel. 46
The Icon help box 49
The help window for the write-file command 50
The “file modified” warning 51
The Tags Selection Box 54

Preface

Wx is an easy-to-use text editor that takes full advantage of the X window system: mouse and keyboard use, over 90 editing functions, macros, powerful language modes, and icons for easy interaction. In addition, **Wx** generates editable FACE language scripts for the macros you create.

This User's Guide explains how to use the **Wx** editor. The Guide is organized as follows:

Getting started in Wx -	Chapter 1
Wx windows and menus -	Chapter 2
Editing techniques -	Chapter 3
Using language modes -	Chapter 4
Creating and editing macros -	Chapter 5
Configuring Wx -	Chapter 6

The first three chapters are an introduction to **Wx**. You should find them useful if you are new to the world of graphic editors or UNIX. The final three chapters describe the use of modes and macros in **Wx**, and explain how to configure **Wx** to suit your preferences. There are several appendices listing functions, commands and other useful information.

Features of the Wx Editor

The main features of the **Wx** editor are the following:

- **Wx windows**

Each **Wx** window has a full set of editing commands available in the Menu Bar, as well as comprehensive on-line Help. In addition, each window has icons for easy access to commonly used features of **Wx**.

- **Manager Box**

The **Wx** Manager Box is designed to give you easy access to your files and the common editing features of **Wx**. Icons allow you to open, save, and show/hide your files. The List Window displays all the files that are currently open. The menu bar gives you access to the Script window for macro editing, and also to the Kill Stack.

- **Predefined language modes**

Wx comes with a set of language modes—working environments that make your editing task easier, whether you are programming, formatting, or editing text.

- **Macros**

Macro creation is simple in **Wx**. In addition, **Wx** generates editable FACE language scripts of the macros you create.

- **Configurability**

Wx's client-server architecture makes it easy for the user to change many aspects of **Wx**, including the look and feel of the application, menus, modes, and macros.

Typesetting Conventions

Type style	Used for
<code>typewriter</code>	Anything you should type as it appears, file names, program contents.
bold	Command names, special and manual references.
<i>italic</i>	Mathematical expressions.
<i>slant</i>	Text editing notions, menu items.
box	Push buttons.
sanserif	Key combinations.

Mouse Button Conventions

This manual uses the following terminology for mouse operations:

Press	Hold down a mouse button.
Release	Release a mouse button after it has been pressed.
Click	Quickly press and release a mouse button without moving the mouse.
Double Click	Click a mouse button twice, quickly.
Triple Click	Click a mouse button three times, quickly.
Shift Click	Click a mouse button, holding the Shift key down.
Drag	Move the mouse while pressing a mouse button.
Button 1: Select	Left mouse button. Used to select text.
Button 2:	Middle mouse button. Used to paste selected text.
Button 3: Custom	Right mouse button. Used to scroll the buffer.

Motif Style Guide Recommendations in **Wx**

Wx uses the standard Motif widget set and the NSL text widget **XnsIWix**. Widget behavior follows the Motif Style Guide recommendations. For example, when a menu is open, the arrow keys can be used to move through the menu items.

However, **Wx** does not follow all the Motif Style Guide recommendations for keys because the **XnsIWix** widget has more commands than can be covered by the keys available on a standard keyboard.

For example, the **osfCancel** key, which usually binds to the **Esc** key, is used to cancel the current operations. In **Wx** however, the **Esc** key is used as a prefix for a set of commands. For example, **Esc s** is used to initiate a text search. (The key combination cancels the current dialog box.) This arrangement allows you to use **Wx** exclusively from the keyboard, if you wish.

You can choose to run a version of **Wx** that follows the Motif Style Guide recommendations, by setting the **motifRecommendations** resource to “true” in your **.Xdefaults** file, or by launching **wxd** with the option **-mr**.

CHAPTER 1: Getting Started

This chapter gets you started using the **Wx** editor.

1.1 Launching Wx

To launch Wx, the X server and a window manager such as **mwm** or **twm** must be running. You can choose to display either the Wx Manager Box or the Wx Rectangle.

- To display the Wx Manager Box enter the command:

```
$ wxd -managerbox
```

This launches the server process and opens the **Wx** Manager Box on your screen. The Manager Box is described in section 1.3.

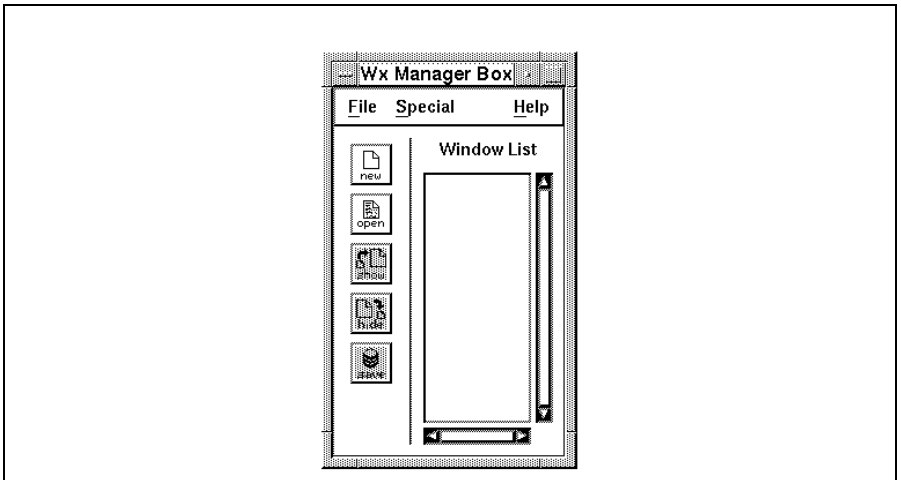


Figure 1.1: The Wx Manager Box

- To display the Wx Rectangle, enter the command:

```
$ wxd
```

This launches the server process and displays the Wx Rectangle on your screen. Click on the Rectangle to open an editing window.

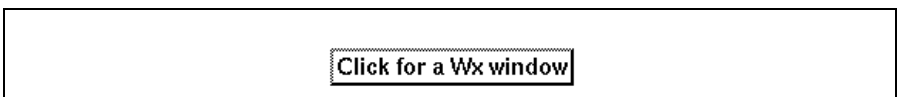


Figure 1.2: The Wx Rectangle

1.2 Opening Wx Windows

Once the server is running, you can open editing windows in any of the following ways:

-  In the Manager Box, click in the **new** icon to open an editing window.
-  In the Manager Box, click in the **open** icon to load an existing file.
- Click in the **Wx** rectangle with the mouse **Select** button. This opens a new editing window. To load an existing file into this window, select the Open command in the File menu and specify the name of the file in the File Selector box.
- Enter the following command to open a new editing window.
`$ wx`
- If you know the name of the file you want to edit, e. g. `myfile`, enter:
`$ wx myfile`

This opens an editing window with the specified file loaded.

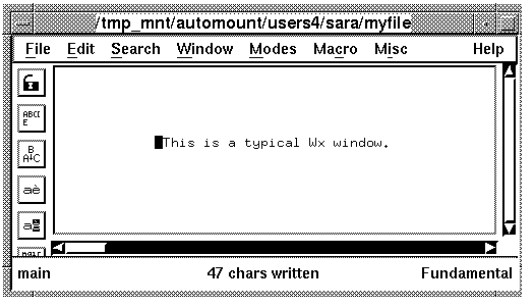


Figure 1.3: A Wx Window

1.3 Using the Manager Box

The **Wx** Manager Box gives you easy access to the files you wish to edit. The Manager Box can be opened in two ways:

- Launch `wxd` with the command `wxd -managerbox`,
- Set the `useManagerBox` resource in your `.Xdefaults` file to `True`, then launch `wxd` with the command `wxd`.

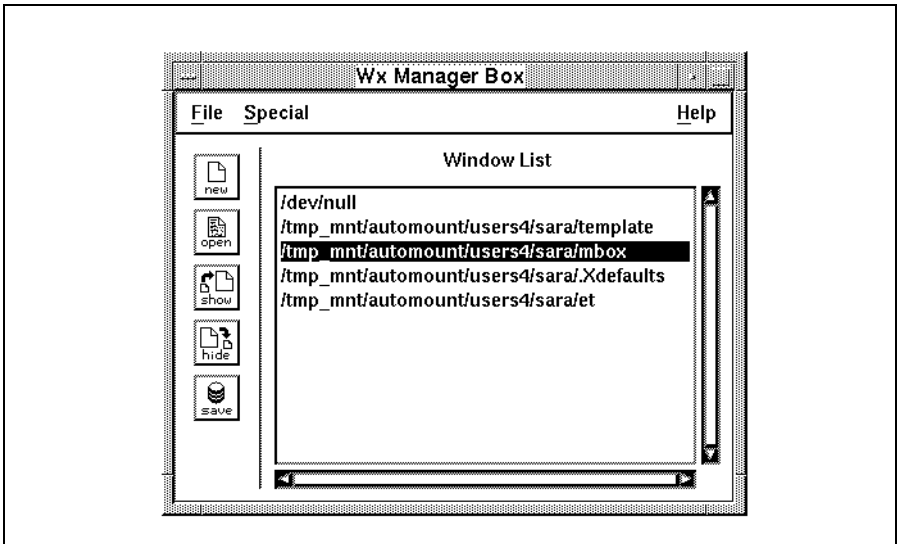


Figure 1.4: The Wx Manager Box

The Manager Box has a menu bar, a set of icons, and a list of open Wx windows. Some of these features are also available in the **Wx** window menus.

- The **Menu Bar** gives you access to the Kill Stack and the Script editing window, as well as to on-line Help.
- The **icons** enable you to open new or existing files, show/hide or save your files. When the Manager Box first appears on the screen, only the **new** and **open** icons are sensitive. The other icons are dimmed until you have selected a file from the Window List.
- The **Window List** displays the file names of all the **Wx** windows that are currently open. Unnamed files are listed as `/dev/null`.

1.3.1 Opening Files

You can open new or existing files using the Manager Box icons or by selecting the appropriate command in the menu of the **Wx** window. To open files using the icons, do the following.



New files Click on the new icon to open a new file. A **Wx** window will appear on your screen, ready for input. (See Chapter 2 on using the **Wx** window). Until you name the file, it appears in the Window List as **/dev/null**.



Existing files Click on the open icon to open an existing file. A **Wx** window will appear with a File Selector box. Select the file you wish to edit, press OK, and **Wx** will load the file into the window. The file's name will appear in the Window List.

1.3.2 Selecting Files

You can open, show/hide, or save files using the **Wx** icons. To do this, you must first *select* one or more files from the Window List. When a file is selected, all the Manager Box icons are sensitive.

Selecting a single file Click on the file name. The name will appear in reverse video.

Selecting several files There are three methods for selecting more than one file at a time.

1. *Select contiguous files* by clicking on any file name and dragging up or down in the Window List. The selected file names will appear in reverse video.
2. *Extend a selection* by pressing the **Shift** key as you click or drag in the Window List. This method selects all contiguous files.
3. *Select non-contiguous files* by pressing the **Control** key and clicking on any files you wish to select.

Deselecting files Press the Control key and click or drag to deselect the files in the Window List.

1.3.3 Showing/Hiding Windows

You can iconify or hide your **Wx** file windows, or bring a window to the foreground of your screen using the Show and Hide icons. To do so, first select the file(s) from the Window List, then click on the appropriate icon. You can also re-display a hidden window by double-clicking on the file's name in the Window List.



The show icon.

- brings the selected **Wx** window to the foreground of your screen. Note that only one window can be brought to the foreground at a time.
- re-opens an iconified window.



The hide icon.

- iconifies or hides the selected **Wx** window (depending on the window manager you are using). Note that you can still save selected files, even when they are hidden.

1.3.4 Saving Files



The save icon.

To save a file or files, select the file(s) in the Window List, then click on the **save** icon in the Manager Box. You can save files even if they have been hidden or iconified. If the selected file is unnamed when you attempt to save it, a File Selector box will be popped up so that you can enter a file name.

Remember that you can also use the **Save** or **Save As...** commands in the **Wx** window menu to save the contents of a window.

1.3.5 The Drag-and-Drop Feature

If your system uses Motif 1.2, you can use the Motif Drag-and-Drop feature in the **Wx** Manager Box. Drag-and-Drop works only on single files. To use Drag-and-Drop:

1. Press mouse button 2 on a file name in the Window List. The file will be selected and the Motif Drag-and-Drop icon will appear.
2. Drag the icon to the appropriate **Wx** icon (show, hide, or save), and release the mouse button.

1.3.6 The Menu Bar Commands

The menu bar in the **Wx** Manager Box contains three menus: ***File***, ***Special***, and ***Help***.

The ***File*** menu contains two commands.

- ***Save All*** Selecting this command saves all **Wx** files that are currently being edited, even if they are hidden or iconified.
- ***Quit*** This command exits the **Wx** editor. If you have not saved your work, an alert box will pop up to inform you.

The ***Special*** menu contains two commands.

- ***Show Script Window*** This command pops up the Script window for editing macro scripts. The Script Window is described in Section 5.2.
- ***Show Kill Stack*** This command pops up the **Wx** clipboard window. For details on the use of the kill stack, see Section 3.7.

The ***Help*** menu contains the same items as the ***Help*** menu in the **Wx** window: ***On Version***, ***On Icons***, and an ***Index*** of Help topics. For more details, refer to Section 3.19 of this *User's Guide*.

CHAPTER 2: Using Wx Windows

This chapter describes how to use **Wx** windows. A summary of the commands available in the window menus is given at the end of this chapter.

2.1 Opening a Wx Window

As explained in Chapter 1, there are several methods for opening a **Wx** editing window.

- Click in the **new** or **open** icons in the Manager box.
- Click on the Wx Rectangle.
- Enter the command
\$ wx
in your xterm.
- If you know the name of the file you want to edit, e. g. *myfile*, enter the command
\$ wx myfile.

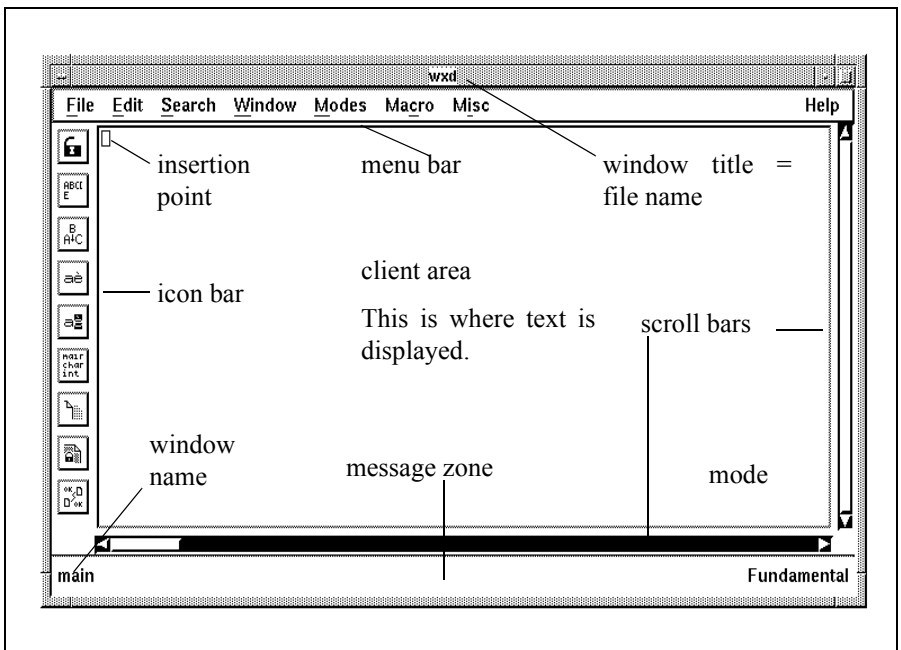


Figure 2.1: A typical Wx window just after opening

It has five main features:

- a *client area* for entering text
- a *menu bar* which gives you access to most of the **Wx** commands
- a set of *icons* to activate some commonly-used **Wx** features
- a *message zone* lists the buffer name, modifications, modes, etc.
- a *title bar* to indicate the name of the current file.

These features of the **Wx** window are described on the following pages.

2.2 The Client Area

The *client area* is where the text buffer is displayed. The text *insertion point* is visualized as a vertical bar (or a small solid rectangle) in the client area: any text you type into the buffer appears at that point. If the text you try to enter does not appear in the window, your **Wx** window probably does not have the focus—slide the mouse pointer into the window. The scroll bars at the bottom and side of the client area allow you to move the portion of text displayed within the viewing window.

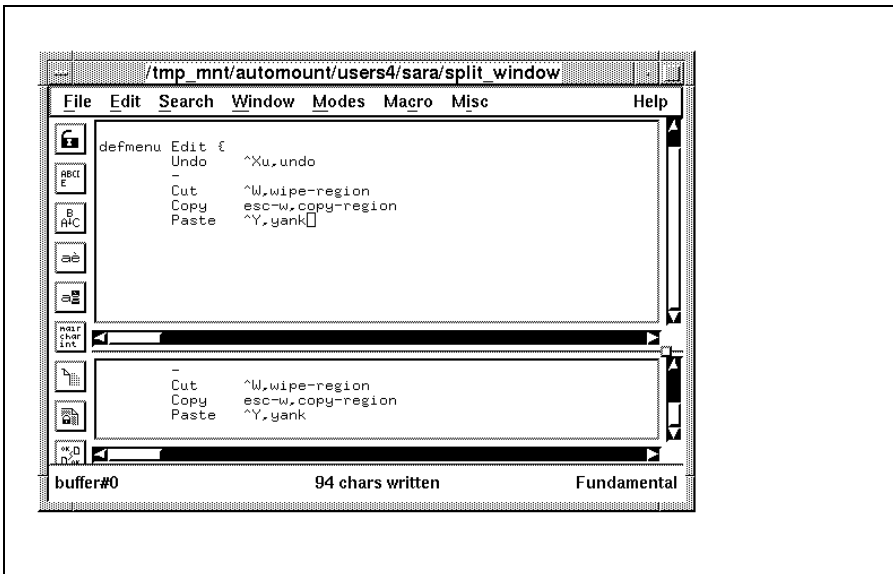


Figure 2.2: A Wx Window Split into Two Panels

The **split-window** command (**^Z^S**) enables you to split the current window into two horizontal panels. With the *sash*, the small square near the right edge of the line separating the two panes, you can change the relative pane sizes. When you place the mouse cursor over the sash, it changes to a plus sign (+) and as you drag the sash vertically, the size of the two panels changes. **Wx** does not let you make one of the panels smaller than one third the total window size.

The panels enable you to have two different views of the same buffer, and you can move the cursor or the insertion point separately in each panel. If the same portion of the buffer is visible in the two panes, text input in one is echoed also in the other. The panel that has the keyboard focus is the last one in which you have **Select** clicked. Each time you change panels, the undo stack is reset to ensure that no undo commands coming from one panel can be applied to the other. See section 3.8

The **unsplit-window** command (**^Z^U**) returns the window to the normal single panel, and resets the undo stack. Only the current window is affected by the **split-window** and **unsplit-window** commands.

2.3 The Menu Bar

Most of the **Wx** features are available on the menu bar at the top of the window. In the standard version of **Wx**, the menu bar contains the following menu names. As you will learn later, it is easy to rearrange the menus to fit your own needs, so your version may be somewhat different:

File	Commands for loading, saving files, changing directories, quitting Wx .
Edit	Commands for cutting or copying text.
Search	Commands for finding and replacing text.
Window	Commands to create or split windows, select fonts.
Modes	Commands for selecting modes.
Macro	Commands for creating and using macros, a list of macros.
Misc	Commands for compiling a file, displaying Wx features.
Help	Help on version, icons, topics, actions, and functions.

When you click the **Select** button on any of these items, the corresponding menu appears. The menu stays on the screen until you select a command or click outside the menu. Drag the mouse pointer down the open menu. This successively highlights the different commands available in that menu. When you click the **Select** button on an option, the command is issued. Next to each is the key sequence that you can type to issue the command from the keyboard, even when the menu is not open. The symbol ^ represents the Control key. See Section 3.21.

2.4 The Icon Bar

The icon bar, on the left side of the **Wx** window presents some commonly used features of **Wx**. The icon bar itself can be displayed or hidden by using the **Show/Hide Icon Bar**¹ in the *Misc* menu. Each icon can be toggled, (clicked on with the **Select** button), to change its current status.

The first five icons correspond to various features of the client area on your window. The last three correspond to three program resources. The last three icons are global resources; that is, when you change the status of the icon in a window, its status is changed in all windows.

The following features are represented on the icon bar.

Read-only This feature allows you to display your work without the risk of altering it accidentally. You can still perform any **Wx** commands that do not modify the window's contents.



The buffer is Read-Write. You can modify it.



The buffer is Read-Only. You cannot modify it.

Wrap This feature automatically breaks the lines of your text to fit the borders of the window in which you are typing.



Lines will break to fit window borders.



Lines will continue beyond window borders.

Replace/Insert mode In Replace mode, any character you type replaces the character under the block cursor, or the one to the right of the line cursor. In Insert mode, the character you type is entered at the insertion point without replacing any other characters.



New characters replace old at the insertion point.



New characters are inserted at insertion point.

1. toggle-icon-bar command **^Z^D**

Display mode This feature allows you to display non-ASCII characters in their regular, octal, or hexadecimal format.



Non-ASCII characters are displayed in regular format.



Non-ASCII characters are displayed in octal format.



Non-ASCII characters are displayed in hexa format.

Cursor choice This feature allows you to choose either a block cursor or a line cursor.



The cursor appears as a solid block.



The cursor appears as a line.

Line numbering This feature displays line numbers to the left of the text in your window.



Line numbers are displayed.



Line numbers are not displayed.

Automatic file backup When you save a file, a copy of the previous version will be made automatically. The name of the backup copy is preceded by a tilde ~.



A backup copy of your file will be made.



No backup copy of your file will be made.

Match Read-only When you load a file, the buffer permission automatically matches the file permission. For instance, if you load a read-only file, the window will automatically revert to read-only status and the read-only icon will reflect this. If you load a read-write file, the window will automatically change to read-write status. If the Match Read-only feature is disabled, a warning box will inform you whenever you load a read-only file. You can then modify the buffer contents, but if you wish to save the modified file, you must give it a different name—use the *Save As...* command.



The buffer permission automatically matches file permission.



Buffer permission does not automatically match file permission.

Confirm Remote File With **Wx** and **wxd** running on different machines, the path to access a given file may differ. “ConfirmRemoteFile” allows you to modify or at least confirm the file access path used by **wxd**, having specified the one for **wx**.



The server will ask for path confirmation.



The server will attempt to load the file without asking for confirmation.

2.5 The Message Zone

The space along the bottom of the window is the message zone—it is reserved for messages from **Wx** to you. The word in the lower right corner, (e.g. **Fundamental**), indicates the mode. The buffer name is displayed in the lower left corner. Next to it, the letter “*M*” is displayed whenever the file has been modified but not saved. The center of the message zone will display the following information at the appropriate time: file size, mark set, macro learning, etc.

2.6 The Title Bar

The window decorations vary according to the window manager you are using. If you are using the Motif window manager, **mwm**, the title of the window is the name of the file you are editing, as soon as this is specified.

2.7 A Summary of Wx Menu Commands

This section is a brief guide to the **Wx** menu items. Some **Wx** commands are not available in a menu, but only from the keyboard. Those commands are described in section 3.21.

- To open a **Wx** menu, click on its name in the menu bar.
- To select a menu command, click on it or enter the key combination (accelerator) that is listed to the right of the item.
- Some menu items are followed by an ellipsis. Clicking on one of these items will pop up a dialog box for you to enter further information, such as a filename.

The **Wx** menu commands are described on the following pages.

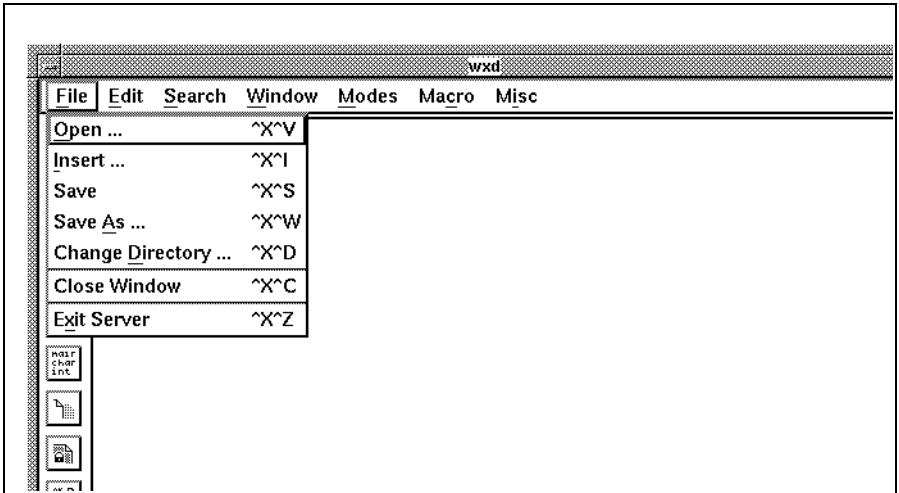


Figure 2.3: The File Menu

File This menu contains the **Wx** commands for opening, saving, and inserting files. It has the following commands:

- **Open...** (**^X^V**) pops up a dialog box for selecting an existing file to open. It is associated with the command **load-file**.
- **Insert...** (**^X^I**) pops up a dialog box for choosing a file to be inserted into the current **Wx** window at the insertion point. It is associated with the command **insert-file**.
- **Save** (**^X^S**) saves the current file. It is associated with the command **save-file**.
- **Save As...** (**^X^W**) pops up a dialog box for entering a new file name before saving your work. The file name you enter becomes the current file name. It is associated with the command **write-file**.
- **Change Directory...** (**^X^D**) pops up a File Selector for selecting the new directory. It is associated with the command **change-directory**.
- **Close Window** (**^X^C**) closes the current **Wx** window. It is associated with the command **delete-window**.
- **Exit Server** (**^X^Z**) exits from **Wx**. If you have modified a file without saving it, an Alert box will warn you. It is associated with the command **quit-the-editor**.

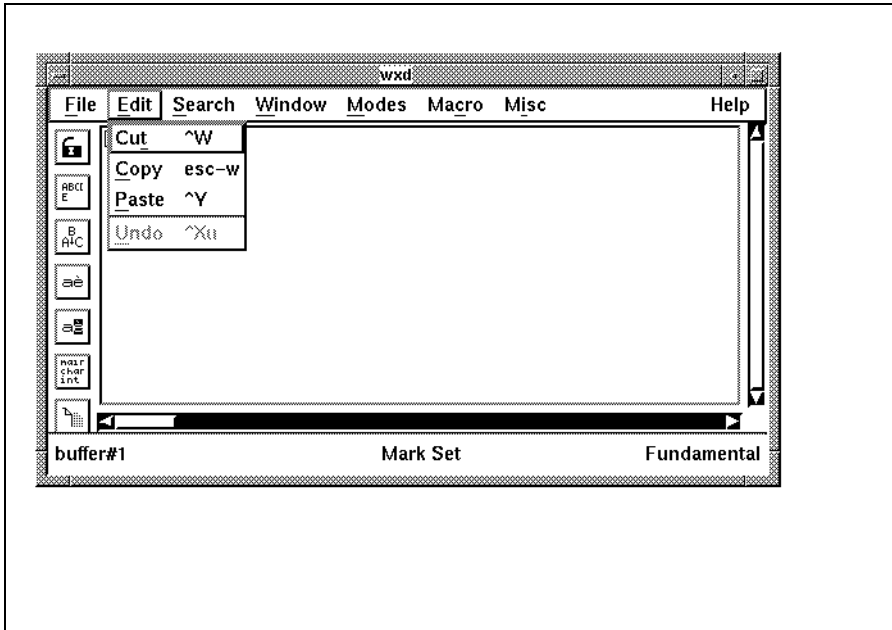


Figure 2.4: The Edit Menu

Edit This menu contains the following editing commands:

- **Undo** (**^X^u**) undoes the most recent command. Note that some commands are irreversible. See Section 3.8. It is associated with the command **undo**.
- **Cut** (**^W**) cuts the selected text, and places it into a temporary buffer. It is associated with the command **wipe-region**.
- **Copy** (**esc-w**) copies the selected text into a temporary buffer. It is associated with the command **copy-region**.
- **Paste** (**^Y**) pastes the contents of the temporary buffer at the designated point. It is associated with the command **yank**.

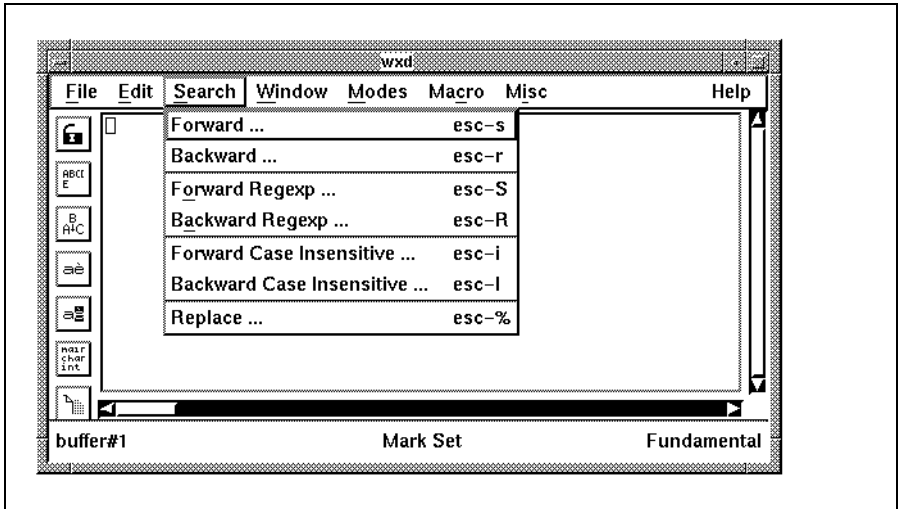


Figure 2.5: The Search Menu

Search This menu contains the following commands:

- **Forward... (esc-s)** searches forward from the insertion point for the specified string. It is associated with the command **forward-search**.
- **Backward... (esc-r)** searches backward from the insertion point for the specified string. It is associated with the command **backward-search**.
- **Forward Regexp... (esc-S)** searches forward from the insertion point for the specified regular expression. It is associated with the command **forward-regexp-search**.
- **Backward Regexp... (esc-R)** searches backward from the insertion point for the specified regular expression. It is associated with the command **backward-regexp-search**.
- **Forward Case Insensitive... (esc-i)** searches forward for the specified string without regard to case. It is associated with the command **forward-ci-search**.
- **Backward Case Insensitive... (esc-l)** searches backward for the specified string without regard to case. It is associated with the command **backward-ci-search**.
- **Replace (esc-%)** searches for the specified string and replaces it with the replacement string. The associated command is **query-replace**.

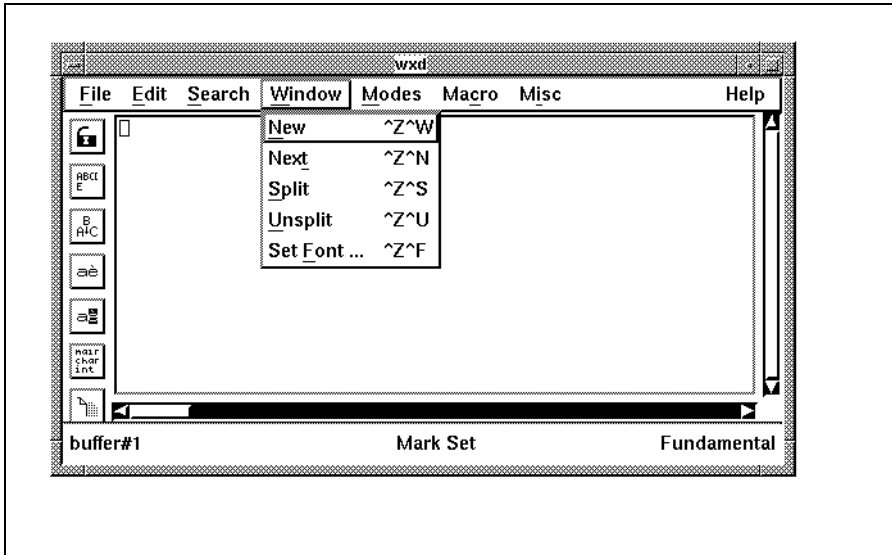


Figure 2.6: The Window Menu

Window This menu contains the following commands:

- *New* (^Z^W) opens a new **Wx** window. It is associated with the command **create-window**.
- *Next* (^Z^N) brings the next **Wx** window to the foreground. It is associated with the command **next-window**.
- *Split* (^Z^S) splits the window to provide two views of the same buffer. It is associated with the command **split-window**.
- *Unsplit* (^Z^U) returns the window to the regular single panel. It is associated with the command **unsplit-window**.
- *Set Font...* (^Z^F) pops up the font selection box for choosing the text font. It is associated with the command **set-font**.

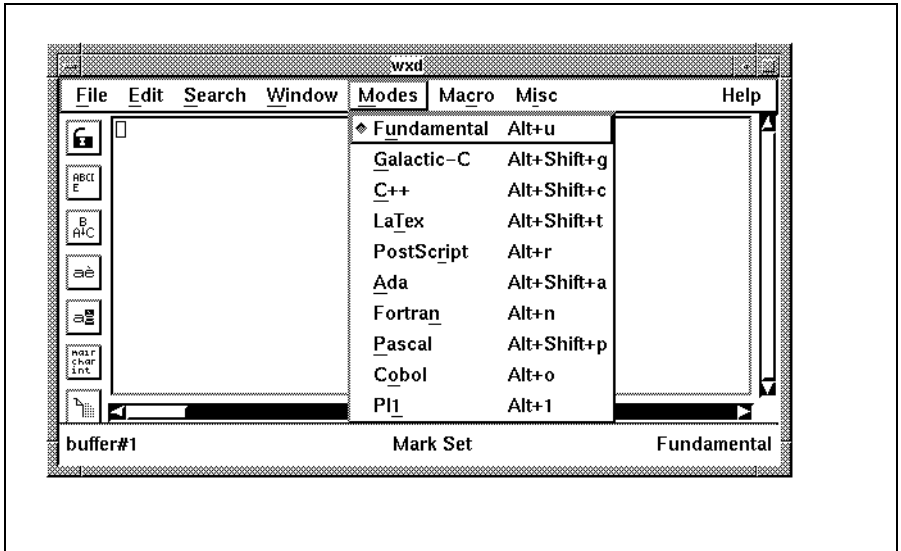


Figure 2.7: The Modes Menu

Modes This menu contains modes for editing in the following environments:

- *Fundamental* used for general purpose text editing.
- *Galactic-C*
- *C++*
- *LaTeX*
- *PostScript*
- *Ada*
- *Fortran*
- *Pascal*
- *Cobol*
- *Pl1*

Figure 2.8: The Macro Menu

Macro This menu contains the following commands:

- ***Edit macro*** appears only when a macro has been created and registered, or loaded in the file. Pops up the Script window for editing the selected macro.
- ***Execute macro*** (**^X^E**) appears only when a macro has been created and registered, or loaded in the file. Executes the selected macro.
- ***Begin macro*** (**^X(**) starts the macro learning process. It is associated with the command **start-macro-learn**.
- ***End macro*** (**^X)**) ends the macro learning process. It is associated with the command **stop-macro-learn**.
- ***Save macro...*** (**^X^M**) saves the current macro. It is associated with the command **save-macro**.

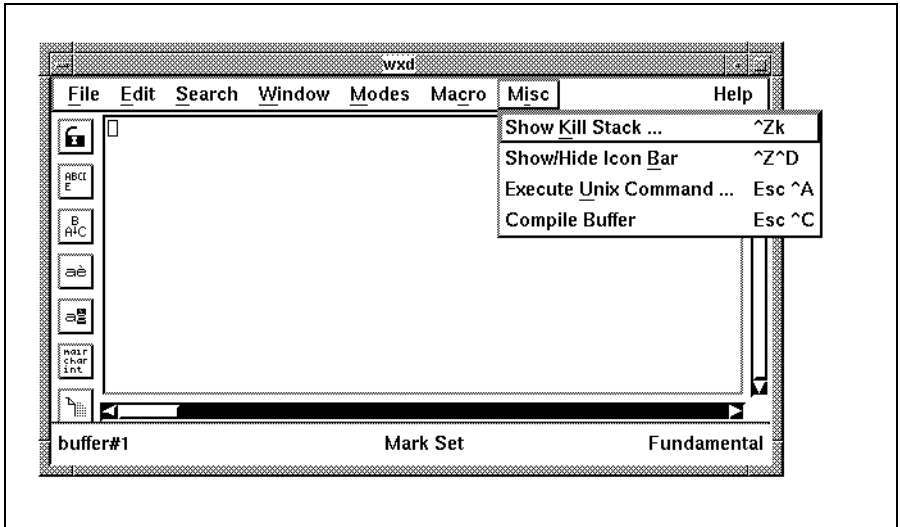


Figure 2.9: The Misc (Miscellaneous) Menu

Misc This menu contains the following miscellaneous commands:

- **Show Kill Stack...** (^Zk) pops up the **Wx Clipboard** window that lists text which has been cut or copied during the current **Wx** session. It is associated with the command **show-kill-stack**.
- **Show/Hide Icon Bar** (^Z^D) is a toggled command that shows or hides the icon bar on the left of the **Wx** window. It is associated with the command **toggle-icon-bar**.
- **Execute Unix Command...** (Esc^A) pops up a prompt box in which to enter a Unix command. It is associated with the command **exec-command**.
- **Compile Buffer** (Esc^C) compiles the contents of the current file. It is associated with the command **compile-buffer**.

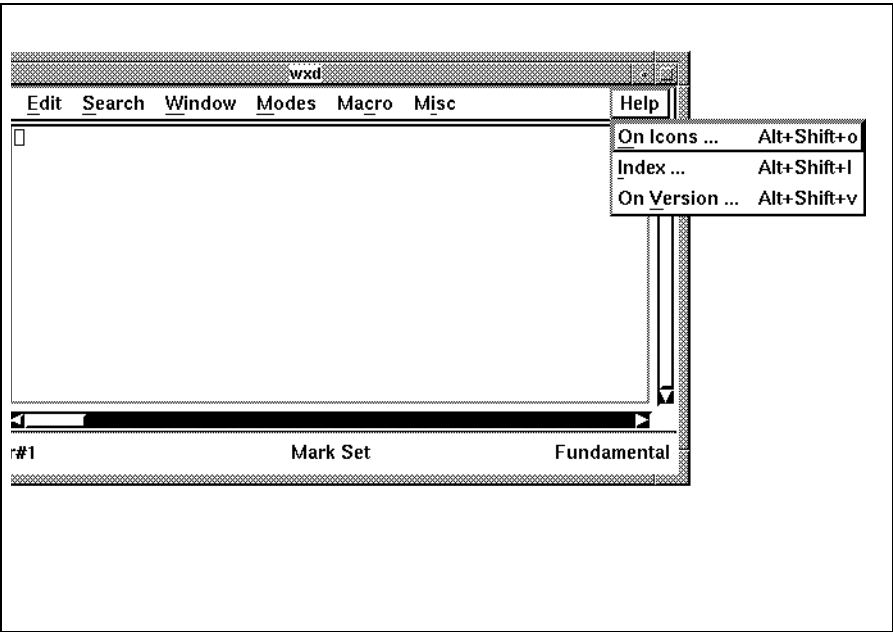


Figure 2.10: The Help Menu

Help contains help on topics, actions, functions, version, and the icons of **Wx**.

CHAPTER 3: Editing Techniques

This chapter describes the editing techniques of **Wx**. If you have already used a text editor, most of this section will be familiar, but you might want to check for features of **Wx** that are new to you. Modes and macros are described in more detail in Chapters 4 and 5.

Wx editing commands can be issued in one of two ways:

- select the command from the menu
- type a key combination.

All of the menu commands have an equivalent key combination. They can be issued either by selecting the item in the menu or by entering the key combination. However, there are some commands that can only be issued from the keyboard. They are discussed in section 3.21, Keyboard-Only Commands.

It is easy to customize your **Wx** environment, so the version you are using may be slightly different from the one described here. See Chapter 6: Configuring **Wx**, for details. At the end of this chapter we list all **Wx** commands by usage, and describe each briefly; in the appendix we list them in alphabetical order.

The following items should be kept in mind:

insertion point The point at which text is inserted. It is indicated by a block or a vertical bar, depending on your particular set-up.

mouse cursor The mark on the screen that moves as you move the mouse around. The mouse cursor is context-sensitive, looking like an I-beam or capital I when it is inside the text window, and like an arrow when it is in the menu or title bar.

dialog boxes Any **Wx** dialog box (except the Motif File Selector) can be aborted by typing `.` This returns **Wx** to the text buffer, cancelling the command that triggered the dialog box, but it does not undo any changes that have already been made. To cancel the Motif file selector box, if you have not enabled the `expand` option, enter the `osfCancel` key, usually `Esc`.

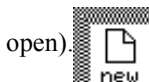
Control vs. Esc In general, keyboard commands involving the **Control** key are valid whether you use upper or lower case, but this is not true for the **Escape** key.

3.1 Opening Files

You can open new or existing files from the Manager Box, an `xterm`, or from a **Wx** window.

A *new* file window can be opened in the following ways:

- Click on the new icon in the Manager Box (or click on the **Wx** rectangle if it is



- Enter **wx** in your xterm.
- Select *New* (**^Z^W**) from the **Window** menu.

An *existing* file can be opened in the following ways:

- Select *Open ...* (**)** from the File menu.

- Click on the open icon in the Manager box.



- Enter **wx filename** in your xterm.

When you select *Open ...* or the **open** icon, a **Wx** window appears along with a standard Motif file selector.



Figure 3.1: The File Selector

The File Selector displays a scrolling list of the files in the current directory. To open a file do one of the following:

- click the **Select** button on its name, then click on **Ok** or press the **Return** key, or
- double-click the **Select** button on the file name.

The selected file is loaded into the buffer—the window title is the name of that file. The file is now the *current file* for that **Wx** window.

If you select **Open ...** when there is already text in the buffer, and changes made in the file have not been saved, the alert message “**Changes not saved, Load a new file?**” appears. Press **Cancel**, the default choice, to abort the **Open ...** command, or press **OK**, to load the file. If you press **OK**, the changes are lost.

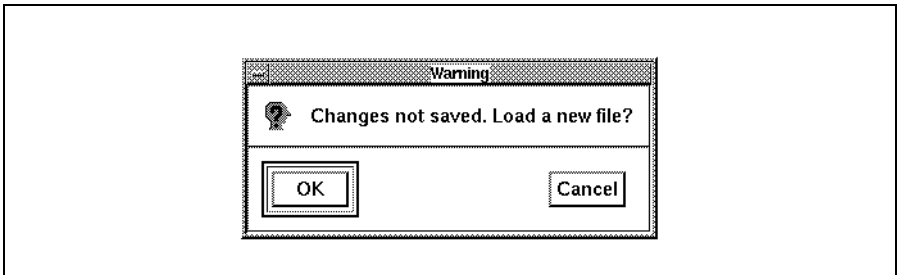


Figure 3.2: The "Changes not saved" message

If you attempt to open a directory file, a warning Beep sounds and no data is loaded in the window. However, if the file you want to open is in a different directory, you can search for it by traversing directories. To look higher in the tree, double-click on **..** which corresponds to the UNIX **dotdot** notation. In UNIX, **..** is a pseudonym symbolizing the parent directory; i. e., the directory directly above the current one. Double-clicking on **..** will make the parent directory the current directory. You can thus browse through the directory hierarchy, by double-clicking on **..** to climb the tree, and double-click on a directory name to see what is lower down.

The file selector has one, optional, non-Motif feature which is enabled by setting the **expand** resource. (See section 6.2). With this option set, you can use the filter shortcut as follows.

Once you have entered a part of the pathname desired in the File Selector’s edit box, hit **Esc** or **Tab** and:

- The contents of the Edit Box is copied to the Filter Box, with a * wild-card character appended.

- The filter command is activated.
- The contents of the directory and the file windows reflect the result of applying the newly specified filter.

To set this resource in your `.Xdefaults` file enter:

```
wxd*expand:true
```

Or, set it when launching `wxd`, with the **-expand** command line option:

```
wxd -expand
```

3.1.1 Changing the Directory

The ***Change Directory...*** item (^X^D) in the **File** menu, pops up a File Selector which contains a list of directories to choose from. You can traverse the UNIX directory hierarchy by double-clicking on `..` to go higher, or on a directory name to go lower. Once you find the desired directory and click on **OK** in the File Selector, the current directory changes. The next time you give the ***Open...*** or ***Save*** command you will get a list of the files in the new current directory. Of course, you must have the appropriate permissions: *read* permission for loading, *write* for saving.

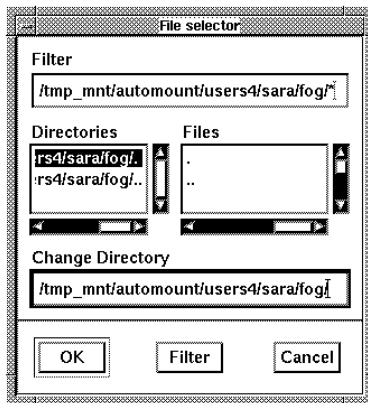


Figure 3.3: The Directory File Selector

Another way to specify the current directory for a window is to open the window by entering **wx filepath/filename** in your xterm window.

- In the default case, the current directory is the directory you were in when you launched **wxd**.
- The current directory is the one accessed by the ***Open...***, ***Insert...***, ***Save*** and ***Save As...*** items.
- You can have a different current directory for each **Wx** window you have opened.
- The current directory associated with a **Wx** window is not necessarily the directory that contains the file currently loaded into the window, since the current directory may have been changed since the file was loaded.

3.2 Entering Text

Any text you type, paste or insert in a **Wx** window will appear at the insertion point. You can delete characters before or after the insertion point by using the **Back-space** or **Delete** keys respectively. For example, type the following text in a window, pressing the **Return** key after each sentence.

**This is a text file.
I'll use it to practice Wx.
When I no longer want it,
I can get rid of it.**

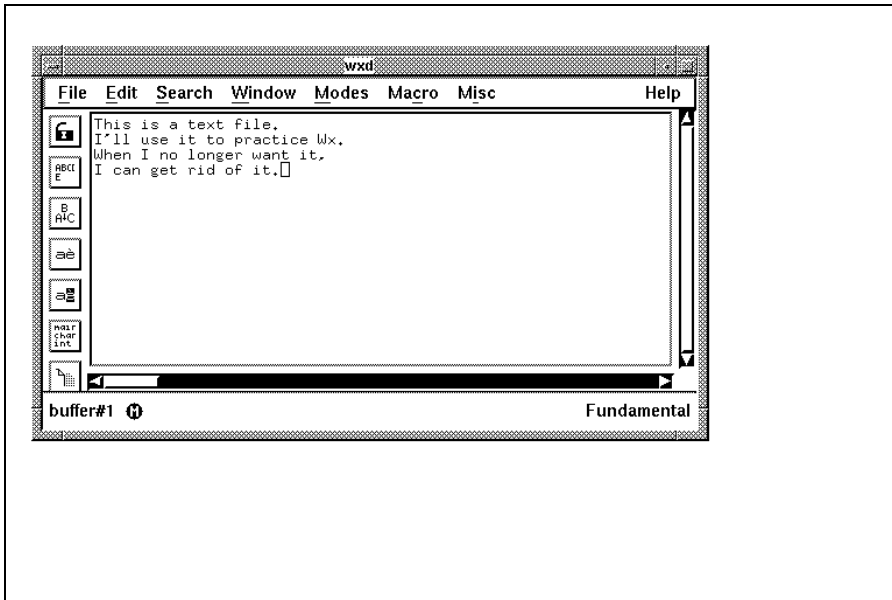


Figure 3.4: A Wx window with newly entered text.

Note the letter **M** in the lower left corner of the window. It indicates that the buffer has been modified and the modifications have not yet been saved. Whenever you have created or modified a text file, you need to save it. See Section 3.10.

3.3 Selecting Fonts

To select a font for the text in your current **Wx** window, choose the **Set Font...** item (**^Z^F**) in the **Window** menu. The Font Selector which pops up lets you select a font interactively. An example of the font you select is displayed in the lower half of the box. This command cannot be undone, and it resets the undo stack.

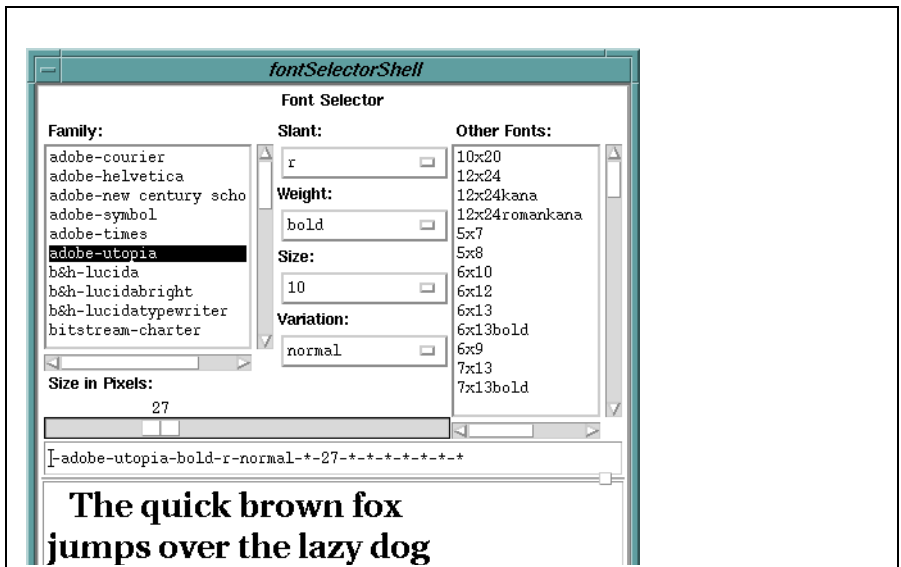


Figure 3.5: The Wx Font Selector

3.4 Moving in Your File

The window on the screen is a window on your text buffer, and when you type enough text, some of it will not be visible in the window. However, you can move the insertion point to wherever you want in the buffer, and as the insertion point moves, the text shown in the window follows along so that the insertion point is always visible.

You can also move the viewing area, without moving the insertion point, by scrolling with the **Custom** button. Then, the insertion point may be outside of the viewing area. If this is the case, as soon as you start typing text, the viewing area will revert back to include the insertion point and show you the text being inserted.

There are several ways to move the viewing area in the window, and/or the position of the insertion point. Use whichever is most convenient for the task at hand. You can use:

The scroll bars To move up or down one line at a time, click with the **Select** button in the down or up arrows in the scroll bar at the right side of the window. The text viewing window moves in the direction of the arrow clicked, the insertion point remains in place.

For moving a distance of more than one windowful of text, using the *slider* is convenient. Its size relative to the full scroll bar indicates the relation of the visible text to the total size of the buffer. As you drag the slider up and down with the **Select** button, the portion of text viewed moves; the insertion point remains in place.

Similarly, you can scroll horizontally, using the slider or the arrow keys at the bottom of the window.

The mouse You can also scroll up or down by dragging the text with the **Custom** button. When you do so, the mouse pointer changes to a pointed finger. The text viewing window moves, the insertion point remains in place.

The Keyboard There are several ways to navigate within text from your keyboard. One is to use the arrow keys: pressing the up arrow moves the insertion point upwards, the left arrow moves it to the left, etc. Another way is to use keyboard commands. For instance:

- type **Esc** < to move the insertion point to the beginning of the text buffer.
- type **Esc** > to move the insertion point to the end of the text buffer.
- type **Esc num** , where *num* is the number of a line, and **G** stands for *go*, to move the insertion point to line number *num*. For example, type **ESC 5** to move to the fifth line from the top of the buffer.
- type **^A** to move the insertion point to the beginning of the current line.
- type **^E** to move the insertion point to the end of the current line.

3.5 Selecting Text

It is often useful to select a region of text in order to delete it, or to copy and paste it in another part of the file or another window. Selected text is stored in the **X Selection** like text selected in an **xterm** window. It can be pasted using mouse **Button 2**. A text region is any *continuous segment of text*. Selected text is highlighted, i.e. displayed in reverse video, or in an alternate color on color terminals.

There are several ways to select text. You can select:

- a **word** by double-clicking **Select** on the word.
- a **line** by triple-clicking **Select** on the line.
- any **region of text** by dragging the mouse cursor over the text segment. You can drag in any direction. If the cursor is above or under the **Wx** window, the text is scrolled as the selection is extended.
- a **region of text** by using the shift-click method. Click the **Select** button at the beginning of the region to set the mark, then move the cursor to the end of the region, press the **Shift** key, then click the **Select** mouse button. This defines the region as all the text between the mark and the mouse cursor position. You can also set the mark by typing
^ **SPACE**. On many keyboards, ^@ also works.

The *exchange point and mark* keyboard command (**^X^X**) allows you to see where the mark has been set without losing the selection that you just made.

Once you have selected text, you can delete it with **Cut**, or make a copy with **Copy** items of the **Edit** menu. These commands store the text segment in a buffer called the **kill stack**. This buffer is a push-down stack of strings, each string being a text segment that was selected and cut or copied. These strings can then be pasted into any **Wx** window.

3.6 Cutting, Copying, and Pasting

To cut or copy a block of text you must first *select* it, then use the **Cut** (**^W**) or **Copy** (**esc-w**) items in the **Edit** menu. When you cut text, it is removed from the file, when you copy, it stays there. In either case, the text is stored as the top item in the kill stack. This is a push-down stack of all text segments you have cut or copied during the current session of **Wx**.

You can recover any text segment from the kill stack and paste it anywhere in the same or another window. To paste, move the insertion point to where you want the selected text to appear, then select **Paste** (**^Y**) from the **Edit** menu or press mouse Button 2: the text is inserted at that point.

Remember that the text which has been pasted into the window is still stored in the kill stack. It will stay there until you leave the **Wx** server, and it will be pushed further down on the stack every time you cut or copy new text.

You can move text segments by selecting **Cut** instead of **Copy** and then pasting. You can select words, single characters, or any other contiguous text segments. You can even select the whole file, copy or cut it, and then paste it with the **Paste** command as often as you want. Section 3.7 describes how to recover text stored at lower layers of the the kill stack.

You can also cut and paste between windows. Select text in one window and cut or copy it, then click in the other window to place the insertion point there and paste the text. The kill stack is independent of which window these commands are called from, and it contains all text you have cut or copied in the current **Wx** session, no matter how many windows are open.

3.7 The Kill Stack

Whenever you cut or copy text during a **Wx** session, it is stored in a buffer called the **kill stack**. The contents of the kill stack are displayed in the **Wx** clipboard. You can view the kill stack by selecting **Show Kill Stack (^Zk)** in the **Misc**(ellaneous) menu. The figure below shows the Wx Clipboard and the contents of the kill stack.

When you select **Paste** in the **Edit** menu, the latest text segment you cut or copied is pasted after the insertion point of the current window. However, the older segments are not discarded: they remain in the kill stack during the **Wx** session. The stack is a push-down stack. Each time you do another cutting or copying operation, the segment is placed at the top above all other segments previously cut or copied in the current **Wx** session.

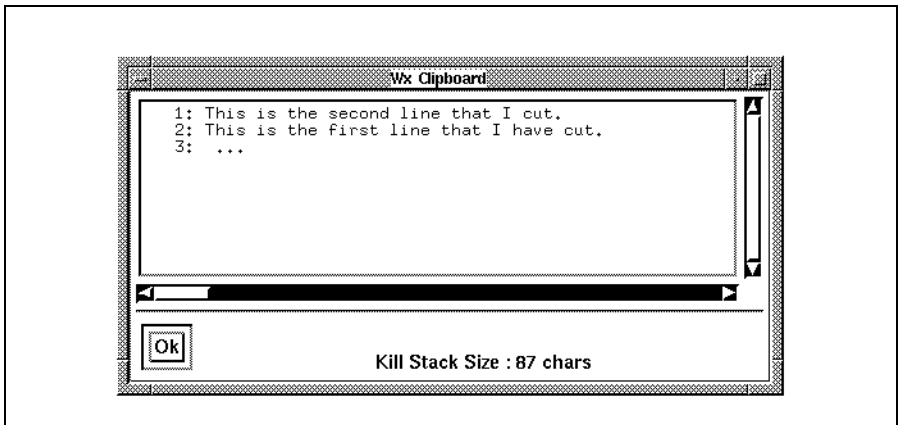


Figure 3.6: The Wx Clipboard showing the kill stack

The **Wx** Clipboard displays a numbered list of the text segments in the kill stack. If a segment contains more text than can fit on a single line, there is a suspension sign (...) at the end of the line. The size of the kill stack is displayed at the bottom of the window.

Any of the segments in the kill stack can be retrieved and pasted. To select a segment other than the top one for pasting, you can use one of two techniques:

1. Click the **Select** button on the text segment, then paste with mouse **Button 2**.
2. Specify the text segment number with **Esc num** where 'num' is the segment number, then paste with .

If the segment you seek is not visible in the clipboard window as it pops up, you can find it by scrolling or resizing the window.

You can remove the top item from the kill stack by giving the **pop-kill stack** command. The kill stack is there as long as you do not quit **Wx**, and you can access any of the segments at any time.

3.8 Undo

Most **Wx** commands can be “undone”. **Undo** appears in the **Edit** menu whenever a command has been issued that can be undone. However, the following commands are irreversible once confirmed. In addition, any commands executed *before* these commands are irreversible.

- **Open...**, **Save**, **Save As...**, **Insert...**, **Set Font...**
- **Split**, **Unsplit**. In split window mode, changing panes.

An **Undo** executed immediately after a **Replace** action ‘undoes’ the entire replace. There is also an **Undo** within the **Replace** action which lets you undo individual replace actions. The depth of commands that can be undone depends on the context and on the order in which the commands have been issued.

context: Each twenty commands – or group of commands (see below) – the current status of the buffer being worked on is saved. As commands are issued, they are stacked in a buffer. When **Undo** is activated, all the commands except the last are executed on the buffer that had been saved. In addition, the un-executed command is unstacked. Thus, as you repeatedly execute undo, the list of commands to be undone is reduced, until there is nothing to undo. If, in this context, you again issue **Undo**, a beep and a message inform you that there is nothing to undo.

command grouping If the same command is issued several times in succession, e. g. delete-word, it is counted as one command in the undo stack. Thus, if you issue **Undo** after three consecutive delete-word’s (**Esc d**), then the three words that were deleted are re-pasted with the one **Undo**. However, if the sequence of commands was, for example, **delete-word**, **insert-char**, **delete-word**, **backward-char**, **delete-word**, each of the commands is stacked individually. **Undo** on such a sequence backs up one command at a time. A complete list of the grouped commands is given in Appendix E.

For the undo stack, executing a macro is counted as one command: issuing **Undo** after a macro has been executed undoes all the commands of the macro. If undo is executed immediately after macro learning ends, the buffer is returned to the state it had before macro learning was started, i.e. all the macro commands are undone. Finally, **Undo** is inactive *during* macro learning.

3.9 Searching and Replacing

The **Search** menu contains options for finding and replacing text. This can be either a simple string, or a regular expression.

- **Forward** and **Backward** find the specified text without changing it.
- **Forward Case Insensitive** and **Backward Case Insensitive** search for all instances of a string without regard to case.
- **Forward Regexp** and **Backward Regexp** allow you to search for any string that matches the regular expression you specify.
- **Replace** finds one specified text segment and replaces it by another, using regular expressions or simple strings in the **Find** and **Replace By** fields.

3.9.1 Searching for Text

A search is started from the insertion point. Here is an example of how to do a forward search of an entire file:

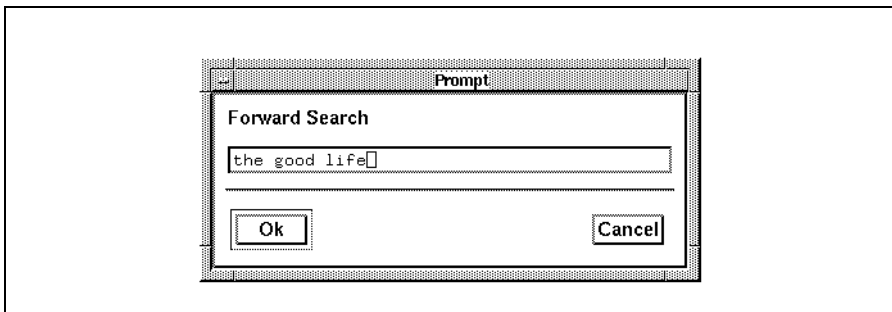


Figure 3.7: The Search window

1. Place the insertion point at the beginning of your file.
2. Select **Forward** in the **Search** menu (or type **Esc S**). A dialog box will appear in which to enter the search string.
3. Type the string you are searching for in the edit box. You can type any characters (including spaces).
4. Confirm your choice, either by pressing **Return** or by clicking on **Ok**. **Wx** searches forward from the insertion point and stops when it reaches the first occurrence of the string.

3.9.2 Replacing Text

The **Replace** command (**esc-%**), lets you find a string and replace it by another. You can either do the replacement step by step, deciding to confirm or refuse the change each time, or you can make the change globally, by choosing the option **Replace All**. In either case, the search proceeds forward from the insertion point to the end of the file.

When you select **Replace**, a dialog box appears, asking you to enter the string to search for into the **Find** field, and the replacement string into the **Replace By** field.

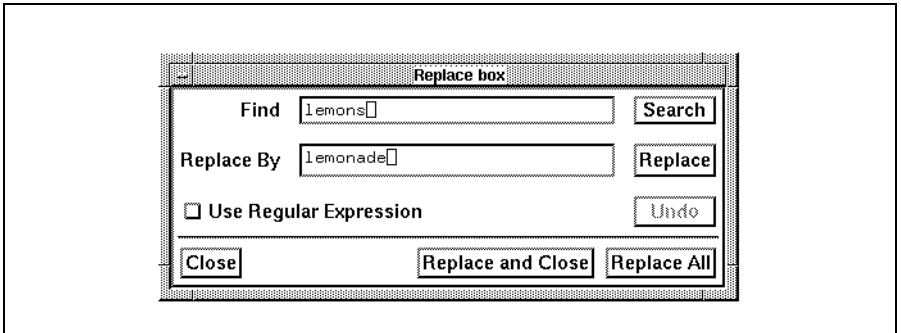


Figure 3.8: The Replace window.

Use the buttons at the bottom of the box to do the following:

- To find the next occurrence of the search string without doing a replace, click on **Search**, or type **^S**.
- To replace the text with the new text, click on **Replace**, or type **^R**.
- To undo the last replacement made, click on **Undo**, or type **^U**. By successively clicking on **Undo**, you can undo the preceding replacements.
- To replace only the current occurrence of the search string by the replacement string, and then exit the Replace box, click on **Replace and Close**, or type **^L**.
- To replace all further occurrences of the search string by the replacement string, click on **Replace All**.
- To quit the **Replace** box, click on **Close** (or type **^G**).
- To search and replace using a regular expression, click on the **Use Regular Expression** toggle button. To return to normal string search, click on the toggle button again.

Here's how you might use **Replace** to replace the string "one" by "you" in the file you are editing:

1. Place the insertion point at the beginning of the file (**Esc <**).
2. Select **Replace** in the **Search** menu (or type **Esc %**) to pop up the Replace window.
3. Type the word **one** in the Find field, and press **Return**. This places the insertion point in the second edit box to enter the replacement string.

4. Type the string **you** in the **Replace By** field. Press **Return** again to start the search. **Wx** will search forward from the insertion point and stop on the first occurrence of **one**. If the search string is not found, **Wx** will beep. If it finds the string, the insertion point is placed right after the string to indicate its position.
5. Press **Return**. This tells **Wx** to make the replacement, and to continue the search.

If you want **Wx** to replace all occurrences of the search string *forward* of the insertion point, click the **Select** button on **Replace All**. If you want to replace only the current occurrence, click on **Replace** and **Close**. You can undo the last replacement you executed by clicking on *Undo*.

The first time you use the search or replace commands, the dialog box is empty. In subsequent uses, the last string entered is shown and highlighted. You can keep this string, edit it, or enter a new string.

To exit from the **Replace** window, or any other dialog window that appears when you give a command, click on **Ok** or type **^G**. This takes you back to the text buffer.

3.9.3 Searching by Regular Expression

The *Forward Regexp...* (**esc-S**), and *Backward Regexp...* (**esc-R**) items in the **Search** menu are a convenient way to search through text.

Regular expressions provide an economical way of representing *patterns*. **Wx** conforms to the standard UNIX syntax for regular expressions, as used, for instance, in the line editor *ed(1)*. Regular expressions can be used both in searching and replacing. When used for replacing they can be very powerful. We will give a few examples here. See your UNIX Reference manual for a full discussion.

A regular expression composed of characters other than **\ [. & * \$** matches itself. For example, any expression containing only alphanumeric characters will match itself. Thus, for example, the expression:

abracadabra

matches itself. Normally, regular expressions are mostly made up of characters that match themselves.

Regular expressions can be written in sequence one after another. The result is a new regular expression denoting any string denoted by the first followed by any string denoted by the second. The string denoted by the first expression is the longest possible one.

You can write expressions that denote classes of characters. To designate any character in a set of characters, write the characters in a string enclosed by brackets. For example, **[abc]** specifies any character that is an **a**, a **b** or a **c**.

The expression **[a-z]** stands for any letter between **a** and **z**. Likewise, **[A-Z]** denotes the letters between **A** and **Z** inclusive, and **[0-9]** denotes the digits between **0** and **9**. These are regular expressions of more than one character that match strings of exactly one character. They can, of course, be combined in the expected ways. For example:

[a-z][1-9] matches any string consisting of one lower case letter followed by one digit

[a-zA-Z] denotes any letter, lower or upper case; it matches a string of exactly one character

To denote the set of characters that do *not* belong to a regular expression, precede that expression by and enclose the whole expression in brackets. A period **.** (dot) stands for any character whatsoever.

For example:

[A-Z]x. matches any string of three characters beginning with a capital letter followed by an **x** followed by any character.

[-zA-Z0-9] denotes any non-alphanumeric character.

A regular expression followed by a ***** (star) is itself a regular expression and denotes an arbitrary number (including 0) of consecutive occurrences of a string denoted by the regular expression preceding the *****.

For example:

1* denotes a string of any number (including none) of 1's

[0-9]* denotes the empty string or any string of digits

[0-9][0-9]* denotes any nonempty string of digits

[-zA-Z0-9]* denotes a (possibly empty) string of non-alphanumeric characters

We have seen that in a regular expression certain characters have special meanings. To cancel the special meaning and denote the character itself, precede it by a **** (backslash). This works for all characters except digits, **"** (") and **"** ("). As we said, regular expressions can be used both in searching and replacing. Particularly useful when you are doing a replace is a feature that allows you to refer, by number, to the string matched by a subexpression.

This is best understood by looking at an example. Suppose we want to reverse the order of two consecutive digits. The pattern for specifying any sequence of two consecutive digits is :

[0-9][0-9]

An equivalent pattern would be the following, where we enclose the two main sub-patterns in parentheses, with \ (and \):

\([0-9]\)\([0-9]\)

Using one of these as the **Find** string and

\2\1

as the **Replace By** string, we obtain the desired result.

Here **\1** denotes the string matched by the expression inside the first set of parentheses, i. e. the first digit, and **\2** stands for the string matched by the expression inside the second set of parentheses.

3.10 Saving Files

To save the current file¹, click on **Save** (**^X^S**) in the **File** menu, or select the file's name in the Manager Box and click on the **Save** icon. **Save** writes the contents of the text buffer to the current file, i.e. the one whose name is given at the top of the **Wx** window. If there is no current file, (if the window's title bar is empty), a File Selector is popped up for entering the file name. Enter a file name, then click on **OK**.

Suppose you enter a filename that already exists? You will see the warning message shown here:

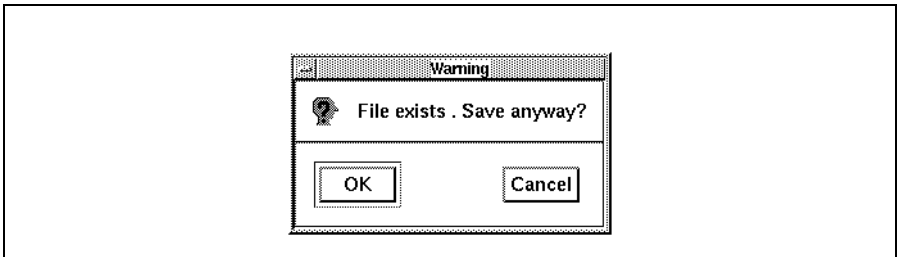


Figure 3.9: Alert message during a *Save As*

- If the Automatic Backup icon is set to “on”, and you click on **OK**, then the existing file will be saved under the same name with the suffix ~ appended to it, before your buffer is written into the file.
- If the Automatic Backup icon is set to “off” then the original file will be overwritten, and only the new version is saved.

The Automatic Backup feature can be disabled by toggling the icon or by changing the resource file. See Chapter 6.

Once the file has been saved, the message “n chars written” (where n is the number of characters in the buffer) is displayed at the bottom of the window.

1. The *current* file is the last file loaded by **Open** or saved to by **Save** or **Save As...** Whenever you open a **Wx** window without giving a file name, the default name / `dev/null` is assigned to the file.

3.11 The Save As... Command

The **Save As...** item in the **File** menu (**^X^W**), lets you change the name of the file the current buffer is saved to. This is useful, for example, if you want to save a back-up copy of your file under another name. Open the **File** menu, and select **Save As...**. The File Selector will appear with the current name of the file in its edit box. Delete the old name (using backspace) and type in the new one. When you have entered the name you want, click on **Ok**, or press the **Return** key.

As with the **Save** item, if you enter a file name that already exists, a warning message appears, asking if you want to overwrite the existing file (see Figure 3.9). Clicking on the **Cancel** button, which is the default button, cancels the command and returns to the **Wx** window. Clicking on the **Ok** button saves the buffer in the file specified, after having copied its contents into a file with the same name but the suffix **~** appended to it *if the Automatic file backup icon is set to “on”*. If the Automatic backup icon is “off”, no copy will be made of the file.

An attempt to save to a directory produces a warning Beep and the buffer is not saved.

It is important to be clear about the difference between the options **Save** and **Save As...** in the **File** menu.

- **Save** saves the contents of the current window to the current file without asking you for a file name unless the current file name is undefined, i.e. is `/dev/null`.
- **Save As...**, on the other hand, always asks you to specify a new file name.

3.12 Inserting Files

The ***Insert...*** item (^X^I) inserts the selected file at the insertion point. This does not erase the text already in the window, but appends the text of the new file to the text preceding the insertion point. The current file name does not change.

Open and ***Insert*** offer two different ways of getting a file into a window. The ***Open...*** item allows you to load a file instead of the one you currently have in your text buffer, while ***Insert...*** allows you to add a file's contents to what you already have in the buffer. The difference is important: when you *open* a file into your current buffer, what you already have in the buffer is eliminated. Thus, if you don't want to lose the text in the buffer, you must save it first. That is why you are asked to confirm this potentially regrettable action. Moreover, when you open a file, it becomes the *current file*. On the other hand, when you *insert* a file into the buffer, it becomes part of the original file. The original file remains the current file.

3.13 Closing Windows

The ***Close Window*** item (^X^C) closes the current window. If you select this option without having saved any modifications you have made, **Wx** will ask you to confirm the command. If you click on **OK**, **Wx** will close the window without saving its contents. If you click on **Cancel**, **Wx** will cancel the ***Close Window*** action. If you give this command when a single **Wx** window is open, you eliminate that window, but do not quit the **Wx** server.

3.14 Wx Modes

Modes are working environments that make your editing task easier when you are programming, formatting, or simply editing text. A complete guide to using language modes in **Wx** is found in Chapter 4.

The **Modes** menu, Figure 2.7, lists all the currently available modes. You can change the mode for the current window by clicking on the desired mode name. The diamond indicates the currently active mode. The startup mode is **Fundamental**, a basic mode used for general purpose text editing. The other modes are for more specific editing tasks.

Modes are defined for various programming languages, such as Galactic-C, C++, Ada, Fortran, Pascal, Cobol, Pli, and PostScript. These modes provide abbreviations and shortcuts for writing programs in each of these languages. In addition, there is the LaTeX mode for formatting text files.

When you load a file (or open a new window with a file specification), the file name's suffix specifies which mode should become current. New modes can be added to the *Modes* menu by modifying `.wxrc`. See Chapter 4, Wx Modes. When you select a mode, its features become available in the current window, and its name is displayed in the lower right corner. Each window can have its own mode.

An example of mode expansion used in **C mode** is an alias for the word **main**. Typing **main** or an initial substring, and pressing the **Tab** key to expand the alias, you obtain the following text:

```
main (argc, argv) char **argv;
{
    o
}
```

where the **o** indicates the position of the insertion point. You can create modes containing a glossary of technical terms, or other text that will be expanded when you type a few letters. See Chapter 4: Wx Modes, for further details on modes and how to create your own modes or augment existing ones. It describes the standard language modes supplied with **Wx**.

3.15 Using Macros

Macros are a convenient way to avoid repetitive tasks, and save time when working on text. Use macros whenever you plan to use a sequence of keyboard commands more than once. A detailed guide to creating and editing macros is found in Chapter 5.

The **Macro** menu contains commands for creating and executing macros. The item *Execute macro*, if present, gives you access to a cascading sub-menu with all the macros currently available. These may have been loaded at startup from the file specified in the `.wxrc` file, with the keyword `loadmacro` . For example, you could specify:

```
loadmacrofull_pathname/.wx-macros
```

where `full_pathname` is the pathname for your macro file directory, and `.wx-macros` is the name of your macro file.

Remember: if no macros are currently loaded, *Execute Macro* and *Edit macro* are not listed in the **Macro** menu.

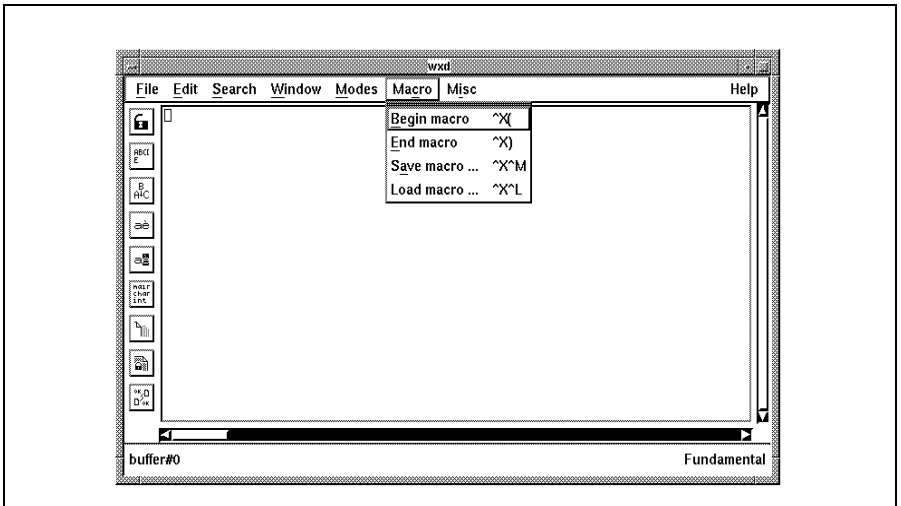


Figure 3.10: The Macros sub-menu

See Chapter 5 for further information about creating, editing, and using macros. You can store any macros you want to re-use in `.wx-macros` or another file. The standard distribution includes a few macros as examples.

3.16 Compiling a Buffer

Compile Buffer (in the **Misc** menu) compiles the contents of the current file . Before launching the **Compile Buffer** command, you should save the latest version of your buffer, since it is the saved version that is compiled. When you give the command, a panel appears, displaying any compilation errors. This panel is labelled **Compilation Diagnostics**.

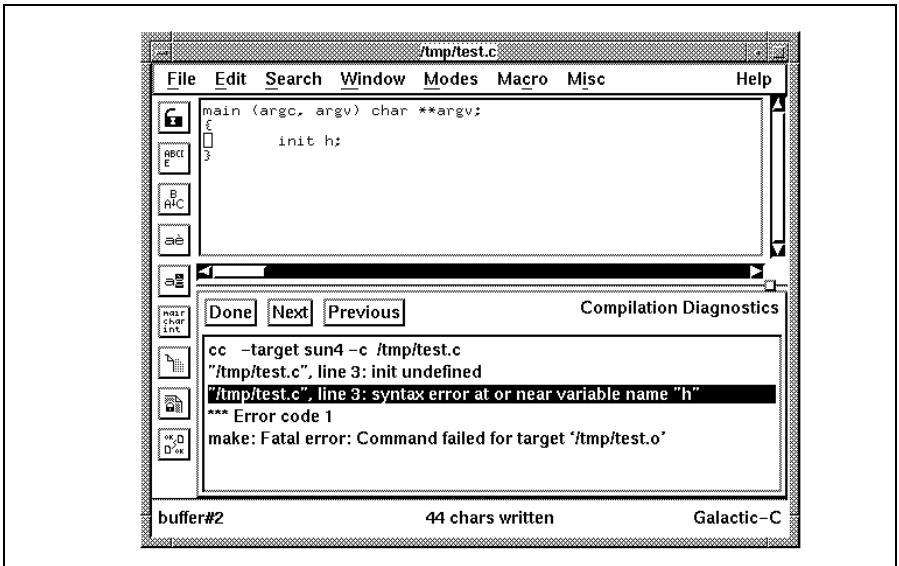


Figure 3.11: The Compilation Diagnostics panel.

The compile operation this command actually launches must be specified in the .wxrc startup file (see section 4.1.1). If no compile operation is specified, the error message **"No compile command defined"** will appear in the message zone. The compile command executed depends on which mode you are in; for instance, in C mode the command is usually **make(1)**.

The compile errors are displayed in the lower panel, with the line number in the source buffer where the error occurred. To correct the erroneous line, you can either go to that line with **Esc num** , where "num" is the line number, or select the error line by **Select** clicking on it. With the second method, the insertion point in the upper panel is placed on the line which caused the error. The line number is determined using the set of regular expressions specified under the **error** keyword.

After correcting one or more errors, you can re-save your file and relaunch the ***Compile Buffer*** command to see if the corrections are acceptable. You can recompile before correcting all errors, but you must re-save the buffer each time. If you changed the number of lines in the buffer, **Wx** will remember where the old errors are, and will indicate their current line numbers. You can click on a line of the lower panel to go to the corresponding line of the buffer, click on the **Next** button to go to the next error in the buffer, or click on **Previous** to go to the preceding error.

3.17 Execute UNIX Command...

This command brings up a Prompt Box in which you can enter the UNIX command you wish to execute. You execute the command by clicking on the **Ok** button or pressing the **Return** key. By clicking on the **Cancel** button, you can exit this option without executing the UNIX command you specified.

3.18 Using the Mini Buffer

To avoid popping dialog windows when you use the keyboard accelerators, enable the mini buffer. Any information that would have been entered or displayed in the dialog window is in a text input field at the bottom of the wx window (the mini buffer).

Wx displays the text input field and forces keyboard focus in it whenever a command that requires additional information or confirmation is triggered with an accelerator, e.g. `^X^V` to load a file. Instead of opening the FileSelector, wx will expect the file name to be typed in the mini buffer. To cancel a command, enter `^G` in the mini buffer.

If, with the mini buffer enabled, you use the menu to launch a command, the dialog windows will still be used for additional input or alert messages.

To enable the mini buffer, launch wxd with the **-umb** command option:

```
wxd -umb
```

or set the wxd resource **useMiniBuffer** to True in your `.Xdefaults` file.

The mini buffer is used for commands such as load-file (`^X^V`), load-macro (`^X^L`), insert-file (`^X^I`), reload-file (`^X^O`), write-file (`^X^W`), exec-command (`Esc ^A`), and all search and substitution commands.

The mini buffer can prove to be a real time-saver once you get used to the accelerators.

3.19 Help

The *Help* menu contains the following items: ***On Version...***, ***On Icons...***, and ***Index...***.

Clicking on ***Version...*** will give you the current version number and other information about **Wx**. The Version window is shown on page iii of this manual.

Clicking ***Help on Icons...*** pops up a window displaying the icons and a brief description of what they do.

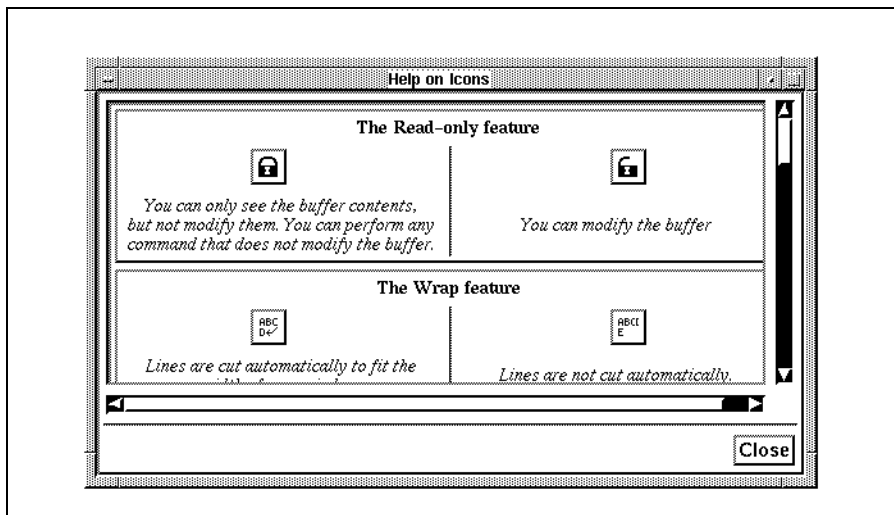


Figure 3.12: The Icon help box

3.19.1 The Help Index

Clicking on *Index...* displays a help box which gives information on **Wx** topics, actions, and a list of functions.

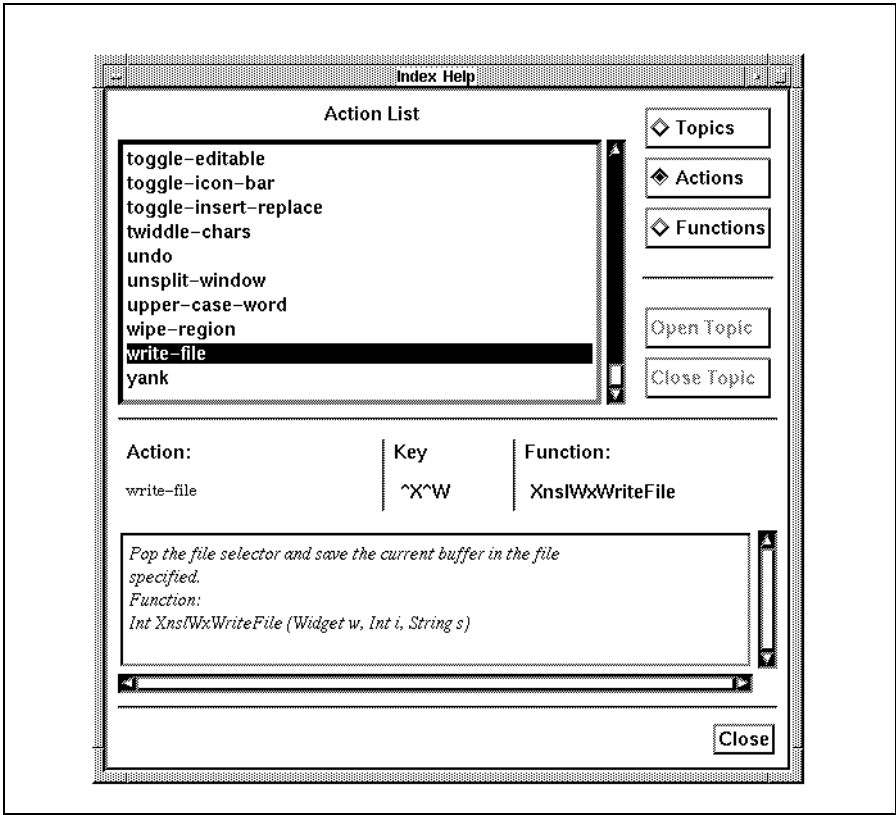


Figure 3.13: The help window for the write-file command

- In the **Topics** list, for instance, clicking on **Character Insertion**, then **Open Topic**, brings up a window with a list of commands involving the insertion of data into the buffer. If you select any command name in this list, a description of that command will be displayed.
- The **Actions** item displays an alphabetical index of **Wx** actions, with information on each.
- The **Functions** item displays an alphabetical list of **Wx** functions.

3.20 Quitting Wx

The *Exit Server* command (**^X^Z**) closes all the currently open windows and quits the server. If you select this command without having saved modifications made in files you are editing, **Wx** pops up a warning box telling you how many windows have been modified and asking you to confirm whether you want to quit the server without saving.

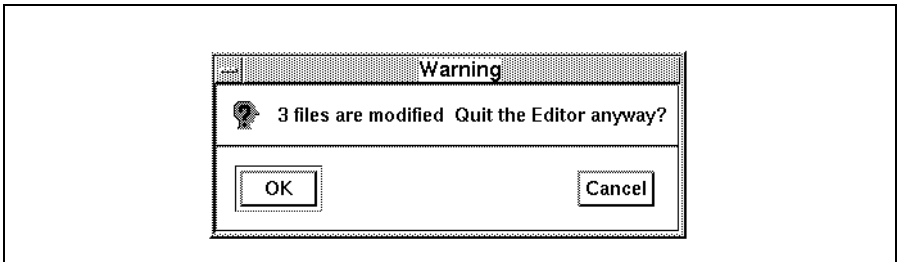


Figure 3.14: The “file modified” warning

If you click on **OK**, **Wx** closes all the windows and exits. If you click on **Cancel**, the *Exit Server* action is cancelled.

- To close the current **Wx** window while keeping the **Wx** server available, select *Close Window* (**^X^C**) in the **File** menu.
- To close the current window and exit the server, select *Exit Server* (**^X^Z**) in the **File** menu.

3.21 Keyboard-Only Commands

Most of the editing commands of **Wx** are accessible with a mouse or from the menu, but there are a few things you can do only from the keyboard. You can easily add these commands to a menu if you wish. See Chapter 6, Configuring **Wx**, for details. A complete list of all **Wx** commands is given at the end of this chapter. There is an alphabetical list of all **Wx** commands in Appendix A at the end of this manual. You can also access on-line Help on all **Wx** commands.

3.21.1 Keyboard Command Symbols

We mentioned the keyboard command symbols to the right of menu options. These symbols indicate how to give the command from the keyboard. They have the same effect as clicking on the command in the menu. The **^** stands for the Control key. Other commands use the Escape key as modifier.

For instance, the **Copy** command can be given by successively pressing **Esc** and then pressing **w**. It is important to remember these two differences in the way the modifiers **^** and **Esc** function:

- with **^**, it doesn't matter if the letter is upper or lower case; with **Esc**, it does matter
- with **^**, you press the **Control** key and the following letter key simultaneously. With **Esc**, you type the keys in succession.

3.21.2 Delete-Text Commands

Wx has a number of commands for deleting segments of various sizes: characters, words, lines, regions.

- To delete selected *text* use **Cut** in the **Edit** menu (or **^W**).
- To delete the *character* immediately preceding the insertion point, press the **Backspace** key.
- To delete the preceding *word*, press the **Escape** key, then **^H**. More accurately, what this does is delete all characters to the left of the insertion point until the first space character, so that if the insertion point is in the middle of a word you won't delete the whole word.
- To delete forward from the insertion point to the next space: **Esc d**.
- To delete the current character, press **^d**.

3.21.3 Metacommands

There are a few useful commands that affect other commands, which we can call *metacommands*.

- The abort command **^G**, is used in a dialog box to *get out* of a command: it cancels the current command, the one that produced the dialog box in the first place.
- The command **Esc num** multiplies the next command you give: for instance **Esc 5 Return** will type five carriage returns, **Esc 5 f** will type a sequence of five **f**s, and, since **^N** moves the insertion point to the *next* line, **Esc 5** will move it down 5 lines.
- The command **^C** repeats the last *command* given. For instance, if you type **Esc 5 ^N**, the insertion point will move down five lines. If you then type **^C**, it will move down five more lines.

3.21.4 The Load Tags Command

The *load tags* feature can be of considerable help to programmers by allowing the user to locate any function quickly – even if you don’t remember the name of the file in which it is located. What’s more, it automatically loads the file and positions the cursor on the correct line. You must first create a tags file with the UNIX command **ctags(1)**. When you wish to locate a function, simply open the Tags Selection Box with the *load tags* command **^X^T**. Select the function name from the alphabetical list, and press the **OK** button.

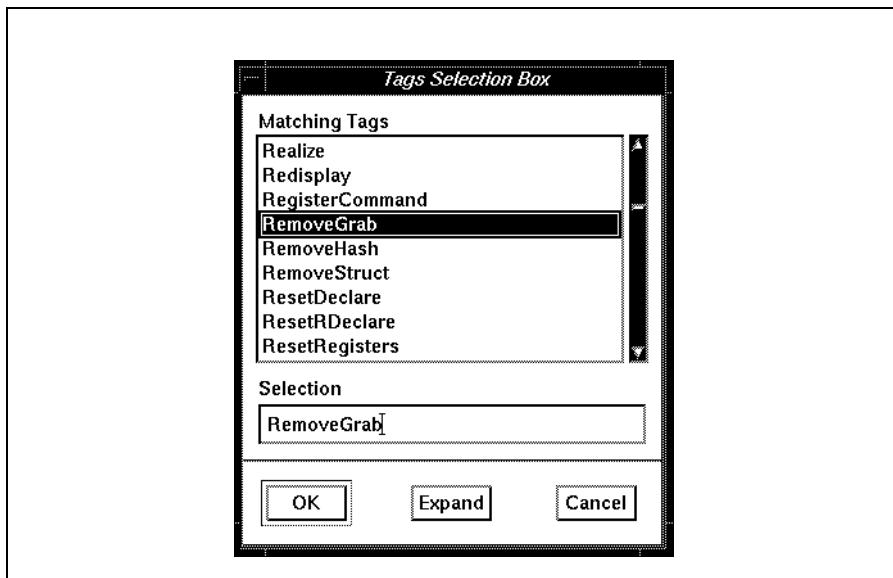


Figure 3.15: The Tags Selection Box

You can search a long list more easily by using the name expansion feature. Simply type the first few characters of the function and press **Expand**, or press the **Tab** key if the expand resource is set to **True**. **Wx** will display a list of any functions that begin with those characters. Select the function you are searching for from the list and press **OK**. The corresponding file will be opened automatically with the cursor positioned on the correct line. You can access the tags file through the environment variable **WXTAGS**, or through the “tags” file in the current directory.

3.21.5 Reload File

The reload-file command (**^X^O**) re-reads the file that you are editing and positions the cursor back on the line on which you started. This is a simple method to delete your latest typing and to return to the beginning.

3.21.6 The Load Macro File Command

The load-macro command (**^X^L**) pops up the file selector box for entering a macro file name. The macros contained in the file will be added to the cascading menu in the **Macro** menu. See section 5.4 for more information about macros.

3.21.7 List of Keyboard-Only Commands

Here are the keyboard commands we have mentioned in this chapter, plus a few additional useful ones. See the appendix for a complete listing.

Esc <	move to beginning of file
Esc >	move to end of file
Esc <i>num</i> ^G	go to line number <i>num</i>
^N	move to next line
^P	move to previous line
^A	move to beginning of current line
^E	move to end of current line
Esc f	move forward one word
Esc b	move backward one word
^B	move backward one character
^F	move forward one character

Esc ^H	delete preceding word
Esc d	delete next word
^R	delete current line (including newline character)
^K	delete from cursor to end of line
^D	delete current character
^H	delete previous character
BS	delete previous character

^G	cancel current dialog box (does not apply to file selector)
Esc num	repeat next command <i>num</i> times
^C	repeat last command

3.22 Summary of All Wx Commands

The following pages list all commands and operations currently available in **Wx**, with their equivalent key combinations, if any. They are grouped by category. Appendix A contains an alphabetical list of **Wx** commands. Command information can also be obtained on-line from the **Help** menu.

3.22.1 Manipulating Files

change-directory	^X^D	Pop the file selector to switch to a new working directory.
insert-file	^X^I	Pop the file selector and insert the file specified.
load-file	^X^V	Pop the file selector and load the file specified.
load-tags	^X^T	Read in the file where the named tag is declared
reload-file	^X^O	Reload the current file and go to the previous line
save-file	^X^S	Save the buffer in the current file.
write-file	^X^W	Pop the file selector and save the current buffer in the file specified.

3.22.2 Inserting Characters

insert-accent	' ^ ~ "/	If the preceding character is: a e i o u y c n s, then this character is translated to the corresponding special character according to the ISO 8859-1 norm <n> times.
insert-char	a l l	Insert the character entered <i>n</i> times.

insert-octal	^Q	Insert the character whose octal code is entered n times.
new-line	^M	Insert a line-feed character at the insertion point.
new-line-indent	^M	Insert a line-feed character and position the insertion point at the current indentation level on the next line.
new-line-indent-c	^M	Insert a line-feed character and position the insertion point at the correct indentation level on the next line in Galactic-C mode.
open-line	Esc o	Open n empty lines above the insertion point.
open-space	^O	Open n empty lines forward of the insertion point.
tabulation	^I	Insert n tab characters.
toggle-insert-replace	osfInsert	Toggle between Insert and Replace mode.

3.22.3 Deleting, Copying and Clearing Text

clear-buffer	^X^K	Clear the buffer.
copy-end-line	Esc k	Copy the end of the current line to the killstack.
copy-region	Esc w	Copy the region (the text between the mark and the insertion point) to the kill stack.
delete-char	^D	Delete n characters.
delete-end-line	^K	Delete from the insertion point to the end of the line.
delete-line	^R	Delete n lines and push them onto the kill stack.
pop-kill-stack	^ZK	Remove the top item of the kill stack.

rubout-char	^H	Delete n preceding characters.
set-mark	^@	Set the mark at the current insertion point.
show-kill-stack	^Zk	Show the Wx clipboard in a window.
wipe-region	^W	Delete the region.
yank	^Y	Insert the n^{th} killed item at the insertion point.

3.22.4 Keyboard Macros

exec-script	Esc Esc	Execute the current face Script in the current Wx window
load-macro	^X^L	Pop the file selector and load the macro file specified.
register-script	Esc e	Register the current Script.
save-macro	^X^M	Pop the file selector and save macros in the file specified.
show-script	^Zs	Show the Script Window.
start-macro-learn	^X(	Begin learning a new macro.
stop-macro-learn	^X)	End learning a macro.

3.22.5 Matching Parentheses

flasher	)}	Insert ‘)’ or ‘}’ n times. Move the cursor briefly to the corresponding opener if there is one.
match-closing-brace	Esc}	Find the closing brace matching the next opening brace.
match-closing-par	Esc)	Find the closing parenthesis that matches the next opening parenthesis.

match-opening-brace	Esc{	Find the opening brace that matches the last closing brace before the insertion point.
match-opening-par	Esc(	Find the opening parenthesis that matches the last closing parenthesis before the insertion point.

Note: These matching commands are available only in certain modes. In the standard distribution of **Wx** they are available in C and C++ modes. To add those to other modes, see Chapter 5.

3.22.6 Moving About Text

backward-char	^B	Move insertion point backward n characters.
backward-line	^P	Move insertion point up n lines.
backward-page	Esc v	Move up n screenfulls.
beginning-of-buffer	Esc <	Move to the beginning of the buffer.
beginning-of-line	^A	Move to the beginning of the current line.
end-of-buffer	Esc >	Move to the end of the buffer.
end-of-line	^E	Move to the end of the current line.
exchange-point-mark	^X^X	Exchange the positions of the insertion point and the mark.
forward-char	^F	Move insertion point forward n characters.
forward-line	^N	Move insertion point down n lines.
forward-page	^V	Move down n screenfulls.
goto-line	Esc<n> ^G	Move to the n^{th} line of the buffer.

3.22.7 Searching and Replacing Text

backward-ci-search	Esc I	Search backward from the insertion point for the next or n^{th} occurrence of the character string regardless to upper and lower case.
backward-regexp-search	Esc R	Search backward from insertion point for the preceding or n^{th} occurrence of the regular expression given as search pattern.
backward-search	Esc R	Search backward from insertion point for the preceding or n^{th} occurrence of the search string.
forward-ci-search	Esc i	Search forward from the insertion point for the next or n^{th} occurrence of the character string without regard to case.
forward-regexp-search	Esc S	Search forward for the next or n^{th} occurrence of the regular expression given as search pattern.
forward-search	^S	Search forward from the insertion point for the next or n^{th} occurrence of the character string.
query-replace	Esc %	Replace the search string by the replace string on user prompt.

3.22.8 Selecting Text With The Mouse

extend-adjust	Bt n1Mov	Select text in the direction of the mouse pointer.
---------------	-----------------	--

extend-end	Btn1Up	The highlighted text becomes the primary selection.
select-line	Btn1Dwn (3)	Select the line under the mouse pointer.
select-word	Btn1Dwn (2)	Select the word under the mouse pointer.
selection-start	Btn1Dwn	Set the mark and begin text selection.
stuff	Btn2Dwn	Copy text from the X selection to the insertion point.

3.22.9 Scrolling The Window

curline-top	Esc ^R	Redisplay the current window with the current line at the top.
scroll-down	^X^N	Scroll the current window down n lines.
scroll-left	^X^B	Scroll the current window left n columns.
scroll-right	^X^F	Scroll the current window right n columns.
scroll-up	^X^P	Scroll the current window up n lines.
scroll-window	Btn3Mov	Scroll the window vertically keeping the insertion point at its current position.

3.22.10 Commands on Words

backward-word	Esc b	Move backward n words.
capitalize-word	Esc c	Switch case of first character of the current word.
delete-word	Esc d	Remove n words forward of the insertion point.
forward-word	Esc f	Move forward n words.

lower-case-word	Esc l	Change all characters between the insertion point and the end of the word to lower case.
rubout-word	Esc ^H	Delete <i>n</i> preceding words.
upper-case-word	Esc u	Change all characters between the insertion point and the end of the word to upper case.

3.22.11 Window Manipulation

create-window	^Z^W	Create a new editing window.
delete-window	^X^C	Delete the current window.
next-window	^Zn	Go to the next window in a circular list.
previous-window	^Zl	Go to the previous window in a circular list.
select-pane	Bt n1Down	Select the active pane.
split-window	^Z^S	Split the current window in two panes, both showing the same buffer
unsplit-window	^Z^U	Close the second window pane.

3.22.12 Other Commands

compile-buffer	Esc ^C	Compile the current buffer and show errors if any.
display-line	Esc ^L	Toggle the display of current line and current column.
exec-command	Esc ^A	Execute a Unix command.
expand-alias	^I	Insert the key word corresponding to the prefix entered.
line-counter	^X=	Display the number of the current line and the total number of lines in the buffer.

multiplier	Esc <N>	Specify <i>n</i> , the numerical argument prefixed to the upcoming command.
print-region	Esc ^P	Print the region
quit-the-editor	^X^Z	Exit the wxd server.
repeat-command	^C	Repeat the last command.
set-font	^Z^F	Pop the file selector to change the font for the current window.
toggle-editable	^Z^T	Disable/Enable buffer modify permission.
toggle-icon-bar	^Z^D	Toggle the display of the Icon Bar.
twiddle-chars	^T	Swap the two characters preceding the insertion point.
undo	^X^U	Undo the previous editing command.

CHAPTER 4: **Wx** Modes

Wx comes with a set of predefined language modes that are available from the **Modes** menu. Modes are working environments that make your editing task easier, whether you are programming, formatting, or just editing text. They allow you to achieve complex effects, such as replacing an abbreviation by an expression, setting up matching brackets and positioning the cursor between them, or generating a complex formatting instruction, with just a few keystrokes.

In the current standard release of **Wx**, the *Modes* menu offers the following choices:

- Fundamental: the basic text editing mode; it cannot be changed or removed.
- LaTeX: provides editing conveniences for documents written in LaTeX.
- Language modes: provide editing conveniences for the programming languages listed below:
- C
- C++
- Fortran
- Ada
- Pascal
- Cobol
- Pl1
- PostScript

In the mode specification, the associated file suffix is given, so that, if the current file has that suffix, the **Wx** window is automatically switched to that mode. A mode can also be specified for the window by selecting the mode in the menu. Mode selection affects only the current window, so you can have different **Wx** windows on your screen with different modes. What modes are available is determined in `.wxrc`, the **Wx** startup file, and you can change this by adding new modes or modifying those that already exist. In this chapter we will show how this is done. See Chapter 6 for more about `.wxrc`.

4.1 Defining Modes

A mode definition starts with the keyword **defmode**, followed by a Tab separator, the mode name, space, an opening brace. It ends with a closing brace. Anything enclosed within the two braces is part of the mode definition. For example,

```
defmode    Galactic-C {
```

starts the mode definition for Galactic-C. Anything contained between the opening brace and the closing brace at the end of the mode definition is part of the galactic-C mode definition. The syntax for **defmode** is:

```
defmode<Tab>mode_name {  
/* mode definition */  
}
```

4.1.1 Keywords

The following keywords are recognized by **Wx** when interpreting the **.wxrc** file within a mode description.

suffix Specifies the file name suffixes which switch the window to the mode being specified. For example, to switch to Galactic-C mode for all **.c** and **.h** files, the standard distribution specifies

```
suffix:*. [ch]
```

in the Galactic-C mode definition.

compile Specifies the UNIX command that is issued when the Compile Buffer item is selected in the menu (or the corresponding key combination is typed). The following parameters are associated with this command.

%b or **%B** stands for the name of the current file in the window, without its suffix (i. e. the full pathname of the file that is to be compiled, with the suffix removed).

%e or **%E** stands for the suffix of the current file (i. e. the file that is being modified).

The typical **compile** command specified in the standard distribution for several of the programming languages is:

```
compile: make %b
```

but you could specify a command such as:

```
compile:indent %b.%e %b.ind; lint -ahxu %b.ind
```

error Specifies a set of regular expressions used to recognize all the error messages produced by the compiler, in order to display them in the error window pane and list the line number in which a given error occurred. Each of the regular expressions specified, except the last one, must be terminated by **"\CR"**

The parameter, **%L** associated with this keyword stands for the line number. The error specification used in the standard distribution for Galactic-C mode is:

```
error:  ^. *, line %L:. *$\n
        ^. *:%L:. *$\n
        [ ][ ]*%L |. *$
```

tabsize Specifies the size of the tabulation for the mode. Tabsize is a resource and can be specified in the **.Xdefaults** file of your home directory. If a tabsize is specified for a mode, however, it supersedes the resource specification given in the **.Xdefaults** file when the window is switched to that mode. With **tab-size**, you specify the number of blanks a **<Tab>** will produce, in the current font. For example, to have a tabulation of four blanks in the current font, you would specify:

```
tabsize:4
```

translations Specifies new translations for the **XnsIWix** widget used by **Wx**. Any translations specified are added in **override** mode; this means that if, for the event specified, an action was defined for the widget in its original translation table, the new action is executed in addition to the one already defined.

Reminder: a translation table, in Motif, sets up the association of events triggered by the user to actions (or procedures) that the widget can execute. To associate actions to events, you can use translations.

For mouse events and non-ASCII keys, only translations can be used to specify actions.

For keyboard actions, using standard ASCII keys and modifiers, the keyword **keytable** can be used instead of translations. If the same key combinations are specified using both translations and keytable definitions, the translations supersede the keytable associations.

In the example below:

```
translations: <Key>F3:      copy-region()\n               <Key>F2:      wipe_region()\n               Ctrl<Key>s: toggle-editable()
```

typing:

- **F3** causes the current selection to be placed in the kill stack;
- **F2** causes the current selection to be placed in the kill stack and deleted from the buffer
- **^S** causes the window to toggle between edit mode and read-only mode. This could be specified in the keytable instead of the translations table.

No translations are defined for the modes in the standard **Wx** release since, in the distribution `.wxrc` file, only ASCII keys and their modifiers are bound to actions, and this is better done using the `keytable` keyword.

keytable Specifies the association of keyboard combinations with **Wx** actions. This is the only way a combination such as `^X^V` can be defined, since translations do not support such combinations. The parameters listed below can be used in association with keystroke definitions.

- **^Control**– Indicates the Control modifier key
- **\$E** Esc – Indicates the Escape key
- **\$S** Space – Indicates the Space key
- **\$U** Up Arrow – Indicates the up arrow key
- **\$D** Down Arrow – Indicates the down arrow key
- **\$L** Left Arrow – Indicates the left arrow key
- **\$R** Right Arrow – Indicates the right arrow key

To specify `^` in a key combination, meaning the circumflex accent and not the Control modifier key, you must enter `\^`, the `\` being used as an escape character. To specify the `\` key, you must enter `\\`.

An example of a `keytable` specification taken from the Galactic-C mode definition in the distribution `.wxrc` is:

```
keytable: ^I    expand-alias
```

alias Specifies the words that are to give rise to an expansion, and what the expansion is to be when the **expand-alias** action is triggered. Examples of alias definitions for the Galactic-C mode are:

```
main    main (argc, argv) char **argv; \i{\+ \* \-} \i
or
{       {\+ \* \-}
```

4.1.2 Rules for Alias Definition

Alias definitions follow the rules below.

1. The alias must be separated from its expansion by a **Tab** character and nothing else.
2. Each alias definition except the last in a block of alias definitions must terminate in a backslash character followed by a carriage return: because the list of aliases is considered as a single user-defined resource, it should appear to be on a single

line; the `\` serves as an escape character for the CR or LF end of line characters.

3. Which alias is activated is determined by **Wx** reading down from the top of the alias list. Thus, for instance, if the line

```
char char\  
precedes the line  
case case\  
then typing a c and Tab will produce char, and to get case you must type at  
least ca and Tab.
```

Thus, each line but the last in an alias definition must have the format
`<alias><separator><extension>\`

and the last line must have the format
`<alias><separator><extension>`

where `<separator>` must be the **Tab** character, `<alias>` must be a string not containing space characters, and `<extension>` can contain any of the metacharacters in the following table:

Metacharacter	Description
<code>\ t</code>	insert tab character
<code>\ n</code>	go to beginning of next line
<code>\ i</code>	go to next line at current indent position
<code>\ +</code>	go to next line and increment indentation by one
<code>\ -</code>	go to next line and decrement indentation by one
<code>\ *</code>	cursor position after abbreviation is expanded
<code>\\</code>	backslash character: <code>\</code>
<code>#</code>	comment (rest of line is ignored).

4.1.3 Rules for Keytable Definition

The rules for keytables are similar to those for alias definitions:

1. The key combination being defined must be separated from the resulting action by a **Tab** character and nothing else.
2. Each keytable definition except the last in a block of keytable definitions must terminate in a backslash character followed by a carriage return: the list of keytable definitions is considered as a single user-defined resource, it should appear to be on a single line; the `\` serves as an escape character for the CR or LF end of line characters.

4.1.4 Format for Mode Definitions

The first line of a mode description must indicate the name of the mode, preceded by `defmode`.

For example,

```
defmode C++ {
```

This line is mandatory. The mode name specified here will appear in the **Modes** menu when the mode is loaded.

Then, specify the various aspects of the mode, using the keywords described above. For each of the keywords, the syntax is:

```
<keyword>:<Tab><specification>
```

Within a mode definition, the keywords and the associated specification may appear in any order. The Mode definition must be terminated by a closing brace. Refer to the `system.wxrc` file in the standard distribution for examples of mode definitions.

CHAPTER 5: Wx Keyboard Macros

Macros should be used any time a sequence of keyboard commands must be repeated several times. When you create and register a macro, **Wx** generates an editable script written in the FACE language. Your macros can also be saved in a file for future use. Macro definition is so easy that you should do it whenever you plan to execute a command sequence more than once.

5.1 Creating Macros

You can create a macro as follows.

1. Select **Begin Macro** from the Macro menu, or type **^X(**.
2. Enter a name for the macro in the dialog box.

The name must be composed of letter characters only—no numbers, punctuation marks, etc. may appear. The message “**Begin macro collection**” will appear at the bottom of your window, and the icon of a cassette tape will appear in the lower left corner of all **Wx** windows open on your screen.

3. Execute the sequence of keyboard commands.

In the present version of **Wx**, only keyboard commands (i.e. no mouse actions), are accepted in macros. Also, the commands **split-window**, **split-window**, **compile-command**, **undo**, **change-directory**, **query-replace**, **exec-command**, **show-kill-stack** and **load-tags** cannot be included.

4. Select **End Macro** from the menu or type **^X)**.

When you do so, the cassette tape icon will disappear from the bottom of your window, and the message “**End macro collection**” will appear.

5. Press **Esc e** to register the macro script.

Two new items will appear at the top of the Macro menu: **Execute macro** and **Edit macro**. These cascading menus, which appear in all of your current **Wx** windows, list the macros you’ve created during the current **Wx** session, or have loaded from your macro file. Select **Execute macro** to choose a macro to run. Select **Edit macro** to pop up the **Script** window to edit the macro’s FACE script. See section 5.2 for details.

6. Choose **Save macro...** and enter a macro file name in the selector box.

If you wish to use the macro in future **Wx** sessions, you must save it in a file. Macros from this file can then be loaded into any **Wx** window by using the **load-macro** command **^X^L**. See section 5.4 below.

Once the macro is created or loaded, you can run it by issuing the **execute-macro** command, **^X^E**, or by selecting it from the *Execute macro* cascading menu. You can then immediately re-run it by typing **^C**, which is the general “repeat last command” key combination.

5.2 Registering a Macro Script

When you create a macro, an editable script written in the FACE language is generated by **Wx**. This script is inserted in the “script buffer” and can be modified. At this point, the macro is not inserted in the **Wx** macro list and its name does not appear in menus.

The FACE script must be registered by typing **Esc e** so that the macro will appear in the macro list. If a macro with the same name already exists, the older version will be overwritten by your new macro. When you register the macro’s script, the macro name is added to the cascading menus *Execute macro* and *Edit macro*. If your macro contains a syntax error, an error message will appear in the window in which you launched the **Wx** session.

5.3 Executing and Editing Macros

The definition of the Macro menu in `.wxrc` includes the lines

```
Execute macro, macro-exec  
Edit macro, macro-edit
```

This creates links so that **Wx** will display two extra menu items when macros are created or loaded. The two extra menu items are:

- *Execute macro*. When you select this item from the *Macro* menu, a cascading menu opens, allowing you to select the macro you wish to run. You can repeat the macro by typing **^C**. If your macro contains a syntax error, an error message will appear in the window in which you launched the **Wx** session when you try to execute the macro.
- *Edit macro*. When you select this item from the **Macro** menu, a cascading menu opens. Select a macro and the **Wx** Script window will open, in which you can edit the macro’s script.

To edit a macro script you need to open the **Script window** in one of the following ways:

- Choose *Edit Macro* from the **Macro** menu, and select a macro name.
- Choose *Show Script Window* from the **Special** menu in the **Wx** Manager Box. Then choose the macro from the cascading list under *Edit macro*.

- Using the command **show-script**, ^Zs, in your **Wx** window. The Script window that opens contains the script of the current macro. You can then choose the macro to be edited from the cascading list under *Edit macro*.

When you modify a script, you must *register* it by pressing **Esc e**, then *save* it in its new form. If you don't register and save it, a dialog box will pop up the next time you try to edit a macro, asking if you wish to register and save the modified version. If you exit the server without doing so, the modified version will be lost. If your modification contains a syntax error, the macro will not be registered, and an error message will appear in the window in which you launched the **Wx** session.

5.4 Loading and Saving Macros

The macro file that is loaded when **Wx** is started is specified in the **.wxrc** file, using the **loadmacro** keyword, and specifying the full pathname of the macro file. This is explained further in Chapter 6, Configuring Wx.

Once you are in **Wx**, you can use the **load-macro** load-macro command (^X^L) to load macros from a file. You will be asked to specify the file's name in the File Selector box. The macros contained in the file will be added to the cascading menu. You can thus create macros, store them in a file, and have them available at any time.

To store macros in a file, select *Save Macro...* (^X^M) in the **Macro** menu. You will be asked to specify the destination file in a File Selector box. The file content is reset before the macros are saved, so that the file contains only the newly saved macros—any previous data in the file is destroyed. A warning box will alert you.

Note: If the current macro has not been registered, a warning box will alert you. Click on **OK** and **Wx** will register this macro, and save it with the macro list in the file.

5.5 Using FACE in Wx Scripts

In **Wx**, the scripts describing macros are written in the FACE language. FACE is a language whose syntax is very close to that of C. It was designed specifically to program the dynamic part of a user interface in **XFaceMaker**, and not as a general-purpose language. It is used in **Wx** to write and edit macro scripts and all the features valid in **XFaceMaker** are valid in **Wx**.

In contrast to ordinary C, FACE scripts are not compiled while in the development phase but are *interpreted*, so that the typical compile-execute cycle is not required. The FACE interpreter reads and executes one statement at a time until the end of the script is reached.

5.6 Keywords in FACE

The following identifiers are reserved for use as keywords, and may not be used otherwise:

```
if    else    while    for
script
self  parent  toplevel  root
global  local  auto
type
function  return
define    include
```

5.6.1 Special Characters

The following characters have special meanings in FACE:

```
; . , : $ @ " { } ( ) < > = ! + - * /
```

5.6.2 Comments

The characters `/*` introduce a comment, which terminates with the characters `*/`. Comments do not nest.

5.6.3 Arithmetic Expressions

Arithmetic expressions have the same syntax as in C. The following operators are supported:

Assignment operator	=				
Unary operator	-				
Arithmetic operators	+	-	*	/	
Logical operators	&&		!		
Relational operators	==	!=	<=	>=	< >

All arithmetic operations work on integer or floating-point operands. If any of the operands is a floating-point value, then the operator performs a floating-point operation, but the other operands are *not* converted automatically to a floating-point type—you must use the function `citof` to convert them.

Expressions can be parenthesized and operator precedence is the same as in C.

5.6.4 Include Files

A file may be included in a FACE script using the `include` keyword, followed by a file name: i.e.,

```
include "project.face";
```

The included file may contain any FACE statements. The including script is parsed as if the contents of the file had been included in the script.

5.7 Basic objects

FACE has the following types of objects:

- constants
- variables
- special variables
- widgets
- widget resources
- X properties

5.7.1 Object Types

The type of objects in FACE is specified by a String whose content is arbitrary; i.e., you may define new variable types if you so desire, as long as they satisfy the type checking and/or type conversion done by FACE when a variable is used.

The usual type strings are defined in `<Xm/Xm.h>`. They correspond to frequently used widget resource types such as “Int”, “String”, “Pointer” etc. In addition FACE has two special types, called “None” and “Any”.

- None

If when attaching a function, i.e. in **FmAttachFunction**, you declare the type of an argument as “None” you are in effect telling FACE that the argument type for this argument is not to be checked whenever the function is called.

- Any

If, when calling a function whose arguments have all been specified, you use a type “Any” for one of the arguments, the type of that argument is not checked for that call.

Unlike C, when variables are used or declared in FACE, their type need not be specified. If the type of a variable is not specified when the variable is declared, the type of the variable is determined each time it is assigned and it takes the type of the function or widget resource to which it is assigned.

For example:

```
x1 = self:x;
```

self:x is a widget resource. Its type is Dimension. With this assignment, **x1** is defined as being of type Dimension.

```
a = f(0);
```

The type of the variable **a** is the type of function **f**'s return value. FACE checks the type of variables in two instances; when assigning a variable, and when passing it as a function parameter.

5.7.2 Constants

Integer and float constants follow the ordinary C conventions; they may be positive or negative. Character strings are placed within double quotes. For example:

```
"Hello world"
```

The quote character may be escaped with \ (backslash) as follows

```
"Awaking he said \"Hello world\"."
```

A string which does not include blanks does not *have* to be placed between quotes but it is good practice to always quote strings: if a variable with the same name exists, an unquoted string is assumed to refer to the variable even though you may have intended it to be a string.

5.7.3 Variables

Variable names are composed of letters and digits. They must start with a letter and are case sensitive. Thus **X** and **x** do not refer to the same variable.

All FACE variables have the same size, that of a word or pointer, whatever their type. For example, a **char** variable, which would normally occupy a byte, is sign-extended if necessary to the size of a word.

There are local and global variables. Local variables are known only in the script in which they appear. Thus a local variable may be known in one script but not in another. You need not declare local variables. They are automatically declared the first time they are used.

```
Message = "Warning: Not found!";  
Size = 10;
```

Here, the variable **Message** has been declared implicitly the first time it has been set. It is of type String. The variable **Size** is also implicitly declared when first used and is of type Int.

Variables may be declared anywhere in the program, as long as they are not used before they are declared. Variables can allocate storage, e.g. for a `sprintf` buffer:

```
a="";  
sprintf(a, "...", ...);
```

although it is better to use **malloc** explicitly.

It is possible to declare a local variable explicitly if it must be used before it is first set. Local variables are declared using the **local** keyword:

```
local i;
```

This declares a local variable named **i**, initializes its value to zero and sets its type to “None”. For example:

```
local toggle;
if(toggle){
    toggle = false;
    doThis();
}else{
    toggle = true;
    doThat();
}
```

the variable **toggle** is tested before it is set so must be declared **local**.

Local variables declared outside FACE functions are *not* automatic variables, i.e., they are not allocated dynamically each time the script is invoked. They are like static variables in a C function: they are initialized to 0 when the script is parsed and retain their value from one call to another.

Local variables declared inside FACE functions *are* automatic, and are allocated and set to 0 each time the function is called.

Global variables can be referenced by all scripts of the same application. They are declared using the **global** keyword:

```
global button_pressed;
```

Global variables must be declared in all the scripts in which they are used. The parser creates the variable and associated storage the first time it encounters the declaration.

It is possible to declare the type of a FACE **local** or **global** variable, either when the variable is declared, with the following syntax:

```
[local|global] <type> <name>;
```

or when a local variable is first set, using:

```
<type> <name> = <value>;
```

When a variable is declared with a type, then the variable is *statically typed*, which means that you cannot assign an expression of a different type to it. If you do so, an error message is given. Static typing is useful for error checking, and also for helping the FACE compiler find the type of a variable; for instance in floating-point expressions.

The keyword **auto** is equivalent to **local**.

5.7.4 Defined Constants

A constant may be defined with the **define** keyword. Defined constants are like variables, but their value cannot be changed. Their value is either an integer, a string or a floating-point constant, and is specified after the constant name:

```
define <name> <value>;
```

Examples:

```
define MAX_WIDGETS 10;
define PI 3.142;
define TITLE "Project X";
```

A defined constant may be defined in several places, but it may not be defined with different values. A defined constant can be used in a script even if it has been declared in another script, provided that the script where the constant has been defined is parsed first.

The FACE interpreter built into **Wx** has a number of predefined constants:

- The constants **true**, **True**, **false**, and **False** are defined globally. Their type is “Boolean”.

```
true  = True  = 1
false = False = 0
```

5.7.4.1 Widgets

The FACE naming scheme uses the . (dot) operator, which lets you travel from a widget to one of its children. In addition, FACE recognizes some *reserved* names for naming widgets. It is not useful in **Wx**; the only feature really needed is the use of the keyword **self**.

self refers to the widget from which the script is called.

parent refers to the parent of **self**.

toplevel refers to the child shell of the ApplicationShell (root) which is an ancestor of **self**.

root refers to the ApplicationShell which is an ancestor of **self**.

5.7.5 X Properties

FACE lets you refer to X properties to store data. X properties are variables associated to an X window. For that reason, they can be used only for widgets that have already been realized.

Since such a variable is associated to the particular instance of the widget for which you define it, its value is independent of any other. Thus, the value of a property with the same name defined for a different instance may be different.

It is legal in FACE to use properties for gadgets but, since gadgets do not have an X window, the property of a gadget is associated to the gadget's parent. Thus, for gadgets, you must be careful not to use the same property for two gadgets with the same parent, or for a gadget and its parent, if you want the variables to be independent. To define a property, an assignment statement is required, such as:

```
widget:<property_name> = self:x;
```

where:

widget is the *realized* widget for which you are defining a property.

property_name is the name of the property; i.e., the name you will use to refer to that variable.

With such a statement, you have defined a new variable name, and its type: the type of the property is the type of the resource or of the variable which is on the right side of the assignment statement. For example, you can write:

```
self:<area> = self:height * self:width;
```

This defines a variable (property) called **area**, of type **Int**, defined as the area of the associated widget. If **self** is a gadget and not a widget, however, **area** is a single variable, whose value is the area of parent after the second statement is executed.

5.8 Type Casters

Type casting functions are available in FACE. They do not *convert* a variable, i.e., they do not change its value but they *cast* the variable type to another type to satisfy FACE type checking requirements. These functions return their argument, which may be of any type, in the appropriate type. The functions are: **Any**, **CursorPosition**, **Float**, **Image**, **Int**, **MenuWidget**, **Pixmap**, **Pointer**, **String**, **Widget**, **Window**, and **XmBackgroundPixmap**.

FACE type checking does not consider all Motif types as distinct. For example, types “Position”, “Dimension”, and “Boolean” which are different in Motif are all equivalent in FACE. See Appendix B for a complete description of these type casters.

5.8.1 Declaring New Type Casters

New type casting functions can be declared using the keyword **type** followed by the variable type to which you want to be able to cast. After such a declaration, the new type casting function exists. For example:

```
type NewType;  
n = NewType(x);
```

5.8.2 Type Conversion

An explicit type conversion function is available for converting a string to the type specified as one of the arguments:

```
ConvertString(widget w, String s1, String target_type);
```

where:

w is the widget used for additional arguments, if required, as in the XtConvert function.

s1 is the string you want to convert.

target_type is the string specifying the type you want to convert to.

return type is type “Any”.

For example:

```
ConvertString(w, "10", "Int");
```

returns the value 10 of type “Any”.

5.9 Statements

A FACE script is a sequence of statements. Each statement is either:

- an assignment
- a control structure
- a function call

and is terminated with a **;** (semicolon).

5.9.1 Control Structures

FACE provides the **if**, **else**, **while** and **for** control structures. These have the same syntax and semantics as in C. The following examples illustrate the control structures.

Example 1:

```
local Loudness;
if (Loudness == 0)
    ResetVolume();
else if (Loudness < 10) {
    Slide = Slide + Slide/10;
    SetVolume(Slide);
} else
    SetVolume(Slide);
```

Example 2:

```
Running = 1;
ResetTable();
while (Running) {
    Key = GetTable();
    Running = equal(Key, 1);
}
DisplayCurrentTable();
```

5.9.2 Built-in Functions

The FACE language has built-in functions from the C library, the X library, and the Xt library. For a complete list of these functions and their documentation, refer to Appendix C.

5.9.3 Wx Editing Functions

In **Wx**, each keystroke has an associated function. In addition, **Wx** has several special functions designed to give the editor more power. These special functions are as follows:

- XnslWxInsert
- XnslWxRedraw
- XnslWxRefresh
- XnslWxNameToBuffer
- XnslWxCreateMark
- XnslWxDeleteMark
- XnslWxGotoMark
- XnslWxGetInsertPosition
- XnslWxGetLastPosition
- XnslWxSetInsertPosition
- XnslWxCopyBuffer
- XnslWxReplace
- XnslWxSetMark
- XnslWxGetRegion
- XnslWxGetCurChar

See Appendix D for a complete list of **Wx** functions and their associated description.

5.9.4 Definition of Functions in the FACE Language

It is possible to define C-like functions in FACE, with the following syntax:

```
function Int MyFunction(String s, Boolean b)
{
    if ( b == True )
        return(strlen(s));
    else
        return(0);
}
```

FACE function declarations can be nested, that is, a function can be declared inside another function, but the scope of functions is always the whole script (functions are declared globally). Recursive calls are also allowed.

FACE functions can be declared anywhere in a FACE script. They can be forward-declared by using the `function` keyword followed by the return value and the function name; e.g.,

```
function Int MyFunction();
```

The **return** keyword has the same semantics as in C, and must be called with one argument, of any type, or with no arguments (i.e. with empty parentheses). It can be used in FACE functions, but also anywhere else in a FACE script, in which case it will abort the whole FACE script (the return value is then ignored). No type checking is done between the type of a FACE function and the argument of **return**.

In FACE functions, local variables are automatic: a new instance of the variable is created every time the function is called.

The scope of variables is defined as follows:

- The scope of a local variable declared outside a function is the whole script, *not including* functions defined in the script;
- The scope of a global variable declared outside a function is the whole script, *including* functions defined in the script (i.e., it is not necessary to declare the global variable again inside the function);
- The scope of a local variable declared inside a function is the function itself, not including other functions defined inside the function;
- The scope of a global variable declared inside a function is the function itself, and the rest of its surrounding script.

5.10 Floating-Point Variables

Single-precision floating-point values are implemented in FACE. A floating-point constant is of the form `i.f`, where `i` and `f` are decimal numbers. Floating-point values work as in C, with the following restrictions:

- Only the first six arguments of a function call can be floating-point values;
- There is no automatic conversion between floating-point and integer (or other) values; values can be explicitly converted using the FACE functions **cftoi** and **citof**, which convert a float to an integer and vice-versa; the definition of the conversion (truncating or rounding) is the same as the definition of the casting operator in C, and depends on the platform;
- An arithmetic operator is assumed to be a floating-point operator if one of its arguments is a float, but the other argument is not converted automatically to float: the converter **citof** must be used if needed; if a non-float argument is passed to a float operator, a parse error occurs;
- Floating-point arguments are passed correctly to functions only in the following cases:
 - The argument is a floating-point literal,
 - The argument is a statically typed variable (or a *defined* constant) of type **Float**,
 - The argument is a call to a function returning a float.

The last case makes it possible to force **Wx** to pass an argument as a float, even if the function was not declared as taking a float, by using the **Float** type caster. For example, **printf** accepts arguments of unspecified type: to pass a float, you must cast it to **Float**, as follows:

```
pi = 3.1415926;
    printf("pi=%f\n", Float(pi));
```

Another solution is to use a statically typed variable, or a constant:

```
define pi 3.1415926;
Float pi2 = pi/2.0;
    printf("pi=%f, pi2=%f\n", pi, pi2);
```

If a FACE-attached application function returns a float, it must be declared as returning a value of type **XtRFloat** in **FmAttachFunction**; otherwise the FACE interpreter will get an incorrect return value.

5.11 A Macro Example.

The following macro is used for simple prototyping of functions. It is given in the standard distribution.

Applying the macro to this function declaration:

```
static void
setFontAction (w, ev, s, ns)
Widget      w;
XKeyEvent   *ev;
String      *s;
int         *ns;
```

gives the following result

```
static void
#ifdef __STDC__ && __STDC__
setFontAction (Widget w, XKeyEvent * ev, String *s,int *ns)
#else
setFontAction (w, ev, s, ns)
Widget      w;
XKeyEvent   *ev;
String      *s;
int         *ns;
#endif
```

This macro does not prototype functions that have arrays or function pointers as parameters, or whose list of parameters extends over several lines.

```
function None
simple_proto (Widget wix)
{
    String    ptr;
```

The function is named **simple_proto**, and it returns nothing.

```
Widget    message;
message = parent.parent.parent.diag_zone.\ ;
message:labelString = " ";
```

Get the widget that displays messages in the **Wx** window. It does not have a name; its name field is represented by backslash and blank.

```
if (wix:readOnly) {
    message:labelString = "The buffer is read only";
    return ();
```

```
}
```

Check to see if the buffer is read-only.

```
result = XnslWxForwardSearch (wix, 1, "(");
if (!result) {
    message:labelString = "no ( found";
    XnslWxRedraw (wix);
    return ();
}
```

Assuming that the cursor is ahead of the function name, this verifies that there is a list of arguments.

```
XnslWixBeginningOfLine(wix);
result = XnslWixDeleteLine(wix, 1);
XnslWixYank(wix, 1);
XnslWixBackwardLine(wix, 1);
XnslWxForwardSearch (wix, 1, "(");
XnslWixCreateMark (wix, 2);
```

The function's name and its arguments are put into the kill stack. A mark and insertion point are placed at the beginning of the argument list.

```
beg_line = self:currentLineNumber;
XnslWixBackwardChar (wix, 1);
XnslWixSetTheMark (wix);
result = XnslWxForwardSearch (wix, 1, "{");
if (!result) {
    message:labelString = "no { found";
    XnslWixGotoMark (wix, 2);
    XnslWixDeleteMark (wix, 2);
    XnslWixRedraw (wix);
    return ();
}
end_line = self:currentLineNumber;
XnslWixExchangePointMark (wix);
```

Next, the first and last lines of the argument declaration list are found.

```
last = False;
while (!last) {
```

Go through the list and insert the argument types.

```
XnslWxReForwardSearch (wix, 1, "[a-zA-Z_]");
cur_line = self:currentLineNumber;
if (cur_line >= end_line) {
    XnslWxGotoMark (wix, 2);
    XnslWxInsert (wix, "void");
    XnslWxRefresh (wix);
    message:labelString = "no args found, void is inserted";
    last = True;
```

Check the beginning of the arguments and verify that there are some; otherwise, insert **void**.

```
    } else {
        XnslWxBackwardChar (wix, 1);
        XnslWxSetTheMark (wix);
        XnslWxCreateMark (wix, 0);
        res_arg = XnslWxReForwardSearch (wix, 1, "[,)]");
        if (!res_arg) {
            message:labelString = "no sep found";
            XnslWxGotoMark (wix, 2);
            XnslWxDeleteMark (wix, 0);
            XnslWxDeleteMark (wix, 1);
            XnslWxDeleteMark (wix, 2);
            XnslWxRefresh (wix);
            return ();
        }
        XnslWxBackwardChar (wix, 1);
        cur_char = XnslWxGetCurChar (wix);
        if (cur_char == 41)
            last = True;
```

Mark the beginning of the argument and search for the next argument separator. If the end of the declaration is reached, prepare to exit the loop.

```
XnslWixForwardChar (wix, 1);
XnslWixCreateMark (wix, 1);
XnslWixBackwardWord (wix, 1);
XnslWixBackwardChar (wix, 1);
XnslWixForwardWord (wix, 1);
str = XnslWixGetRegion (wix);
XnslWixSetTheMark (wix);
if (!str) {
    message:labelString = "Error empty arg";
    XnslWixRefresh (wix);
    return ();
}
```

Mark the position of the next argument. Backup over the argument to copy it into the string str.

```
len = strlen ("[* ]") + strlen (str);
len = len + strlen ("[ ]*[;,]") + 1;
find = malloc (len);
sprintf (find, "[* ]%s[ ]*[;,]", str);
result = XnslWixReForwardSearch (wix, 1, find);
free (find);
cur_line = self:currentLineNumber;
```

Find the argument declaration by using a regular expression.

```
if (!result || cur_line > end_line) {
    XnslWixGotoMark (wix, 0);
    XnslWixInsert (wix, "int ");
    free (str);
} else {
```

If the declaration is not found in the declaration list, the argument is not declared: it is an `int`.

```
len = strlen ("*[ *]*") + strlen (str) + 1;
find = malloc (len);
sprintf (find, "[ *]*%s", str);
XnslWixSetTheMark (wix);
result = XnslWxReBackwardSearch (wix, 1, find);
free (find);
cur_line = self:currentLineNumber;
if (!result || cur_line <= beg_line) {
    ptr = String (0);
    XnslWixExchangePointMark (wix);
} else {
    XnslWixSetTheMark (wix);
    XnslWixForwardWord (wix, 1);
    XnslWixBackwardWord (wix, 1);
    ptr = XnslWixGetRegion (wix);
}
free (str);
```

Once the declaration is found, use a regular expression to check if the argument is a pointer.

```
XnslWixBeginningOfLine (wix);
result = XnslWxReForwardSearch (wix, 1, "[a-zA-Z_]");
XnslWixBackwardChar (wix, 1);
XnslWixSetTheMark (wix);
result = XnslWxReForwardSearch (wix, 1, "[,;]");
XnslWixBackwardWord (wix, 2);
XnslWixForwardWord (wix, 1);
str = XnslWixGetRegion (wix);
```

Get the type of the argument.

```
    XnslWixGotoMark (wix, 0);
    XnslWixInsert (wix, str);
    free (str);
    if (ptr) {
        XnslWixInsert (wix, ptr);
        free (ptr);
    }
    XnslWixInsert (wix, " ");
}
}
```

Insert the type and the pointers if there are any.

```
XnslWixDeleteMark (wix, 0);
XnslWixGotoMark (wix, 1);
XnslWixDeleteMark (wix, 1);
}
```

Go to the next argument.

```
XnslWixBeginningOfLine(wix);
XnslWixInsertChar (wix, 1, 35);
XnslWixInsert(wix, "if (defined(__STDC__)&& __STDC__)\n");
XnslWixForwardLine(wix, 1);
XnslWixBeginningOfLine(wix);
XnslWixInsertChar (wix, 1, 35);
XnslWixInsert(wix, "else\n");
XnslWixYank(wix, 1);
XnslWxPopKillStack(wix);
XnslWxForwardSearch(wix, 1, "{");
XnslWixBeginningOfLine(wix);
XnslWixInsertChar (wix, 1, 35);
XnslWixInsert(wix, "endif\n");
XnslWixDeleteMark (wix, 0);
XnslWixDeleteMark (wix, 1);
XnslWixDeleteMark (wix, 2);
XnslWixRefresh (wix);
}
simple_proto (self);
```

The `ifdef` block is inserted and the marks are removed.

CHAPTER 6: Configuring Wx

Wx is delivered in a standard configuration on which this manual is based. However, its client-server architecture makes it easy to change many **Wx** characteristics so as to adapt them to a particular user's desires and needs. Among the aspects of **Wx** that can be changed are the following:

Look and Feel: resources and command line options can be used to change the look and the behavior of **Wx**.

Menus: the menus available, their names, the order in which they are displayed in the menu bar, and the items in each menu can be entirely redefined.

Modes: new modes can be created, and loaded on startup; existing modes can be modified.

Macros: macros can be defined and stored in a file. This file can be loaded on startup or with the **load-macro** command in the **Macro** menu.

6.1 Creating Your Own .wxrc File

If your home directory does not contain a `.wxrc` file, you can create one. Copy the file `/usr/lib/X11/system.wxrc` to your home directory, change its name to `.wxrc`, then reconfigure.

6.2 Resources

Resource settings can be specified using the standard X resource-setting files, specifically the `.Xdefaults` file in your home directory. This overrides resource settings specified in the file

`/usr/lib/X11/appdefaults/wx`

This file, which is part of the **Wx** standard distribution should not be changed, or deleted. The resources it sets are required for the proper operation of the editor. The `.Xdefaults` file can specify new settings for the resources described below. Resources specified using the command line options override resource settings in the `.Xdefaults` file.

6.2.1 Standard Motif Widgets in Wx

Wx uses the following OSF/Motif widgets. By modifying resources for these widgets you can modify the look of your **Wx** windows.

- ApplicationShell
- XmDialogShell
- XmFileSelectionBox
- XmForm
- XmPanedWindow
- XmRowColumn
- XmScrolledWindow
- XmScrollBar
- XmPushButton
- XmSeparator
- XmLabel
- XmCascadeButton

You can find a list of resources for these widgets in your Motif documentation, e. g. The OSF/Motif Programmer's Reference.

6.2.2 The Wx Window Widget Hierarchy

- Widget **Form** is an XmForm
 - Widget **menubar** is an XmMenuBar
 - Widget **File** is an XmCascadeButtonGadget
 - Widget **Edit** is an XmCascadeButtonGadget
 - Widget **Search** is an XmCascadeButtonGadget
 - Widget **Window** is an XmCascadeButtonGadget
 - Widget **Modes** is an XmCascadeButtonGadget
 - Widget **Macro** is an XmCascadeButtonGadget
 - Widget **Misc** is an XmCascadeButtonGadget
 - Widget **Help** is an XmCascadeButtonGadget
 - Widget **diag_zone** is an XmForm
 - Widget **mode** is an XmLabelGadget

- Widget **name** is an `XmLabelGadget`
- Widget **modif** is an `XmLabelGadget`
- Widget **macro_learn** is an `XmLabel`
- Widget **line_col_count** is an `XmLabelGadget`
- Widget¹ is an `XmLabelGadget`
- Widget **sep** is an `XmSeparatorGadget`
- Widget **row_button** is an `XmRowColumn`
 - Widget **read_only** is an `XmLabelGadget`
 - Widget **word_wrap** is an `XmLabelGadget`
 - Widget **insert_replace** is an `XmLabelGadget`
 - Widget **bin_edit** is an `XmLabelGadget`
 - Widget **block_cursor** is an `XmLabelGadget`
 - Widget **show_line** is an `XmLabelGadget`
 - Widget **auto_backup** is an `XmLabelGadget`
 - Widget **lock_ro** is an `XmLabelGadget`
 - Widget **ask_foreign** is an `XmLabelGadget`
- Widget **pane** is an `XmPanedWindow`
 - Widget **wix-sw** is an `XmScrolledWindow`
 - Widget **edit** is an `XnslWix`
 - Widget **vsb** is an `XmScrollBar`
 - Widget **hsb** is an `XmScrollBar`
 - Widget **separator** is an `XmSeparatorGadget`

6.2.3 The XnslWix Widget Resources

Wx also uses a proprietary widget, the **XnslWix** widget. The **XnslWix** widget class is built from the **Core** and **Primitive** classes. The class name is **XnslWix**. The **XnslWix** widget class defines the new set of resources listed in the table below.

Name	Type	Default
blockCursor	Boolean	False

1. This widget has no name.

Name	Type	Default
cursor	Cursor	“xterm”
font	FontStruct*	“Fixed”
motionCursor	Cursor	“hand2”
readOnly	Boolean	False
tabsize	Int	8
wrap	Boolean	False
replace	Boolean	False
selectedForeground	Pixel	calculated
selectedBackground	Pixel	calculated
showLines	Boolean	False
lineNumberColor	Pixel	“black”
undoDepth	Int	20

blockCursor Specifies which of two shapes is used for the insertion point cursor: if True, a rectangle (or block); if False, a vertical bar.

cursor Specifies the name of the cursor used as mouse pointer. The default is the “xterm” or I-beam pointer.

font Specifies the current font name.

motionCursor Specifies the name of the cursor used as motion pointer, i. e. when the text is scrolled up or down using the *Custom* button. The default motion cursor is the hand.

readOnly Set if the buffer is not to be modified; i.e., is a read-only buffer.

selectedForeground Specifies the color of the foreground pixels for selected text.

selectedBackground Specifies the color of the background pixels for selected text.

tabsize Specifies the width of tab, with the units equalling the width of the letter ‘n’ in the current font; e. g. value 8 makes tab-length equal to the length of ‘nnnnnnnn’.

wrap If set, lines which are longer than the window’s width are displayed on as many more lines as necessary. The truncation occurs at the following separation characters: **<Space>** **<Tab>** and period (.).

replace If set, each character typed replaces the one under the block cursor, or to the right of the line cursor. Otherwise, each character typed is inserted without

replacing any other character.

showLines If set, line numbers are displayed on the left of the window.

lineNumberColor Specifies the color used to display line numbers.

undoDepth Specifies the depth of the undo. The depth cannot be less than 10.

6.2.4 The XnsLWix Widget translations

The default translations specify the actions associated with mouse button press and release events and with focus, enter, and leave events. All the keypress events are not handled directly in the translations. The **keyboard-action** action is used to execute the **Wx** action given as the argument. It is also used to look in a “translation table” called the keytable, for the action associated with the key combination typed. The standard translations can be rebound to different keys in the `.Xdefaults` file. The following table lists the default translations for the XnsLWix widget:

Key	Bound to
Shift<Btn1Down>:	extend-selection()
<Btn1Down>:	select-pane() selection-start()
<Btn1Up>:	extend-end()
<Btn1Down>,<Btn1Motion>(1+):	extend-adjust()
<Btn1Down>,<Btn1Up>:	extend-end()
<Btn1Down>(2):	select-word()
<Btn1Down>(3):	select-line()
<Btn2Down>:	select-pane() stuff()
<Btn3Down>:	select-pane() scroll-window()
<Btn3Motion>:	scroll-window()
<Btn3Up>:	scroll-window()

Key	Bound to
Shift ~Meta ~Ctrl <Key>Tab:	prev-tab-group()
Ctrl ~Meta ~Shift <Key>Tab:	next-tab-group()
Ctrl Shift ~Meta <Key>Tab:	prev-tab-group()
~Meta ~Shift ~Ctrl <Key>osfDown:	traverse-down()
~Meta ~Shift ~Ctrl <Key>osfUp:	traverse-up()

Key	Bound to
Shift ~Meta ~Ctrl <Key>osfLeft:	traverse-left()
Shift ~Meta ~Ctrl <Key>osfRight:	traverse-right()

Key	Bound to
Meta ~Ctrl ~Shift<Key>osfCopy:	keyboard-action(stuff)
Meta Ctrl ~Shift<Key>osfInsert:	keyboard-action(stuff)
~Meta ~Ctrl Shift<Key>osfInsert:	keyboard-action(yank)
~Meta Ctrl ~Shift<Key>osfInsert:	keyboard-action(copy-region)
~Meta ~Ctrl Shift<Key>osfDelete:	keyboard-action(wipe-region)
Meta ~Ctrl ~Shift<Key>osfBackSpace:	keyboard-action(undo)
~Ctrl ~Shift ~Meta<Key>osfCopy:	keyboard-action(copy-region)
~Ctrl ~Shift ~Meta<Key>osfCut:	keyboard-action(wipe-region)
~Ctrl ~Shift ~Meta<Key>osfPaste:	keyboard-action(yank)
~Shift ~Meta<Key>osfPageUp:	keyboard-action(backward-page)
~Shift ~Meta<Key>osfPageDown:	keyboard-action(forward-page)
~Ctrl ~Shift ~Meta<Key>osfEndLine:	keyboard-action(end-of-line)
~Ctrl ~Shift ~Meta<Key>osfBeginLine:	keyboard-action(beginning-of-line)
Ctrl ~Shift ~Meta<Key>osfEndLine:	keyboard-action(end-of-buffer)
Ctrl ~Shift ~Meta<Key>osfBeginLine:	keyboard-action(beginning-of-buffer)
~Ctrl ~Shift ~Meta<Key>osfClear:	keyboard-action(clear-buffer)
~Ctrl ~Shift ~Meta<Key>osfUndo:	keyboard-action(undo)
~Ctrl ~Shift ~Meta<Key>osfInsert:	toggle-insert-replace()
<Key>:	keyboard-action()

6.2.5 The wxd Program Resources

The wxd program supports the resources listed below. These can be set in your .Xdefaults file.

expand The default setting for this resource is false. When expand is true, the file and directory names in the file selector, and tags, can be expanded using the Tab or the Esc key. This ressource is incompatible with the motifRecommendation resource.

directory Specifies the default directory for Wx windows. The default setting for

the **directory** resource is the current directory, i. e. the directory from which **wxd** is launched.

home The default setting for this resource is false. When **home** is true, the default directory for **Wx** windows is the user's home directory.

machine The default setting for this resource is false. When **machine** is true, the text in the **Wx** rectangle, or the manager box, includes the name of the host from which **wxd** was launched. For instance, you could have text such as:

Click for a Wx window on randolphe

if **randolphe** is the hostname.

displayLinesColumns The default setting is false. When **displayLinesColumns** is true, the current position of the insertion pointer is displayed in the **Wx** window message area, as a line and column number.

clickgeometry Specifies the position at which the **Wx** rectangle is displayed. The geometry is specified in the standard X format:

+x +y (where **x** and **y** are numerical values) specifies the position relative to the top left of the screen whose coordinates are 0, 0.

-x -y (where **x** and **y** are numerical values) specifies the position relative to the bottom right of the screen whose coordinates are 0, 0.

You can of course specify **+x -y** or **-x +y**.

menu The default setting for this resource is true. When **menu** is false, the menus are not displayed in the **Wx** windows.

wish If true, disables the "click for a Wx window" button.

automaticBackUp The default setting for this resource is true. When **automaticBackUp** is true, a copy of the previous contents of the edited file is saved before writing the edited file on the disk.

matchReadOnly The default setting for this resource is false. When **matchReadOnly** is true, the buffer permission is set according to the file permission.

confirmRemoteFile The default setting for this resource is false. When **confirmRemoteFile** is true, if you request a file from a remote system via the **Wx** program, the server will ask for confirmation before creating attempting to access the file.

displayIconBar The default setting for this resource is false. When **displayIconBar** is true, the icon bar is not displayed on the left of the window.

verbose The default setting is false. When true, **Wx** displays information about the program's initialization.

useManagerBox The default setting is false. When true, the Manager Box appears when you launch wxd.

useMiniBuffer The default setting is false. When true, and commands are launched with the accelerators, input is expected in the mini buffer instead of dialog boxes.

motifRecommendation The default setting is false. When true, the application follows Motif recommendations for keys.

warpPointer The default setting is false. When true, the pointer is warped between windows when you use the following actions:

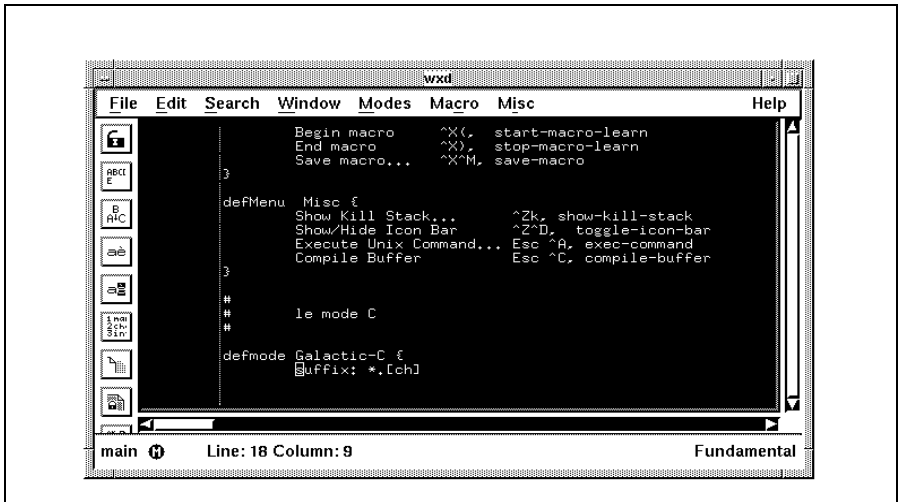
- next-window
- previous-window
- load-file
- load-tags

For example, you can insert the following lines in your `.Xdefaults` file:

```
wxd*expand:      True
wxd*machine:      True
```

6.2.6 Example of a `.Xdefaults` File

The `.Xdefaults` file below shows resource settings applied to some of the OSF/Motif standard widgets, to the XnsIWix widget and to **wxd** itself. The result is shown in the following figure.



```

wxd*font:8x13
wxd*fontList:*helvetica-bold-r*14*
!
wxd*XnsLWix.background: black
wxd*XnsLWix*blockCursor: true
wxd*XnsLWix*showLines: true
wxd*XnsLWix*lineNumberColor: yellow
!
wxd*expand:true
wxd*machine:true
wxd*displayLinesColumns: true
wxd*clickgeometry: -0+0
!
wxd*XmList.backgroundPixmap : 50_foreground
wxd*XmText.backgroundPixmap : 50_foreground
wxd*XmScale*backgroundPixmap : 50_foreground
wxd*XmRowColumn.backgroundPixmap : 50_foreground
wxd*XmPanedWindow.backgroundPixmap : 50_foreground
wxd*XmForm.backgroundPixmap : 50_foreground
wxd*XmPushButton.backgroundPixmap : 50_foreground
wxd*XmArrowButton.backgroundPixmap : 50_foreground
wxd*XmToggleButton.backgroundPixmap : 50_foreground
wxd*XmDrawnButton.backgroundPixmap : 50_foreground
wxd*XmSelectionBox.backgroundPixmap : 50_foreground
wxd*XmLabel.backgroundPixmap : 50_foreground

```

6.3 Command Line Options

The command line options listed below can be used to change some of the **Wx** resource settings at the time the **wxd** server is launched. Note that resource settings using the command line options supersede resource settings specified in the **.Xdefaults** file.

-expand Set the **expand** resource to true. The default setting for this resource is false. When **expand** is true, file and directory names in the file selector can be expanded using the **Tab** or the **Esc** key. This option is incompatible with the **-mr** option.

-directory Specifies the default directory for **Wx** windows. The default setting for the **directory** resource is the current directory, i. e. the directory from which **wxd** is launched. The default directory for a window can be re-specified, using the **Change Directory** option in the menu.

-home Set the **home** resource to true. The default setting is false. When **home** is true, the default directory for **Wx** windows is the user's home directory. The default directory for a window can be re-specified, using the **Change Directory** option in the menu.

-machine Set the **machine** resource to true. The default setting for this resource is false. When **machine** is true, the text in the **Wx** rectangle includes the name of the machine from which **wxd** was launched. For instance, you could have text such as:

Click for a Wx window on randolphe
if randolphe is the machine name.

-dlc Set the **displayLinesColumns** resource to true. The default setting is false. When **displayLinesColumns** is true, the current position of the insertion pointer is displayed in the **Wx** window message area, as a line and column number.

-clickgeometry Set the **clickgeometry** which specifies the position at which the **Wx** rectangle is displayed. The geometry is specified in the standard X format:

+x +y (where **x** and **y** are numerical values) specifies the position relative to the top left of the screen whose coordinates are 0, 0.

-x -y (where **x** and **y** are numerical values) specifies the position relative to the bottom right of the screen whose coordinates are 0, 0.

You can of course specify **+x -y** or **-x +y**.

- menu** Set the `menu` resource to false. The default setting is true. When `menu` is false, the menus are not displayed in the **Wx** windows.
- button** Disable the “click for a Wx window” button. This sets the `wish` resource to true.
- ab** Set the `automaticBackUp` resource to false. The default setting is true. When `automaticBackUp` is false, a copy of the previous contents of the edited file is not saved before writing the edited file on the disk. To change this, click on the automatic file backup icon.
- mro** Set the `matchReadOnly` resource to true. The default setting is false. When `matchReadOnly` is true, the buffer permission is set according to the file permission. This can be changed by clicking in the match read-only icon on the icon bar.
- crf** Set the `confirmRemoteFile` resource to true. The default setting is false. When `confirmRemoteFile` is true, if you request a file from a remote system via the `wx` program, the server will ask for confirmation before creating a new window and loading the file.
- dib** Set the `displayIconBar` resource to false. The default setting is true. When `displayIconBar` is false, the icon bar is not displayed on the left of the window. This can be changed with the function **toggle-icon-bar**.
- verbose** Set the `verbose` resource to true. The default setting is false. When `verbose` is true, information is displayed about the program’s initialization.
- managerbox** Set the `useManagerBox` resource to true. The default setting is false. When `useManagerBox` is true, the Wx Manager Box is displayed when you launch `wxd`.
- mr** Set the `motifRecommendation` resource to true. The default is false. When `motifRecommendation` is true, the application follows Motif recommendations.
- warp** Set the `warpPointer` resource to true. The default is false. When `warpPointer` is true, the pointer is warped between windows when you use the following actions:
 - next-window
 - previous-window
 - load-file
 - load-tags
- umb** Use mini buffer instead of dialog boxes for data input and alert messages associated to commands triggered with accelerators.

An example command line could be: **wxd -dlc -home -machine**

6.4 Environment variable

6.4.1 The TMPDIR environment variable

In order to provide several levels of **Undo** in each of the currently opened windows, **Wx** needs to create temporary files in which to save each of the buffers and the additional information required. The default directory in which these files are created is **/tmp**.

If it is set, the **TMPDIR** environment variable specifies the directory in which these temporary files are to be created. The size of the disk space which must be available for these files depends on the number of windows that are open and on the buffer size of each of the windows.

6.4.2 The NSLHOME environment variable

If the installation has not been done in the standard directories, you must use the **NSLHOME** environment variable to access the **Wx** default configuration files.

6.5 Configuring .wxrc

.wxrc is the **Wx** configuration file. When you launch **wxd**, the server looks for and reads your personal version of **.wxrc**, stored in your home directory. If it does not find one there, it opens the file

`/usr/lib/X11/system.wxrc`

or

`NSLHOME/lib/x11/system.wxrc` .

In **.wxrc**, you can specify the menus and their items, the list of modes and their content, as well as the macro file to load.

6.5.1 Keywords

The following keywords are recognized by **Wx** when interpreting the **.wxrc** file.

loadmacro Load the macro file whose full pathname is given as argument. The Tab separator placed between the keyword and the file pathname is mandatory. For example:

`loadmacro<Tab>/tmp/my_macros`

defmenu Start a menu definition. The syntax for this keyword is given in the next section.

defmode Start a mode definition. The syntax for this keyword is given in section 4.1.

print Specify the UNIX command to be executed when the **print-region** command is issued. The associated parameter is **%s** which represents the buffer to print, and replaces the file name in the UNIX command. For example, you could specify:

```
print:lpr %s
```

or, using the UNIX pipe,

```
print:cat %s | lpr
```

The file is removed after the command is completed.

wxkeys Specifies the association of keyboard ASCII key combinations with **Wx** actions. This has the same syntax as the **keytable** mode definition keyword described in section 4.1.1. Key-to-action associations specified here apply globally to the **Wx** session. When a mode is active in a window, its keytable definition supersedes the **wxkeys** specification. Translations, however, supersede **wxkeys** specifications.

6.5.2 Rules for Mode and Macro Definitions

The rules for the definition of modes in **.wxrc** are explained in Chapter 4, **Wx Modes**. The methods for creating and storing macros are explained in Chapter 5, **Wx Macros**.

6.5.3 Rules for Menu Definition

A menu definition starts with the keyword **defmenu** , followed by the menu name and an opening brace. Anything between the opening brace and the corresponding closing brace is part of the menu definition. Within a menu definition:

- a line with nothing but a minus sign in it (-) indicates that there is to be a separator bar in the menu.
- **#** indicates a comment (the rest of the line is ignored)
- the menu command name (for instance *Split*) is separated from the keyboard equivalent text (for instance `<F7>`) by a single tab character, and this, in turn, is followed by a comma. The comma is mandatory even if there is no keyboard equivalent.
- the internal name of the command (for instance *split-window*) follows the comma, and is written in lower case with hyphen separators.

For example, the definition of the File menu in the standard distribution contains the following lines:

```
defmenu File {
```

```
Open...      ^X^V, load-file
Insert...    ^X^I, insert-file
Save         ^X^S, save-file
Save As...   ^X^W, write-file
Change Directory... ^X^D, change-directory
-
Close Window ^X^C, delete-window
-
Exit Server  ^X^Z, quit-the-editor
}
```

The order of menu blocks in `.wxrc` is the order in which menu names appear in the menu bar.

Non-alphanumeric characters in the menu command name are replaced by underscore (`_`) characters for the associated widget name. For example, the name associated to the menu command name ***Open...*** is “Open” followed by three underscore characters; i.e., `Open___`.

6.5.4 Key Combinations for Menu Items

For user comfort, the key combination text in a menu item must be in agreement with the keyboard combination which effectively triggers the action associated with the menu item. The equivalent key combinations for commands are hard-wired in the program.

In order to change the short-cut key combinations which are to trigger menu actions, you must define new key-to-action combinations using the `wxkeys` keyword described above. If, as a result of the new definitions, key combinations which were associated with menu items no longer are used, they should be associated with the `noop` action. If you do change the shortcut key combinations, don’t forget to change the corresponding text for the menu items concerned.

For example, if you wanted the *Open* menu command to be triggered by instead of and you wanted no action associated with , you would specify `wxkeys` as follows:

```
wxkeys: ^X^R      load-file\  
        ^X^V      noop
```

and you would redefine the File menu as follows:

```
defmenu File {  
    Open ...      ^X^R, load-file  
    Insert ...    ^X^I, insert-file  
    Save          ^X^S, save-file  
    Save As ...   ^X^W, write-file  
    Change Directory ... ^X^D, change-directory  
    -  
    Close Window  ^X^C, delete-window  
    -  
    Exit Server   ^X^Z, quit-the-editor  
}
```

You cannot use translations for these specifications, because translations do not support key combinations such as .

6.6 What Cannot Be Configured in .wxrc

You cannot specify default values, such as window size (this is always determined by **Wx** from your **.Xdefaults** file, the default mode (which is always Fundamental mode), or the font in which text is displayed when a file is loaded.

6.7 Internationalization

Wx follows the **XOpen** conventions for internationalization. On systems which support the XOpen General Language Support system, the distribution includes a default message file, called `wxd.msg`. A message file for a target foreign language can be created by translating the messages found in this file, then storing the result in a file whose path and name fit the XOpen system. For information on how to install `wxd.msg` and the associated message files, refer to your System Administrator Guide. For details on internationalization, refer to function **setlocale(3C)**.

6.8 Wx and Special ISO Latin-1 Characters

Most European languages other than English use special characters. These are defined in the ISO 8859-1 norm and **Wx** can be configured so as to accept these characters, depending on your system set-up.

6.8.1 ISO Keyboard

If you have an ISO keyboard which includes the special characters, you need only type them into **Wx**. You can check your keyboard arrangement by displaying the Keysyms, using

```
xmodmap -pk | more
```

6.8.2 wxkeys and the insert-accent Action

If your keyboard does not include the special characters, you can associate the **insert-accent** action with the accents in your **wxkeys** definition in **.wxrc**, or in the **keytable** of a mode definition if you wish to use the special characters in certain modes only. When the **insert-accent** action is set, typing a letter followed by its modifying accent produces the accented character. For example, typing **e'** produces **é**. Similarly, to obtain **ç**, you would enter **c/** and to obtain **ß**, you would enter **s/**. An example of a **keytable** of a mode which accepts all accented characters as well as the special characters **ç** and **ß** is given below.

```

      .
      (mode definition)
      .
keytable:      .  \^      insert-accent\
                '      insert-accent\
                '      insert-accent\
                "      insert-accent\
                ~      insert-accent\
                /      insert-accent
      .
      .
      .

```

6.8.3 Entering Octal Codes

You can also enter the special characters by entering **^Q** followed by the character's octal code.

APPENDIX A: The Wx Commands

backward-char (^B)	XnslWixBackwardChar
Move insertion point backward n characters.	
backward-ci-search (ESC I)	XnslWxCIBackwardSearch
Search backward from the insertion point for the next or n^{th} occurrence of the character string regardless to upper and lower case.	
backward-line (^P)	XnslWixBackwardLine
Move insertion point up n lines.	
backward-page (ESC v)	XnslWixBackwardPage
Move up n screenfulls.	
backward-regexp-search (ESC R)	XnslWxReBackwardSearch
Search backward from the insertion point for the preceding or n^{th} occurrence of the regular expression given as search pattern.	
backward-search (ESC R)	XnslWixBackwardSearch
Search backward from the insertion point for the preceding or n^{th} occurrence of the search string.	
backward-word (ESC b)	XnslWixBackwardWord
Move backward n words.	
beginning-of-buffer (ESC <)	XnslWixBeginningOfBuffer
Move to the beginning of the buffer.	
beginning-of-line (^A)	XnslWixBeginningOfLine
Move to the beginning of the current line.	
capitalize-word (ESC c)	XnslWixCapitalizeWord
Switch case of first character of the current word.	
change-directory (^X^D)	
Switch to a new working directory.	
clear-buffer (^X^K)	XnslWxClearBuffer
Clear the buffer.	
compile-buffer (ESC ^C)	

Compile the current buffer and show errors if any.		
copy-end-line	(Esc k)	XnslWixCopyEndLine
Copy the end of the current line to the kill-stack.		
copy-region	(Esc w)	XnslWixCopyRegion
Copy the region (the text between the mark and the insertion point) to the kill-stack.		
create-window	(^Z^W)	XnslWxCreateWindow
Create a new editing window.		
curline-top	(Esc ^R)	XnslWixCurLineTop
Redisplay the current window with the current line at the top.		
delete-char	(^D)	XnslWixDeleteChar
Delete <i>n</i> characters.		
delete-end-line	(^K)	XnslWixDeleteEndLine
Delete from the insertion point to the end of the line.		
delete-line	(^R)	XnslWixDeleteLine
Delete <i>n</i> lines and push them onto the kill-stack.		
delete-window	(^X^C)	XnslWxDeleteWindow
Delete the current window.		
delete-word	(Esc d)	XnslWixDeleteWord
Remove <i>n</i> words forward of the insertion point.		
display-line	(Esc ^L)	
Toggle the display of current line and current column.		
end-of-buffer	(Esc >)	XnslWixEndOfBuffer
Move to the end of the buffer.		
end-of-line	(^E)	XnslWixEndOfLine
Move to the end of the current line.		
exchange-point-mark	(^X^X)	XnslWixExchangePointMark
Exchange the positions of the insertion point and the mark.		
exec-command	(Esc ^A)	

Execute an Unix command.

exec-script (Esc Esc)

Execute the current face Script in the current Wx window.

expand-alias (^I) XnslWixAlias

Insert the key word corresponding to the prefix entered.

extend-adjust (Btn1Mov)

Select text in the direction of the mouse pointer.

extend-end (Btn1Up)

The highlighted text becomes the primary selection.

flasher ()}) XnslWixFlasher

Insert ‘)’ or ‘{’ n times. Move the cursor briefly to the corresponding opener if there is one.

forward-char (^F) XnslWixForwardChar

Move insertion point forward n characters.

forward-ci-search (Esc i) XnslWxCIForwardSearch

Search forward from the insertion point for the next or n^{th} occurrence of the character string regardless to upper and lower case.

forward-line (^N) XnslWixForwardLine

Move insertion point down n lines.

forward-page (^V) XnslWixForwardPage

Move down n screenfulls.

forward-regexp-search (Esc S) XnslWxReForwardSearch

Search forward for the next or n^{th} occurrence of the regular expression given as search pattern.

forward-search (^S) XnslWxForwardSearch

Search forward from the insertion point for the next or n^{th} occurrence of the character string.

forward-word (Esc f) XnslWixForwardWord

Move forward n words.

goto-line (Esc <n> ^G) XnslWixGotoLine

Move to the n^{th} line of the buffer.

insert-accent (' ' ^ " /) XnslWixInsertAccent

If the preceding character is: A a E e I i O o U u Y y C c N n s, then this character is translated to the corresponding special character according to the ISO 8859-1 norm n times.

insert-char (all) XnslWixInsertChar

Insert the character entered n times.

insert-file (^X^I) XnslWixInsertFile

Pop the file selector and insert the file specified.

insert-octal (^Q)

Insert the character whose octal code is entered n times.

line-counter (^X=)

Display the number of the current line and the total number of lines in the buffer.

load-file (^X^V) XnslWixLoadFile

Pop the file selector and load the file specified.

load-macro (^X^L)

Pop the file selector and load the macro file specified.

load-tags (^X^T)

Reads in the file where the named tag is declared.

lower-case-word (Esc l) XnslWixLowerCaseWord

Change all characters between the insertion point and the end of the word to lower case.

match-closing-brace (Esc }) XnslWixFindClosingBrace

Find the closing brace matching the next opening brace.

match-closing-par (Esc)) XnslWixFindClosingParenthesis

Find the closing parenthesis that matches the next open parenthesis.

match-opening-brace (Esc{) XnslWixFindOpeningBrace

Find the opening brace that matches the last closing brace before the insertion point.

match-opening-par (Esc() XnslWixFindOpeningParenthesis

Find the opening parenthesis that matches the last closing parenthesis before the insertion point.

multiplier (Esc <N>)

Specify *n*, the numerical argument prefixed to the upcoming command.

new-line (^M) **XnslWixNewLine**

Insert a line-feed character at the insertion point.

new-line-indent (^M) **XnslWixNewLineIndent**

Insert a line-feed character and position the insertion point at the current indentation level on the next line.

new-line-indent-c (^M) **XnslWixNewLineIndentC**

Insert a line-feed character and position the insertion point at the correct indentation level on the next line in Galactic-C mode.

next-window (^Zn) **XnslWxNextWindow**

Go to the next window in a circular list.

open-line (Esc o) **XnslWixOpenLine**

Open *n* empty lines above the insertion point.

open-space (^O) **XnslWixOpenSpace**

Open *n* empty lines forward of the insertion point.

pop-kill-stack (^ZK) **XnslWxPopKillStack**

Remove the top item of the kill-stack.

previous-window (^ZI) **XnslWxPreviousWindow**

Go to the previous window in a circular list.

print-region (Esc ^P)

Print the region.

query-replace (Esc %)

Replace the search string by the replace string on user prompt.

quit-the-editor (^X^Z)

Exit the wxd server.

register-script (Esc e)

Register the current Script.

reload-file (^X^O)	XnslWxReLoadFile
Reload the current file and go to the previous line	
repeat-command (^C)	
Repeat the last command.	
rubout-char (^H)	XnslWixRuboutChar
Delete <i>n</i> preceding characters.	
rubout-word (Esc ^H)	XnslWixRuboutWord
Delete <i>n</i> preceding words.	
save-file (^X^S)	XnslWxSaveFile
Save the buffer in the current file.	
save-macro (^X^M)	
Pop the file selector and save the current macro in the file specified.	
scroll-down (^X^N)	XnslWixScrollDown
Scroll the current window down <i>n</i> lines.	
scroll-left (^X^B)	XnslWixScrollLeft
Scroll the current window left <i>n</i> columns.	
scroll-right (^X^F)	XnslWixScrollRight
Scroll the current window right <i>n</i> columns.	
scroll-up (^X^P)	XnslWixScrollUp
Scroll the current window up <i>n</i> lines.	
scroll-window (Btn3Mov)	
Scroll the window vertically keeping the insertion point at its current position.	
select-line (Btn1Dwn (3))	
Select the line under the mouse pointer.	
select-pane (Btn1Down)	
Select the active pane.	
select-word (Btn1Dwn (2))	
Select the word under the mouse pointer.	

selection-start (Btn1Dwn)

Set the mark and begin text selection.

set-font (^Z^F)

Change the character font for the current window.

set-mark (^@)

XnslWixSetTheMark

Set the mark at the current insertion point.

show-kill-stack (^Zk)

Show the Wx clipboard in a window.

show-script (^Zs)

Show the Script Window.

split-window (^Z^S)

Split the current window in two panes, both showing the same buffer.

start-macro-learn (^X())

Begin learning a new macro.

stop-macro-learn (^X))

End learning a macro.

stuff (Btn2Dwn)

Copy text from the X selection to the insertion point.

tabulation (^I)

XnslWixTabulation

Insert *n* tab characters.

toggle-editable (^Z^T)

Disable/Enable buffer modify permission.

toggle-icon-bar (^Z^D)

Toggle the display of the Icon Bar.

toggle-insert-replace (osfInsert)

Toggle between Insert and Replace mode.

twiddle-chars (^T)

XnslWixTwiddleChar

Swap the two characters preceding the insertion point.

undo (^XU)

Undo the previous editing command.

unsplit-window (^Z^U)

Close the second window pane.

upper-case-word (Esc u) **XnslWixUpperCaseWord**

Change all characters between the insertion point and the end of the word to upper case.

wipe-region (^W) **XnslWixWipeRegion**

Delete the region.

write-file (^X^W) **XnslWxWriteFile**

Pop the file selector and save the current buffer in the file specified.

yank (^Y) **XnslWixYank**

Insert the n^{th} killed item at the insertion point.

APPENDIX B: FACE Typecasting Functions

Type cast functions are needed in FACE scripts. You can create your own using the **type** keyword.

Any(None)

Return type: **Any**

CursorPosition(Int)

Return type: **CursorPosition**

Float(None)

Return type: **Float**

Image(None)

Return type: **Image**

Int(None)

Return type: **Int**

MenuWidget(None)

Return type: **MenuWidget**

Pixmap(None)

Return type: **Pixmap**

Pointer(None)

Return type: **Pointer**

String(None)

Return type: **String**

Widget(None)

Return type: **Widget**

Window(None)

Return type: **Window**

XmBackgroundPixmap(None)

Return type: **XmBackgroundPixmap**

APPENDIX C: Built-in Face Functions

The following functions are available but not described here:

AddGrab

CallCallbacks

DestroyWidget

IsSensitive

malloc

MapWidget

MoveWidget

NewString

Popdown

Popup

RemoveGrab

SetKeyboardFocus

SetSensitive

Unmanagechild

UnmapWidget

NAME

atoi

SYNOPSIS

```
Int  atoi(  
    String  s)
```

DESCRIPTION

Call the C function atoi.

SCOPE

FACE scripts, the application

EXAMPLE

SEE ALSO

NAME

callback

SYNOPSIS

```
none callback(  
    Widget      w,  
    String      s,  
    Int         i)
```

DESCRIPTION

Call the XtCallCallbacks Toolkit function.

SCOPE

EXAMPLE

SEE ALSO

NAME

cftoi

SYNOPSIS

```
Int      cftoi (  
Float    f)
```

DESCRIPTION

Convert a floating-point number to an integer.

SCOPE

FACE scripts

EXAMPLE

Perform a calculation in Float and convert to Int.

```
Float ymin;  
Float ymax; Float y;  
Int v;  
min = self:sliderMin;  
max = self:sliderMax;  
v = cftoi((y - ymin) * citof(max - min) / (ymax - ymin));
```

SEE ALSO

citof

NAME

citof

SYNOPSIS

```
Float    citof (  
Int      i)
```

DESCRIPTION

Convert an integer to a floating-point number.

SCOPE

FACE scripts

EXAMPLE

Perform a calculation in Float and convert to Int.

```
Float ymin;  
Float ymax; Float y;  
Int v;  
min = self:sliderMin;  
max = self:sliderMax;  
v = cftoi((y - ymin) * citof(max - min) / (ymax - ymin));
```

SEE ALSO

cftoi

NAME

ConfigureWidget

SYNOPSIS

```
None      ConfigureWidget (  
          Widget          w,  
          Position        x,  
          Position        y,  
          Dimension       width,  
          Dimension       h,  
          Dimension       border_width)
```

DESCRIPTION

Move and resize widget *w*. *x* and *y* are the coordinates of the upper left corner.

SCOPE

EXAMPLE

SEE ALSO

NAME

dtoi

SYNOPSIS

Int dto

 i(
Pointer *dp*)

DESCRIPTION

Return the conversion of the *double* pointed to by *dp* into an integer.

SCOPE

FACE scripts application

EXAMPLE

```
parent.scale:value = dto
```

SEE ALSO

itod

atoi

NAME

equal

SYNOPSIS

```
Boolean  equalstring(  
    String  s1,  
    String  s2)
```

DESCRIPTION

Return *true* if strings *s1* and *s2* are identical.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

free

SYNOPSIS

```
None free(  
    String    s)
```

DESCRIPTION

This is the standard C function.

SCOPE

EXAMPLE

SEE ALSO

NAME

ftoi

SYNOPSIS

```
Int  ftoi(  
    Pointer  fp)
```

DESCRIPTION

Return the result of converting a float pointed to by *fp* into an integer.

SCOPE

EXAMPLE

SEE ALSO

NAME

itoa

SYNOPSIS

```
String    itoa(  
Int      i)
```

DESCRIPTION

Convert integer *i* into a statically allocated string.

SCOPE

EXAMPLE

SEE ALSO

NAME

itod

SYNOPSIS

```
None      itod(  
           Int      i,  
           Pointer  dp)
```

DESCRIPTION

Convert integer *i* into a `double` and store the result in the `double` variable pointed to by *dp*.

SCOPE

EXAMPLE

SEE ALSO

`itoa`

`dtoi`

NAME

itof

SYNOPSIS

```
None      itof(  
          Int      i,  
          Pointer  fp)
```

DESCRIPTION

Convert integer *i* into a `float` and store the result in the `float` variable pointed to by *fp*.

SCOPE

EXAMPLE

SEE ALSO

`ctof`

NAME

LowerWidget

SYNOPSIS

```
None          LowerWidget(  
                Widget    w)
```

DESCRIPTION

Call XLowerWindow on *w*'s window.

SCOPE

EXAMPLE

SEE ALSO

RaiseWidget

Hide

NAME

ManageChild

SYNOPSIS

```
None ManageChild(  
    widget    child)
```

DESCRIPTION

Add widget *child* to its parent's list of managed children.

SCOPE**EXAMPLE****SEE ALSO**

RaiseWidget

Hide

NAME

not_equal

SYNOPSIS

```
Boolean    not_equal(  
            String    s1,  
            String    s2)
```

DESCRIPTION

Returns `true` if strings `s1` and `s2` are different.

SCOPE

FACE scripts application

EXAMPLE

Print the string returned by the callbacks of a message box if it is not Cancel.

```
str=PopupAndWait  
(parent.parent.XmDialogShell0.XmMessageBox0);  
if (not_equal(str,"Cancel"))  
printf("%s\n",str);
```

SEE ALSO

`strcmp`

NAME

printf

SYNOPSIS

```
None      printf(  
None      f  
... )
```

DESCRIPTION

Call the C function printf. No type checking is done.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

RaiseWidget

SYNOPSIS

```
None      RaiseWidget(  
          widget      w)
```

DESCRIPTION

Call `XRaiseWindow` on *w*'s X Window.

SCOPE

FACE scripts application

EXAMPLE

Find a widget called `List0` starting from the top of the interface hierarchy and raise it:

```
w = XtNameToWidget(root, "*List0");  
RaiseWidget(w);
```

SEE ALSO

`FmLowerWidget`

`FmShow`

NAME

random

SYNOPSIS

```
Int random (  
    Int  num)
```

DESCRIPTION

Return a random integer whose value lies between 0 and *num*.

SCOPE

FACE scripts only

EXAMPLE

In a pushbutton, set the `labelString` to a random value between 0 and 300, as long as the button is armed. Get a new random number every half second.

```
armCallback = global t;  
global i;  
t = init_timer(self, "global i; i=random(300);  
self:labelString=itoa(i);",0,500,True); disarmCallback =  
global t;  
stop_timer(t);
```

SEE ALSO

NAME

ResizeWidget

SYNOPSIS

```
None ResizeWidget (  
    Widget          w,  
    Dimension       width,  
    Dimension       height,  
    Dimension       border_width)
```

DESCRIPTION

Change the widget's width, height, and border width.

SCOPE

EXAMPLE

SEE ALSO

NAME

SetValues

SYNOPSIS

```
Int SetValues(  
    Widget    w,  
    Int       num_resources,  
    String    resource1,  
    None      value1,  
    String    resource2,  
    None      value2,  
    ...)
```

DESCRIPTION

Set multiple resource values in one call to `XtSetValues`.

num_resources specifies the number of resources to be set. A maximum of 5 resources can be set simultaneously.

resource1 is the name of the first resource.

value1 is the value to which it will be set.

The values must be given in their internal form, not in their String form. The internal form of a resource can be obtained by using `ConvertString`.

SCOPE

EXAMPLE

Set the resources for the scale widget in a float resource editing box:

```
SetValues(parent.scale, 3,  
    "minimum", cftoi(min * 100.0),  
    "maximum", cftoi(max * 100.0),  
    "value", cftoi(v * 100.0));
```

SEE ALSO

`ConvertString`

`XtSetValue`

NAME

printf

SYNOPSIS

```
None printf(  
    None    targ,  
    None    f,  
    ...)
```

DESCRIPTION

Call the C function printf. No type checking is done.

SCOPE

EXAMPLE

SEE ALSO

NAME

sscanf

SYNOPSIS

```
None      sscanf(  
None      src,  
None      format,  
... )
```

DESCRIPTION

Call the C function `sscanf`. No type checking is done.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

strcmp

SYNOPSIS

```
Int  strcmp(  
    String    s1,  
    string    s2)
```

DESCRIPTION

Call the C function strcmp.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

strcpy

SYNOPSIS

```
None strcpy(  
    String    targ,  
    String    src)
```

DESCRIPTION

Call the C function strcpy.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

strlen

SYNOPSIS

```
Int  strlen(  
    String  src)
```

DESCRIPTION

Call the C function `strlen`.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

strncmp

SYNOPSIS

```
Int strncmp (  
    String    s1,  
    String    s2,  
    Int       i)
```

DESCRIPTION

Call the C function `strncmp`.

SCOPE

FACE scripts application

EXAMPLE

SEE ALSO

NAME

WidgetName

SYNOPSIS

```
String    WidgetName (  
    widget    w)
```

DESCRIPTION

Return the name of widget *w*. The string is returned in a static buffer and should not be freed.

SCOPE

EXAMPLE

SEE ALSO

APPENDIX D: The Wx Functions

The following is a list of the Wx functions in alphabetical order.

FUNCTION NAME**XnslWixAlias****COMMAND NAME****expand_alias****SYNOPSIS**

```
Int XnslWixAlias (  
    widget    w,  
    Int       i,  
    Int       c)
```

DESCRIPTION

This function inserts the keyword corresponding to the prefix entered. If no keyword exists, the function makes a backward search of the buffer for a word beginning with the prefix, and inserts it. If no such word exists, the character *c* is inserted. The second argument is ignored. The function returns 0 if the buffer is read-only.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixBackwardChar****COMMAND NAME****backward_char****SYNOPSIS**

```
Int  XnslWixBackwardChar (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function moves the insertion point backward *i* characters. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixBackwardLine****COMMAND NAME****backward-line****SYNOPSIS**

```
Int  XnslBackwardLine (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function moves the insertion point up *i* lines. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixBackwardPage****COMMAND NAME****backward-page****SYNOPSIS**

```
Int  XnslBackwardPage (
    Widget      w,
    Int         i)
```

DESCRIPTION

This function moves the insertion point up *i* screenfuls. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixBackwardWord****COMMAND NAME****backward-word****SYNOPSIS**

```
Int  XnslWixBackwardWord (
    widget  w,
    Int      i)
```

DESCRIPTION

This function moves the insertion point backward *i* words. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixBeginningOfBuffer****COMMAND NAME****beginning-of-buffer****SYNOPSIS**

```
Int  XnslWixBeginningOfBuffer (  
    Widget    w)
```

DESCRIPTION

This function moves the insertion point to the beginning of the buffer.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixBeginningOfLine****COMMAND NAME****beginning-of-line****SYNOPSIS**

```
Int  XnslWixBeginningOfLine (  
    widget    w)
```

DESCRIPTION

This function moves the insertion point to the beginning of the current line.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixCapitalizeWord****COMMAND NAME****capitalize-word****SYNOPSIS**

```
Int  XnslWixCapitalizeWord (  
    Widget    w)
```

DESCRIPTION

This function switches the case of the first character of the current word. It returns 0 if the buffer is read-only.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixCopyBuffer****SYNOPSIS**

```
String    XnslWixCopyBuffer (  
          Widget    w)  
          String    s2,  
          Int       i)
```

DESCRIPTION

This function returns the contents of the buffer. The string is allocated by the widget.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixCopyEndLine****COMMAND NAME****copy-end-line****SYNOPSIS**

```
Int  XnslWixCopyEndLine (  
    Widget    w)
```

DESCRIPTION

This function copies the end of the current line to the kill-stack. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixCopyRegion****COMMAND NAME****copy-region****SYNOPSIS**

```
Int  XnslWixCopyRegion (  
    Widget    w)
```

DESCRIPTION

This function copies the region (the text between the mark and the insertion point) to the kill-stack. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixCreateMark****SYNOPSIS**

```
Int   XnslWixCreateMark(  
      Widget      w,  
      None        n)
```

DESCRIPTION

This function creates a mark at the insertion point and gives it the value *n*.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixCurlineTop****COMMAND NAME****curline-top****SYNOPSIS**

```
Int  XnslWixCurlineTop (  
    widget    w)
```

DESCRIPTION

This function positions the current line at the top of the window. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixDeleteChar****COMMAND NAME****delete-char****SYNOPSIS**

```
Int  XnslWixDeleteChar (  
    Widget      w,  
    Int         i)
```

DESCRIPTION

This function deletes *i* characters after the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnsLWixDeleteEndLine****COMMAND NAME****delete-end-line****SYNOPSIS**

```
Int  XnsLWixDeleteEndLine (
    Widget  w,
    Int     i)
```

DESCRIPTION

This function deletes text from the insertion point to the end of the line and pushes it onto the kill-stack. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixDeleteLine****COMMAND NAME****delete-line****SYNOPSIS**

```
Int  XnslWixDeleteLine (  
    Widget  w,  
    Int     i)
```

DESCRIPTION

This function deletes *i* lines and pushes them onto the kill-stack. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME

XnslWixDeleteMark

COMMAND NAME

SYNOPSIS

```
Boolean   XnslWixDeleteMark (  
          Widget      w,  
          None        n)
```

DESCRIPTION

This function destroys the mark whose value is *n*. If the mark exists the function returns True; if not, False.

SCOPE

EXAMPLE

SEE ALSO

FUNCTION NAME**XnslWixDeleteWord****COMMAND NAME****delete-word****SYNOPSIS**

```
Int  XnslWixDeleteword (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function removes *i* words forward of the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixEndOfBuffer****COMMAND NAME****end-of-buffer****SYNOPSIS**

```
Int  XnslWixEndOfBuffer (
    Widget  w)
```

DESCRIPTION

This function moves the insertion point to the end of the buffer. It returns 0 if the insertion point is already at the end of the buffer.

*

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixEndOfLine****COMMAND NAME****end-of-line****SYNOPSIS**

```
None XnslWixEndOfLine (  
    Widget      w)
```

DESCRIPTION

This function moves the insertion point to the end of the current line.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME	COMMAND NAME
XnslWixExchangePointMark	exchange-point-mark

SYNOPSIS

```
None XnslWixExchangePointMark (  
    widget      w)
```

DESCRIPTION

This function exchanges the positions of the insertion point and the mark.

SCOPE

EXAMPLE

SEE ALSO

FUNCTION NAME**XnslWixFindClosingBrace****COMMAND NAME****match-closing-brace****SYNOPSIS**

```
Int XnslWixFindClosingBrace (  
    Widget    w)
```

DESCRIPTION

This function moves the insertion point to the closing brace that matches the next opening brace after the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixFindClosingParenthesis****COMMAND NAME****match-closing-par****SYNOPSIS**

```
Int XnslWixFindClosingParenthesis (  
    widget    w)
```

DESCRIPTION

This function moves the insertion point to the closing parenthesis that matches the next opening parenthesis after the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixFindOpeningBrace****COMMAND NAME****match-opening-brace****SYNOPSIS**

```
Int XnslWixFindOpeningBrace (  
    Widget    w)
```

DESCRIPTION

This function moves the insertion point to the opening brace that matches the last closing brace before the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixFindOpeningParenthesis****COMMAND NAME****match-opening-par****SYNOPSIS**

```
Int XnslWixFindOpeningParenthesis (  
    widget    w)
```

DESCRIPTION

This function moves the insertion point to the opening parenthesis that matches the last closing parenthesis before the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixFlasher****COMMAND NAME****flasher****SYNOPSIS**

```
Int  XnslWixFlasher (  
    Widget    w,  
    Int       i,  
    Int       c)
```

DESCRIPTION

This function inserts the character *c*, *i* times. If *c* is in *)*, *}*, or *]*, the cursor moves briefly to the corresponding opening *(*, *{*, or *[*. If there is no opener, the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixForwardChar****COMMAND NAME****forward-char****SYNOPSIS**

```
Int  XnslWixForwardChar (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function moves the insertion point forward *i* characters. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixForwardLine****COMMAND NAME****forward-line****SYNOPSIS**

```
Int XnslWixForwardLine (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function moves the insertion point down *i* lines. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixForwardPage****COMMAND NAME****forward-page****SYNOPSIS**

```
Int  XnslWixForwardPage (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function moves the insertion point down *i* screenfuls. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixForwardWord****COMMAND NAME****forward-word****SYNOPSIS**

```
Int  XnslWixForwardWord (
    Widget    w,
    Int       i)
```

DESCRIPTION

This function moves the insertion point forward *i* words. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixGetCurChar****SYNOPSIS**

```
Int  XnslWixGetCurChar (  
    Widget    w)
```

DESCRIPTION

This function returns the code of the character at the insertion point as an Int. If the buffer is empty the function returns -1.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixGetInsertPosition****SYNOPSIS**

```
Int XnslWixGetInsertPosition (  
    Widget      w)
```

DESCRIPTION

This function returns the position of the insertion point.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixGetLastPosition****SYNOPSIS**

```
Int  XnslWixGetLastPosition (  
    Widget    w)
```

DESCRIPTION

This function returns the position of the last character in the buffer.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixGetRegion****SYNOPSIS**

```
String    XnslWixGetRegion (  
Widget    w)
```

DESCRIPTION

This function returns the contents of the region in a null-terminated string. This string is allocated. It must be freed after use. If the region is empty this function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixGotoLine****COMMAND NAME****goto-line****SYNOPSIS**

```
None XnslWixGotoLine (  
    widget    w,  
    Int       i)
```

DESCRIPTION

This function moves the insertion point to the beginning of the line *i* of the buffer.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME

COMMAND NAME

XnslWixGotoMark

SYNOPSIS

Boolean XnslWixGotoMark (
 Widget *w*,
 None *n*)

DESCRIPTION

This function moves the cursor to the mark associated with the value *n*. If the mark exists the function returns True; if not, False.

SCOPE

EXAMPLE

SEE ALSO

FUNCTION NAME**COMMAND NAME****XnslWixInsert****SYNOPSIS**

```
Int  XnslWixInsert(  
    Widget    w,  
    String    s)
```

DESCRIPTION

This function inserts the string *s* at the insertion point.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME

XnslWixInsertAccent

COMMAND NAME

insert-accent

SYNOPSIS

```
Int  XnslWixInsertAccent (
    Widget      w,
    Int          i,
    Int          c)
```

DESCRIPTION

This function translates the character before the insertion point to a special character according to ISO 8859-1 norm, and inserts it *i* times. The preceding character must be one of the following:

A a E e I i O o U u Y y C c N n s.

The *c* character must be one of the following:

' ' ^ / ".

If no translation is possible the *C* character is inserted *i* times. It returns 1 on success.

SCOPE

EXAMPLE

SEE ALSO

FUNCTION NAME**XnslWixInsertChar****COMMAND NAME****insert-char****SYNOPSIS**

```
Int  XnslWixInsertChar (  
    Widget    w,  
    Insert    i,  
    Insert    c)
```

DESCRIPTION

This function inserts the character *c*, *i* times. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixLowerCaseWord****COMMAND NAME****lower-case-word****SYNOPSIS**

```
Int  XnslWixLowerCaseWord (
    Widget      w,
    Int         i)
```

DESCRIPTION

This function changes all characters between the insertion point and the end of the word to lower case. It returns 0 if the buffer is read-only.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixNewLine****COMMAND NAME****new-line****SYNOPSIS**

```
Int  XnslWixNewLine (  
    widget    w,  
    Int       i)
```

DESCRIPTION

This function inserts *i* line-feed characters at the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixNewLineIndent****COMMAND NAME****new-line-indent****SYNOPSIS**

```
Int  XnslWixNewLineIndent (
    Widget      w,
    Int         i)
```

DESCRIPTION

This function inserts *i* line-feed characters and positions the insertion point at the current indentation level on the next line. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixNewLineIndentC****COMMAND NAME****new-line-indent-c****SYNOPSIS**

```
Int  XnslWixNewLineIndentC (  
    Widget  w,  
    Int     i)
```

DESCRIPTION

This function inserts *i* line-feed characters and positions the insertion point at the current indentation level on the next line in the Galactic-C mode. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixOpenLine****COMMAND NAME****open-line****SYNOPSIS**

```
Int  XnslWixOpenLine (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function opens *i* empty lines above the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixOpenSpace****COMMAND NAME****open-space****SYNOPSIS**

```
Int  XnslWixOpenSpace (
    Widget  w,
    Int     i)
```

DESCRIPTION

This function opens *i* empty lines forward of the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixRedraw****SYNOPSIS**

```
Int  XnslWixRedraw (  
    Widget    w)
```

DESCRIPTION

This function displays the modified parts of the widget.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixRefresh****SYNOPSIS**

```
None XnslWixRefresh (  
    widget    w)
```

DESCRIPTION

This function displays the entire contents of the widget.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixReplace****SYNOPSIS**

```
Int  XnslWixReplace (  
    Widget    w,  
    Int       f,  
    Int       t,  
    String    s,  
    Int       l)
```

DESCRIPTION

This function destroys the characters between the positions *f* and *t* and inserts *l* characters of the string *s*. If *t* is negative, all characters between *f* and the end of the buffer are destroyed. If *l* is negative, the entire string *s* is inserted.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixRuboutChar****COMMAND NAME****rubout-char****SYNOPSIS**

```
Int  XnslWixRuboutChar (
    Widget  w,
    Int     i)
```

DESCRIPTION

This function deletes *i* characters preceding the insertion point. If *i* is greater than 1 the characters are pushed onto the kill-stack. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixRuboutWord****COMMAND NAME****rubout-word****SYNOPSIS**

```
Int  XnslWixRuboutWord (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function deletes *i* preceding words. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixScrollDown****COMMAND NAME****scroll-down****SYNOPSIS**

```
Int  XnslWixScrollDown (  
    widget    w,  
    Int       i)
```

DESCRIPTION

This function scrolls the current window down *i* lines. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixScrollLeft****COMMAND NAME****scroll-left****SYNOPSIS**

```
None XnslWixScrollLeft  
Widget w)
```

DESCRIPTION

This function scrolls the current window left the distance of one tabulation.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixScrollRight****COMMAND NAME****scroll-right****SYNOPSIS**

```
Int XnslWixScrollRight (  
    Widget    w)
```

DESCRIPTION

This function scrolls the current window right the distance of one tabulation.
It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixScrollUp****COMMAND NAME****scroll-up****SYNOPSIS**

```
Int  XnslWixScrollUp (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function scrolls the current window up *i* lines. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWixSetInsertPosition****SYNOPSIS**

```
Int  XnslWixSetInsertPosition (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function positions the insertion point at the *i* character from the beginning of the buffer.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixSetMark****COMMAND NAME****none****SYNOPSIS**

```
Boolean  XnslWixSetMark (  
Widget   w,  
None     n)
```

DESCRIPTION

This function positions the mark associated with the value *n*, at the position of the insertion point. If the mark exists the function returns True; if not, False.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME

XnslWixSetTheMark

COMMAND NAME

set-mark

SYNOPSIS

None XnslWixSetTheMark (
 widget *w*)

DESCRIPTION

This function sets the mark at the current insertion point.

SCOPE

EXAMPLE

SEE ALSO

FUNCTION NAME**XnslWixTabulation****COMMAND NAME****tabulation****SYNOPSIS**

```
Int XnslWixTabulation (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function inserts *i* tab characters. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixTwiddleChar****COMMAND NAME****twiddle-chars****SYNOPSIS**

```
Int XnslWixTwiddleChar (  
    Widget    w)
```

DESCRIPTION

This function swaps the two characters preceding the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixUpperCaseWord****COMMAND NAME****upper-case-word****SYNOPSIS**

```
Int  XnslWixUpperCaseWord (
    Widget      w,
    Int         i)
```

DESCRIPTION

This function changes all characters between the insertion point and the end of the word to upper case. It returns 0 if the buffer is read-only.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixWipeRegion****COMMAND NAME****wipe-region****SYNOPSIS**

```
Int XnslWixWipeRegion (  
    widget    w)
```

DESCRIPTION

This function deletes the region (the text between the mark and the insertion point) and copies it to the kill-stack. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWixYank****COMMAND NAME****yank****SYNOPSIS**

```
Int  XnslWixYank (  
    Widget    w,  
    Int       i)
```

DESCRIPTION

This function inserts the *i*th killed item at the insertion point. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxBackwardSearch****COMMAND NAME****backward-search****SYNOPSIS**

```
Int  XnslWxBackwardSearch (
    Widget    w,
    Int        i,
    String     s)
```

DESCRIPTION

This function searches backward from the insertion point for the *i*th occurrence of the search string *S*. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxCIBackwardSearch****COMMAND NAME****backward-ci-search****SYNOPSIS**

```
Int  XnslWxCIBackwardSearch (  
    Widget    w,  
    Int       i,  
    String    s)
```

DESCRIPTION

This function searches backward from the insertion point for the *i*th occurrence of the character string *s* regardless of upper and lower case. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxCIForwardSearch****COMMAND NAME****forward-ci-search****SYNOPSIS**

```
Int  XnslWxCIForwardSearch (
    Widget    w,
    Int        i,
    string     s)
```

DESCRIPTION

This function searches forward from the insertion point for the *i*th occurrence of the character string *S* regardless of upper and lower case. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxClearBuffer****COMMAND NAME****clear-buffer****SYNOPSIS**

```
Int  XnslWxClearBuffer (  
    widget      w)
```

DESCRIPTION

This function removes the contents of the buffer. If the buffer is modified the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME

XnslWxCreateWindow

COMMAND NAME

create-window

SYNOPSIS

Widget XnslWxCreateWindow (
 Widget *w*)

DESCRIPTION

This function creates a new editing window. It returns the XnslWix Widget associated with the new window.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxDeleteWindow****COMMAND NAME****delete-window****SYNOPSIS**

```
Int  XnslWxDeletewindow (  
    Widget    w)
```

DESCRIPTION

This function deletes the Wx window associated to the Widget w. If the buffer is modified, the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxForwardSearch****COMMAND NAME****forward-search****SYNOPSIS**

```
Int  XnslWxForwardSearch (  
    Widget    w,  
    Int       i,  
    String    s)
```

DESCRIPTION

This function searches forward from the insertion point for the *i*th occurrence of the character string *s*. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxInsertFile****COMMAND NAME****insert-file****SYNOPSIS**

```
Int  XnslWxInsertFile (  
    Widget    w,  
    Int       i,  
    String    s)
```

DESCRIPTION

This function inserts the contents of the file *s*. The second parameter is ignored. If an error occurs while the file is being read, or during the insertion of text in the widget, the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxLoadFile****COMMAND NAME****load-file****SYNOPSIS**

```
Int  XnslWxLoadFile (  
    Widget    w,  
    Int       i,  
    String    s)
```

DESCRIPTION

This function loads the contents of the file *s*. The second parameter is ignored. If an error occurs while the file is being read, or during the insertion of text in the widget, the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**COMMAND NAME****XnslWxNameToBuffer****SYNOPSIS**

```
Widget    XnslWxNameToBuffer (
          Int      i)
```

DESCRIPTION

This function returns a pointer on the widget at the *i* position in a circular list of buffers.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxNextWindow****COMMAND NAME****next-window****SYNOPSIS**

```
Widget      XnslWxNextWindow (  
            Widget      w)
```

DESCRIPTION

This function returns a XnslWix Widget associated to the next Wx window in a circular list. It also raises the Wx window and moves the mouse cursor to the middle of this window.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxPopKillStack****COMMAND NAME****pop-kill-stack****SYNOPSIS**

```
Int  XnslWxPopKillStack (  
    Widget    w)
```

DESCRIPTION

This function removes the top item of the kill-stack.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxPreviousWindow****COMMAND NAME****previous-window****SYNOPSIS**

```
Widget    XnslWxPreviousWindow (  
          Widget    w)
```

DESCRIPTION

This function returns a XnslWix Widget associated to the previous Wx window in a circular list. It also raises the Wx window and moves the mouse cursor to the middle of this window.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxReBackwardSearch****COMMAND NAME****backward-regexp-search****SYNOPSIS**

```
Int  XnslWxReBackwardSearch (  
    Widget    w,  
    Int       i,  
    String    s)
```

DESCRIPTION

This function searches backward from the insertion point for the *i*th occurrence of the regular expression *s*. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxReForwardSearch****COMMAND NAME****forward-regexp-search****SYNOPSIS**

```
Int  XnslWxReForwardSearch (
    Widget    w,
    Int        i,
    String     s)
```

DESCRIPTION

This function searches forward from the insertion point for the *i*th occurrence of the regular expression *s*. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxReLoadFile****COMMAND NAME****reload-file****SYNOPSIS**

```
Int XnslWxReLoadFile (  
    Widget    w)
```

DESCRIPTION

This reloads the file associated to the Wx window and goes to the previous line. It returns 1 on success.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxSaveFile****COMMAND NAME****save-file****SYNOPSIS**

```
Int XnslWxSaveFile (  
    widget    w)
```

DESCRIPTION

This function saves the contents of the buffer in the file associated with the Wx window. If an error occurs while the file is being written, the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

FUNCTION NAME**XnslWxWriteFile****COMMAND NAME****write-file****SYNOPSIS**

```
Int XnslWxWriteFile (  
    Widget    w,  
    Int       i,  
    String    s)
```

DESCRIPTION

This function writes the contents of the buffer in the file *s*. The second parameter is ignored. If an error occurs while the file is being written, the function returns 0.

SCOPE**EXAMPLE****SEE ALSO**

APPENDIX E: Undo Grouped Commands

If any of the commands listed below is executed several times consecutively, it is placed in the undo stack with the number of times it was executed. Issuing the undo command undoes the command the number of times it was executed, thus treating *n* identical commands as one.

- backward_char
- backward_line
- backward_page
- backward_word
- beginning_of_buffer
- beginning_of_line
- curline_top
- delete_char
- delete_end_line
- delete_line
- delete_word
- end_of_buffer
- end_of_line
- forward_char
- forward_line
- forward_page
- forward_word
- insert_char
- insert_octal
- insert_accent
- lower_case_word
- open_line
- open_space
- rubout_char
- rubout_word

-
- scroll_down
 - scroll_left
 - scroll_right
 - scroll_up
 - set_mark
 - upper_case_word
 - wipe_region
 - scroll_window
 - select_line, select_word, selection_start, extend_selection, extend_adjust

The commands in the last line are grouped, regardless of the text segment

APPENDIX F: Glossary

. . See dotdot.

.wxrc The startup file for **Wx**. Each user has a copy of **.wxrc** in his or her home directory, and can change it to customize most aspects of **Wx**.

abbreviation An alias that consists of the initial characters of an expression.

abort To abort a dialog box is to cancel it. You do this by typing **^G**.

alias A short sequence of characters which expands to a longer text sequence. (See abbreviation).

ASCII (American Standard Code for Information Interchange) A standard for data transmission which defines a 7-bit code for each character in an Ascii character set.

automatic completion Enabled only if the **expand** resource is true, a way of automatically completing a file name in a file selector box: for instance, if you want to load a file named Myfile, you can type the letters My. If there is no other file in the current directory whose name begins with these letters, the name will be automatically completed. If there are several such files, the names will be displayed in a list.

binding A sequence of keys typed to issue a command. For instance, to go to the beginning of the buffer, you type **Esc >**. Also called a keyboard shortcut or key.

boolean option An option with only two choices: TRUE and FALSE.

Bourne shell One of the commonly used UNIX shells. See shell.

buffer An area of working memory set aside for storing what you type in a window. Each **Wx** window has an associated buffer.

current directory The directory **Wx** will load from and save to by default. This is not necessarily the home directory, since you can change the current directory from inside **Wx**. See home directory.

current paragraph The paragraph the cursor is in.

current sentence The sentence the cursor is in.

current window The window the cursor is in.

current word The word the cursor is on.

default values Values assumed by **Wx**.

deleting See killer.

directory In UNIX, a node of the hierarchical file system where files are stored.

See home directory, current directory.

dotdot The dotdot symbol (..) is a standard UNIX pseudonym for the parent directory, i. e. , the directory immediately above the current directory. By clicking on .. in a **Wx** file selector window, you can change the current directory to its parent. This gives you a way to climb up the directory tree structure.

expansion See **file name expansion**.

file name expansion Enabled only if the **expand** resource is true. When you type a few letters in an edit box, then press **Tab**, if there is only one filename beginning with those letters, **Wx** will expand them to the full name; if there are several, **Wx** will list the choices so that you can select the name you want.

flashing In Language Modes, when you type a closing parenthesis or brace, the corresponding opening separator, if there is one, will flash. This provides a convenient way of checking whether separators are correctly matched.

Fundamental Mode The basic **Wx** Mode, the default Mode when you enter **Wx** and no file name is specified.

home directory In UNIX, each user has a home directory, which is normally the one you enter when you log in. Certain important files are stored there, such as the configuration files indicating a user's personal preferences. See **.wxrc**, current directory.

indentation The number of blanks by which text is separated from the beginning of a line. The indent size is normally specified as a constant in the startup file. In Language Modes, the level of indentation is kept track of, and when the Return key is pressed the new line will be aligned with the current level. This level is incremented when a new block is begun.

insertion point The point at which text is inserted when you type. It is represented as a rectangle, or an I beam, depending on the cursor specified. You can change the insertion point by clicking the Select button elsewhere in the window.

key A sequence of keys typed to issue a command. For instance, to go to the beginning of the buffer, you type **Esc >**. Also called a key binding or keyboard shortcut.

kill stack A structure (a push-down stack) in which text regions that you cut or copy using a killer command are automatically stored in succession, and can be recovered by the yank or paste command.

killer A cut or copy command that places the text cut or copied on the kill stack. Most cut and copy commands are killers, but some deletion commands are non-killers, i. e. when they are used to delete a region, that region is not placed on the kill stack.

language mode A **Wx** mode designed to facilitate editing of programs in a particular language, with expansion of keywords, automatic indentation, parenthesis flashing, etc.

level of indentation A language mode keeps track of the level of indentation, increasing it by one each time you begin a new block.

linefeed One of two ASCII characters (the other is CR) indicating the beginning of a new line (also called the newline NL character).

macro A memorized sequence of commands associated with a single key binding.

mark The mark indicates one boundary of a text region, the other end being identified by the position of the cursor. Before you use a cutting or copying command you must indicate the text region to which it applies, and this may be done by setting the mark.

menu A graphic display containing a list of commands. You can click on a command name, or move the selection focus to a command on an open menu, using the arrow keys, then press the Return key to issue the command.

menu bar The line that appears at the top of a **Wx** window containing the names of all **Wx** menus.

metacommand A command that affects other commands. For example, the abort command (**^G**) cancels a dialog box, and thus aborts the command that opened it; the repeat command (**Esc num**) is prefixed to another command to indicate how many times it is to be executed.

mini buffer A text input field which replaces dialog boxes for data input and message output when commands are triggered using accelerators. When enabled and input is expected, it is displayed at the bottom of the current wx window. The mini buffer is disabled by default.

Mode Each **Wx** window is in some Mode, and Modes differ in the capabilities they

offer: for instance, in C++ Language Mode certain aliases facilitate writing C++ programs, there is automatic flashing of matching parentheses, etc.

parenthesis flashing See **flashing**.

push-down stack See **kill stack**.

read-only window This is a window that allows you to perform any **Wx** command in its buffer, as long as that command does not modify the buffer.

region A continuous segment of text that is operated on by a killer command (i. e. a command that does cutting or copying).

regular expression A method of specifying a text pattern to be used in searching for words of a given type, or a replacement. For instance, `[a-z][0-9]` matches any string consisting of a lower case letter followed by a digit.

scroll bar The bar you see along the left edge or bottom of a **Wx** window. By moving the slider, you can change the portion of the text buffer visible in the window.

shell The UNIX command interpreter.

slider The rectangle you move in the scroll bar to scroll the contents of a text window.

startup file The file **.wxrc**, which is read when you launch **Wx**. It specifies various configuration options. See **.wxrc**.

window A graphic view on a buffer.

yank Another name for the Paste command.

Symbols

.. 25

.. (dotdot) 25

.wx-macros 45

.wxrc 104

 configuring 104

 creating 93

 modes in 65

.Xdefaults xi, 93, 107

/usr/lib/X11/system.wxrc 93, 104

A

-ab 103

abbreviation 65

aborting dialog box 23, 53

Ada 19, 44, 65

AddGrab() 119

alias 68

 definition 68

 keyword 68

Any() 117

ASCII characters

 display ASCII 12

 display non-ASCII 12

assignment 82

atoi() 120

auto 74

automatic backup 41

automatic completion 98

automaticBackUp 13, 41, 99

 argument 103

 resource 99

B

backward 17

backward case insensitive 17

- backward regexp 17
- backward_char 149
- backward-char 109
- backward-ci-search 17, 109, 211
- backward-line 109, 150
- backward-page 109, 151
- backward-regexp-search 17, 109, 223
- backward-search 17, 109, 210
- backward-word 109, 152
- beginning-of-buffer 109, 153
- beginning-of-line 109, 154
- blockCursor
 - resource 96
- buffer
 - compile 46
 - cutting in 31
 - loading text 25
 - moving in 30
 - saving text 41
 - text 29
- button 103
- C
- C 65
- C++ 19, 44, 65
- callback() 121
- CallCallbacks() 119
- capitalize-word 109, 155
- cftoi() 122
- change-directory 15, 26, 71, 109
- citof() 123
- clear-buffer 109, 213
- clickgeometry 102
- clickgeometry 99
 - argument 102

- resource 99
- clicking x
- client area 8
- clipboard 33
- Cobol 19, 44, 65
- command line options 102
- commands 10
 - from keyboard 10
 - from menu 10
 - keyboard-only 52
 - load-tags 53
 - non-menu 52
 - reload-file 54
 - summarized 57
 - UNIX 47
- comments 74
- compile 66
 - keyword 66
- compile buffer
 - diagnostic panel 46
- compile-buffer 21, 46, 109
- compile-command 71
- ConfigureWidget() 124
- configuring Wx 93
 - .wxrc 93
 - command line options 102
 - defining macros 105
 - defining menus 105
 - defining modes 105
 - resources 93
- confirmRemoteFile 13, 99
 - argument 103
 - resource 99
- constants 77

- defining 79
- control key 23
- control structure 82
- ConvertString 81
- copy 31
- copy-end-line 110, 157
- copy-region 16, 110, 158
- create-window 18, 110, 214
- crf 103
- curline-top 110, 160
- current file 25, 41, 43
- cursor 23
 - choice of 12
 - resource 96
- CursorPosition() 117
- custom button x
- cut 31
- D
- define 74, 79
- defmenu 104, 105
- defmode 66, 70, 104
- delete-char 110, 161
- delete-end-line 110, 162
- delete-line 110, 163
- delete-window 15, 110, 215
- delete-word 110, 165
- DestroyWidget() 119
- dialog box 23
- dib 103
- directory 102
- directory 98
 - argument 102
 - changing 26
 - resource 98

- display mode 12
- displayIconBar 99
 - argument 103
 - resource 99
- display-line 110
- displayLinesColumns 99
 - argument 102
 - resource 99
- dlc 102
- dotdot 25
- drag x
- drag-and-drop 5
- dtoi() 125
- E
- Edit menu 10, 16
 - copy 16
 - cut 16
 - paste 16
 - undo 16
- else 74, 82
- end-of-buffer 110, 166
- end-of-line 110, 167
- environment variable
 - NSLHOME 104
 - TMPDIR 104
 - WXTAGS 54
- equal() 126
- error 46, 66
 - keyword 46, 66
- escape key 23
- exchange-point-mark 110, 168
- exec-command 21, 71, 110
- exec-script 111
- execute UNIX command 47

execute-macro 72

exit server 51

-expand 26, 102

expand 98

- argument 102

- resource 25, 98

expand_alias 148

expand-alias 68, 111

extend-adjust 111

extend-end 111

F

FACE 73

- arithmetic expression 74

- assignment 82

- basic objects 75

- comments 74

- control structure 82

- defining constant in 79

- function 83

- function call 82

- function definition 83

- include files 75

- interpreter 73

- keywords 74

- object types 75

- return 84

- setting constants in 77

- special characters 74

- statement 82

- type casters 81

- typecasting functions 117

- using variable in 77, 78

- widget naming scheme 79

file 29

- automatic backup 13, 41
- backup 42
- current 41
- insert 43
- name change 42
- opening 4, 23
- saving 5, 41, 42
- selecting 4
- show/hide window 5
- temporary 104
- File menu 10, 15
 - change directory 15
 - close window 15
 - exit 15
 - insert 15
 - open 15
 - save 15
 - save as 15
- file name expansion 25, 102
- file selector box 24
- flasher 111, 173
- Float() 117
- focus 8
- font
 - resource 96
- for 74, 82
- Fortran 19, 44, 65
- forward 17
- forward case insensitive 17
- forward regexp 17
- forward-char 111, 174
- forward-ci-search 17, 111, 212
- forward-line 111, 175
- forward-page 111, 176

forward-regexp-search 17, 111, 224

forward-search 17, 111, 216

forward-word 111, 177

free() 127

ftoi() 128

function 74, 83

- FACE 119

- typecasting 117

- Wx 147

function call 82

Fundamental 19, 44, 65

G

Galactic-C 19, 44

global 74, 78

goto-line 111, 182

H

Help menu 10, 22

- index 50

hide icon 5

-home 102

home 99

- argument 102

- resource 99

I

icon bar

- in manager box 4

- on Wx window 11

- show/hide 11

icons

- in manager box 3

- on Wx window 11

if 74, 82

Image() 117

include 74, 75

- insert file 43
- insert-accent 112, 185
- insert-char 112, 186
- insert-file 15, 112, 217
- insertion point 8, 23, 30
- insert-octal 112
- Int() 117
- internationalization 107
- ISO 8859-1 108
- ISO keyboard 108
- IsSensitive() 119
- itoa() 129
- itod() 130
- itof() 131
- K
- key combinations 106
- keyboard commands 10
 - deleting 55
 - metacommands 53
 - summary 57
 - symbols 52
- keyboard-action 97
- keytable 68, 69
 - definition 69
 - keyword 68
- keyword
 - alias 68
 - auto 74
 - define 74, 79
 - defmenu 104, 105
 - defmode 104
 - else 74, 82
 - error 46, 66
 - for 74, 82

- function 74, 83
- global 74, 78
- if 74, 82
- in .wxrc file 104
- in FACE 74
- in mode definition 66
- include 74, 75
- keytable 68
- loadmacro 45, 104
- local 74, 77
- parent 74, 79
- print 105
- return 74, 84
- root 74, 79
- script 74
- self 74, 79
- suffix 66
- tabsize 67
- toplevel 74, 79
- translations 67
- type 74
- while 74, 82
- wxkeys 105
- kill stack 32, 33
 - copying from 33
 - removing item in 34
 - viewing 6, 21, 33
- L
- language modes 44
- LaTeX 19, 44, 65
- line numbering 12
- line-counter 112
- lineNumberColor
 - resource 97

- load 73
- load-file 15, 100, 103, 112, 218
- load-macro 54, 71, 93, 112
- loadmacro 104
 - keyword 45, 73
- load-tags 53, 71, 100, 103, 112
- local 74, 77
- lower-case-word 112, 187
- LowerWidget() 132
- M
- machine 102
- machine 99
 - argument 102
 - resource 99
- Macro menu 10, 20
 - begin macro 20
 - edit macro 20
 - end macro 20
 - execute macro 20
 - save macro 20
- macros 71
 - creating 71
 - editing 6, 72
 - executing 72
 - loading 73
 - macro file 73
 - registering 72
 - saving 73
 - syntax error 72
- malloc() 119
- ManageChild() 133
- manager box 1, 3
 - drag and drop 5
 - icons 3

- menus in 6
- opening 3
- managerbox 1, 103
- managerbox 3
- MapWidget() 119
- match-closing-brace 112, 169
- match-closing-par 112, 170
- match-opening-brace 112, 171
- match-opening-par 112, 172
- matchReadOnly 13, 99
 - argument 103
 - resource 99
- menu 103
- menu 99
 - argument 103
 - resource 99
- menu bar
 - in Wx window 10
 - on manager box 6
- menu commands 14, 23
- menus 10
 - defining 105
 - in manager box 6
 - Wx window 14
- MenuWidget() 117
- message zone 14
- metacommands 53
- mini buffer 48
 - argument 103
- Misc menu 10, 21
 - compile buffer 21
 - execute Unix command 21
 - show kill stack 21
 - show/hide icon bar 21

- modes 65
 - adding 44
 - defining 66
 - expansion 44
 - Fundamental 65
 - keywords 66
 - language 44, 65
- Modes menu 10, 19
 - Ada 19
 - C++ 19
 - Cobol 19
 - Fortran 19
 - Fundamental 19
 - Galactic-C 19
 - LaTeX 19
 - Pascal 19
 - Pl1 19
 - PostScript 19
- Motif 81
- Motif Style Guide recommendations xi, 100, 103
- Motif widgets 94
- motifRecommendation 100
 - argument 103
 - resource 98, 100
- motionCursor
 - resource 96
- mouse button x
- MoveWidget() 119
- mr xi, 102, 103
- mro 103
- multiplier 113
- N
- new 18
- new icon 4

new-line 113, 188
new-line-indent 113, 189
new-line-indent-c 113, 190
NewString() 119
next 18
next-window 18, 100, 103, 113, 220
not_equal() 134
NSLHOME 104
NSLHOME/lib/x11/system.wxrc 104
O
octal codes 108
open icon 4
opening files 4
open-line 113, 191
open-space 113, 192
P
P11 65
parent 74, 79
parent directory 25
Pascal 19, 44, 65
paste 32
Pixmap() 117
P11 19, 44
Pointer() 117
Popdown() 119
pop-kill-stack 113, 221
Popup() 119
PostScript 19, 44, 65
previous-window 100, 103, 113, 222
print 105
printf() 135
print-region 105, 113
properties 80

Q

query-replace 17, 71, 113

quit-the-editor 6, 15, 113

R

RaiseWidget() 136

random() 137

readOnly

- resource 96

read-only 11

register-script 113

regular expressions 38

reload-file 54, 114, 225

RemoveGrab() 119

repeat-command 114

replace 17

- resource 96

replace/insert mode 11

ResizeWidget() 138

resource

- blockCursor 96

- cursor 96

- font 96

- lineNumberColor 97

- motionCursor 96

- readOnly 96

- replace 96

- selectedBackground 96

- selectedForeground 96

- showLines 97

- tabsize 96

- translations 97

- undoDepth 97

- wrap 96

- XnslWix widget 95

- resources 93
 - wxd Program 98
- return 74, 84
- root 74, 79
- rubout-char 114, 196
- rubout-word 114, 197
- S
- save icon 5
- save-file 15, 114, 226
- save-macro 20, 114
- saving files 41
- script 71, 72, 74
- Script window 71
- scroll bar 8
- scroll-down 114, 198
- scrolling 30
- scroll-left 114, 199
- scroll-right 114, 200
- scroll-up 114, 201
- scroll-window 114
- search 35
 - and replace 36
 - backward 36
 - by regular expressions 36, 39
 - case insensitive 36
 - forward 36
 - menu 17
 - window 36
- Search menu 10, 17
 - backward 17
 - backward case insensitive 17
 - backward regexp 17
 - forward 17
 - forward case insensitive 17

- forward regexp 17
- replace 17
- select button x
- selectedBackground
 - resource 96
- selectedForeground
 - resource 96
- selecting
 - files 24
 - menu items 10
 - text 31
- selection-start 115
- select-line 114
- select-pane 114
- select-word 114
- self 74, 79
- set-font 18, 115
- SetKeyboardFocus() 119
- set-mark 115, 204
- SetSensitive() 119
- SetValues() 139
- show icon 5
- show-kill-stack 21, 71, 115
- showLines
 - resource 97
- show-script 73, 115
- split 18
- split/unsplit window 9
- split-window 9, 18, 71, 115
- sprintf() 77, 140
- sscanf() 141
- start-macro-learn 20, 115
- statement 82
- stop-macro-learn 20, 115

- strcmp() 142
- strcpy() 143
- String() 117
- strlen() 144
- strncmp() 145
- stuff 115
- suffix 66
 - keyword 66
- T
- tabsize 67
 - keyword 67
 - resource 96
- tabulation 115, 205
- tags 98
- text
 - replace 36
- text buffer 43
- title bar 14
- TMPDIR 104
- toggle-editable 115
- toggle-icon-bar 21, 115
- toggle-insert-replace 11, 115
- toplevel 74, 79
- translations 67
 - keyword 67
 - resource 97
- twiddle-chars 115, 206
- type 74
 - casting 81
 - checking 76, 81
 - conversion 81
 - keyword 81
- type casters 81

U

-umb 103

undo 16, 35, 71, 115

 in macros 35

 with a split window 9

undo commands 229

undoDepth

 resource 97

UNIX commands 47

Unmanagechild() 119

UnmapWidget() 119

unsplit 18

unsplit-window 9, 18, 116

upper-case-word 116, 207

useManagerBox 3, 100

 argument 103

 resource 100

useMiniBuffer 100

V

variables 77

 global 78

 local 77

-verbose 103

verbose 99

 argument 103

 resource 99

W

-warp 103

warpPointer 100

 argument 103

 resource 100

while 74, 82

widget 79, 94

 Motif 94

- name in FACE 79
- name in menu 106
- XnslWix 95
- Widget() 117
- WidgetName() 146
- window
 - close 43
 - cut and paste between 32
 - message zone 14
 - mode name 14
 - modified flag 28
 - opening 2
 - show/hide 5
- Window menu 10, 18
 - new 18
 - next 18
 - set font 18
 - split 18
 - unsplit 18
- Window() 117
- wipe-region 16, 116, 208
- wish 99
 - argument 103
 - resource 99
- wrap 11
 - resource 96
- write-file 15, 116, 227
- Wx clipboard 21
- wxkeys 105
- WXTAGS 54
- X
- X properties 80
- Xm.h 75
- XmBackgroundPixmap() 117

XnslWix
 resources 95
 translations 97
XnslWixAlias() 148
XnslWixBackwardChar() 149
 example 87
XnslWixBackwardLine() 150
 example 87
XnslWixBackwardPage() 151
XnslWixBackwardWord() 152
 example 89
XnslWixBeginningOfBuffer() 153
XnslWixBeginningOfLine() 154
 example 87
XnslWixCapitalizeWord() 155
XnslWixCopyBuffer() 156
XnslWixCopyEndLine() 157
XnslWixCopyRegion() 158
XnslWixCreateMark() 159
 example 87
XnslWixCurlineTop() 160
XnslWixDeleteChar() 161
XnslWixDeleteEndLine() 162
XnslWixDeleteLine() 163
 example 87
XnslWixDeleteMark() 164
 example 87
XnslWixDeleteWord() 165
XnslWixEndOfBuffer() 166
XnslWixEndOfLine() 167
XnslWixExchangePointMark() 168
 example 87
XnslWixFindClosingBrace() 169
XnslWixFindClosingParenthesis() 170

XnslWixFindOpeningBrace() 171
XnslWixFindOpeningParenthesis() 172
XnslWixFlasher() 173
XnslWixForwardChar() 174
XnslWixForwardLine() 175
XnslWixForwardPage() 176
XnslWixForwardWord() 177
 example 89
XnslWixGetCurChar() 178
 example 88
XnslWixGetInsertPosition() 179
XnslWixGetLastPosition() 180
XnslWixGetRegion() 181
 example 89
XnslWixGotoLine() 182
XnslWixGotoMark() 183
 example 87
XnslWixInsert() 184
 example 88
XnslWixInsertAccent() 185
XnslWixInsertChar() 186
XnslWixLowerCaseWord() 187
XnslWixNewLine() 188
XnslWixNewLineIndent() 189
XnslWixNewLineIndentC() 190
XnslWixOpenLine() 191
XnslWixOpenSpace() 192
XnslWixRedraw() 193
 example 87
XnslWixRefresh() 194
 example 88
XnslWixReplace() 195
XnslWixRuboutChar() 196
XnslWixRuboutWord() 197

XnslWixScrollDown() 198
XnslWixScrollLeft() 199
XnslWixScrollRight() 200
XnslWixScrollUp() 201
XnslWixSetInsertPosition() 202
XnslWixSetMark() 203
XnslWixSetTheMark() 204
 example 87
XnslWixTabulation() 205
XnslWixTwiddleChar() 206
XnslWixUpperCaseWord() 207
XnslWixWipeRegion() 208
XnslWixYank() 209
 example 87
XnslWxBackwardSearch() 210
XnslWxCIBackwardSearch() 211
XnslWxCIForwardSearch() 212
XnslWxClearBuffer() 213
XnslWxCreateWindow() 214
XnslWxDeleteWindow() 215
XnslWxForwardSearch() 216
 example 87
XnslWxInsertFile() 217
XnslWxLoadFile() 218
XnslWxNameToBuffer() 219
XnslWxNextWindow() 220
XnslWxPopKillStack() 221
 example 91
XnslWxPreviousWindow() 222
XnslWxReBackwardSearch() 223
XnslWxReForwardSearch() 224
 example 88
XnslWxReLoadFile() 225
XnslWxSaveFile() 226

XnslWxWriteFile() 227

Y

yank 16, 116, 209